REPUBLIC OF YEMEN
UNIVERSITY OF ADEN
FACULTY OF ENGINEERING
DEPARTMENT OF INFORMATION TECHNOLOGY

Optical Mark Recognition Mobile Application
(SMART OMR)

Prepared By:

Bashar Abdo Mohammed Al-Awasi	215011
Muaadh Mohammed Qasem Ali	217025
Omer Abdulbaset Mohammed Hashem	217038

Supervisor : Dr. Adel Sallam

2025-2026

ACKNOWLEDGMENT

First and foremost, We would like to thank Almighty Allah for giving us the strength, patience, and determination to complete this project successfully.

We are deeply grateful to our supervisor **Dr. Adel Sallam**, whose constant guidance, constructive feedback, and encouragement have been a cornerstone throughout the development of this Flutter-based Optical Mark Recognition (OMR) application. His insights have helped us refine the objectives of the project and overcome many challenges.

Our sincere appreciation goes to our teachers and academic department, who have provided us with the technical knowledge and research skills necessary to undertake this work.

We also wish to extend our thank to our colleagues and classmates who shared valuable suggestions, tested different features of the application, and provided practical feedback that improved the performance and usability of the system.

Lastly, We acknowledge all researchers and authors whose previous studies and technical contributions in the field of mobile application development formed a solid foundation upon which this project was built.

This project is the result of combined efforts, guidance, and encouragement from all these individuals and communities. To all of them, we extend our sincerest appreciation and gratitude.

ABSTRACT

This project presents the design and development of a mobile application using Flutter framework that utilizes Optical Mark Recognition (OMR) technology. The main objective of the application is to provide an efficient, accurate, and user-friendly tool for recognizing and processing marked answers in examination sheets, surveys, and multiple-choice forms through mobile devices.

The system allows users to capture or upload images of answer sheets, applies preprocessing techniques to enhance image quality, and then detects shaded marks in predefined areas. The extracted data is processed and converted into meaningful results such as selected answers, scores, or summary reports. The application also offers options to export the results in various formats (e.g., CSV, PDF) for further analysis.

By integrating OMR technology into a cross-platform Flutter application, this project contributes to simplifying the evaluation process, reducing manual errors, and providing a cost-effective digital solution suitable for academic institutions, organizations, and research purposes.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

NO.	Abbreviations	Meaning
1	OMR	Optical Mark Recognition
2	UI	User Interface
3	IDE	Integrated Development Environment
4	CSV	Comma-Separated Values
5	OpenCV	Open Source Computer Vision Library
6	PDF	Portable Document Format
7	OS	Operating System
8	iOS	iPhone Operating System
9	Git	Global information tracker
10	APP	Application
11	GUI	Graphic User Interface
12	BaaS	Backend-as-a-Service
13	SDK	Software Development Kit
14	API	Application Programming Interface
15	HTTP	Hypertext Transfer Protocol
16	CLI	Command Line Interface
17	APK	Android Package Kit
18	RTL	Right-To-Left
19	NoSQL	Not Only SQL

CHAPTER 1: INTRODUCTION

1.1 OVERVIEW

In modern educational ecosystems, assessment systems serve as a primary mechanism for measuring academic comprehension and evaluating instructional efficacy. Among various evaluation formats, multiple-choice questions (MCQs) have emerged as a global standard due to their ability to objectively test a broad spectrum of curriculum material within a compressed time frame. However, the administrative workload associated with grading MCQs remains a significant challenge. Traditionally, educational institutions relied on dedicated, legacy Optical Mark Recognition (OMR) machines. While highly accurate, these hardware-based systems are expensive to procure, require specialized high-density paper sheets that must be printed under strict alignment tolerances, and demand continuous maintenance. This creates a severe accessibility barrier for individual educators, schools with limited budgets, and regional departments, such as those within the University of Aden.

To address these limitations, the transition towards software-defined mobile OMR systems has gained traction, driven by the ubiquity and increasing processing capability of modern smartphones. By utilizing the built-in cameras on mobile devices, software applications can capture images of examination papers and execute digital image processing algorithms to detect shaded marks. However, developing a robust mobile OMR solution requires resolving key technical challenges, including adapting to varying ambient lighting conditions, handling paper orientation and skew, and ensuring low-latency processing on consumer-grade hardware.

This project presents the design and development of an Optical Mark Recognition (OMR)-based exam management system engineered using the Flutter framework. Flutter, a modern cross-platform framework developed by Google, allows the application to run seamlessly across Android, iOS, Web, and desktop environments (Windows, macOS, and Linux) from a single codebase. Rather than relying on a pure cloud-based processing model, which introduces high network latency and data privacy concerns, this application implements a robust offline-first data management architecture. The application employs a local SQLite database (sqflite) as its primary data store, allowing grading, exam management, and statistical analytics to run entirely without internet access. When a network connection is established, the application seamlessly synchronizes local data with Firebase Firestore, providing cross-device continuity and cloud-based backups. By combining on-device computer vision processing,

offline-first persistence, and real-time cloud integration, this system offers a highly secure, cost-effective, and flexible grading platform tailored to the modern educator

1.2 PROBLEM STATEMENT

Traditional methods of exam management and result processing rely heavily on manual evaluation, which is time-consuming, prone to errors, and lacks real-time analytics. Teachers and administrators often face challenges in manually evaluating answer sheets, especially when dealing with large-scale exams. This can lead to delays in result processing, inaccuracies in evaluation, and a lack of actionable insights.

Furthermore, manual processes do not provide the ability to analyze class performance or identify trends in student results. This limits the ability of educators to make data-driven decisions for improving teaching strategies and addressing student weaknesses.

This project addresses these challenges by introducing an automated OMR-based solution. The system enables efficient scanning of answer sheets, instant result processing, and detailed analytics. By automating these processes, the application reduces the workload on educators, minimizes errors, and provides real-time insights into student performance.

1.3 PROJECT OBJECTIVES

The objectives of this project are as follows:

- To develop a cross-platform OMR-based exam management system using Flutter, ensuring compatibility across Android, iOS, Web, Windows, Linux, and macOS.
- To enable efficient scanning and processing of OMR sheets, ensuring accuracy and speed in result evaluation.
- To provide real-time analytics and result visualization, allowing educators to gain insights into student performance and class trends.
- To simplify the process of creating and managing exams, making it accessible for teachers and administrators with minimal technical expertise.
- To create a foundation for future enhancements, such as advanced OMR algorithms, encrypted data storage, and large-scale deployment for educational institutions.

These objectives aim to address the limitations of traditional exam management systems and provide a modern, efficient solution for educators.

1.4 PROJECT SCOPE

The scope of the project includes the development of a Flutter-based application that provides a comprehensive solution for exam management. The application is designed to cater to the needs of small to medium-scale educational institutions, such as schools, coaching centers, and training organizations.

Key Features

- Developing a user-friendly interface for creating exams, scanning OMR sheets, and viewing results.
- Implementing OMR scanning functionality to process answer sheets efficiently and accurately.
- Providing real-time result processing and analytics, enabling educators to make data-driven decisions.
- Supporting multiple platforms, ensuring accessibility across various devices and operating systems.

Limitations

- The current version of the application includes a database for persistent data storage “Firebase” . .
- Advanced features, such as encrypted data storage, face recognition, and anomaly detection, are not yet implemented.
- The system is designed for local deployment and small to medium-scale use cases. Large-scale or cloud deployment would require further enhancements to the system architecture.

CHAPTER 2: PREVIOUS WORKS

2.1 OVERVIEW

Designing and implementing a high-accuracy, real-time Optical Mark Recognition (OMR) system requires a structured development methodology. This process begins with a comprehensive system requirements analysis and concludes with iterative software testing and deployment. In the field of computer vision and mobile document processing, developers routinely encounter technical bottlenecks, such as variable ambient lighting, paper perspective distortion, lens focus issues, and on-device computational constraints.

To minimize these risks, it is essential to analyze the architectural patterns and user interfaces of existing market-leading solutions. By examining these systems, the design team can learn from established workflows, identify technical constraints, and design features that address existing functional gaps. In the domain of automated academic grading, a review of existing applications provides critical insights into optimal layout designs, robust bubble-detection algorithms, and effective data serialization techniques.

2.2 RELATED WORK

During the pre-development phase of the OMR1 project, several commercial grading applications and OMR frameworks were analyzed. This section details the two primary platforms that set the standard for modern mobile-based academic grading: ZipGrade and GradeCam.

2.2.1 ZipGrade

ZipGrade is a widely adopted mobile grading application that transforms standard consumer smartphones and tablets into high-speed Optical Mark Recognition scanners. The system is designed to provide teachers with instant grading capabilities directly in the classroom, facilitating immediate formative feedback for students. The core workflow of ZipGrade is centered around custom-designed, printable bubble sheets.

The system uses specific corner alignment targets (fiducial markers) printed on the boundaries of the page to calibrate the device camera's perspective matrix in real-time. Once these markers are aligned within the camera's viewfinder, as illustrated in Figure 2-1, the application automatically captures the frame, corrects any rotation or tilt, and performs thresholding to determine which circles are shaded.

ZipGrade

FIGURE 2- 1:ZipGrade

To review student performance and analyze class progress, ZipGrade provides a dedicated analytics panel, as shown in Figure 2-2. This interface breaks down the grading results by student, displaying total scores, item responses, and statistical trends.

This figure displays the academic item analysis dashboard within the ZipGrade app. The screen showcases aggregated test statistics, including correct-answer ratios, in the multiple choices, determined if the answer is correct or not as seen in the correct answer that added by the teacher that found in the app be four do the student do the exam and answer the multiple choices, and the ZipGrade do question difficulty metrics, and detailed breakdowns of student response selections

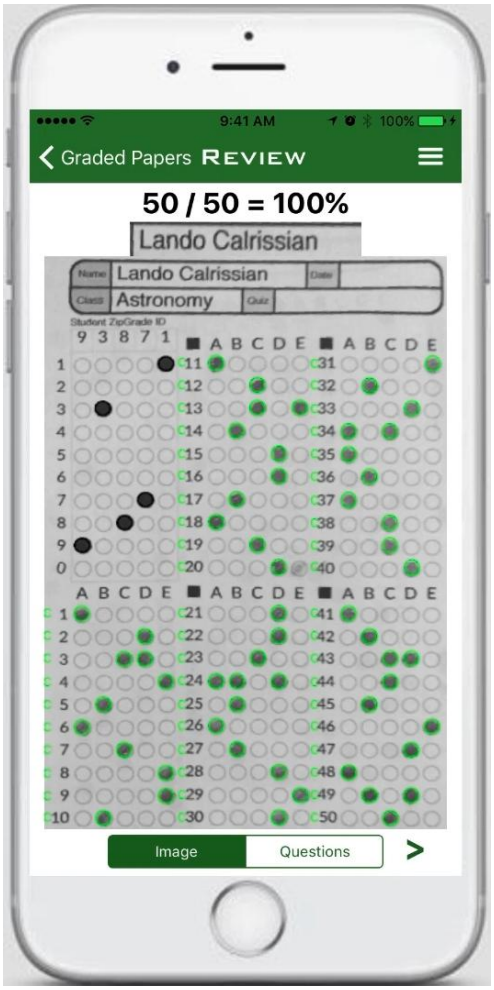

FIGURE 2- 2: ZipGrade Items Review

For showing and distribution, ZipGrade provides standardized sheet templates, typically in 20, 50, or 100-question formats. Figure 2-3 showcases a typical the grade of each student after scanning the student's paper sheet . These sheets feature specific alignment boxes and designated regions for student IDs to

automate roster mapping.

This figure illustrates the layout of a result analysis item screen in ZipGrade and show the answer sheet. The page displays the header input fields,the analysis of the grade of students and what is there score if A,B,C,or F in general that mean how many students have each score,and some other details.

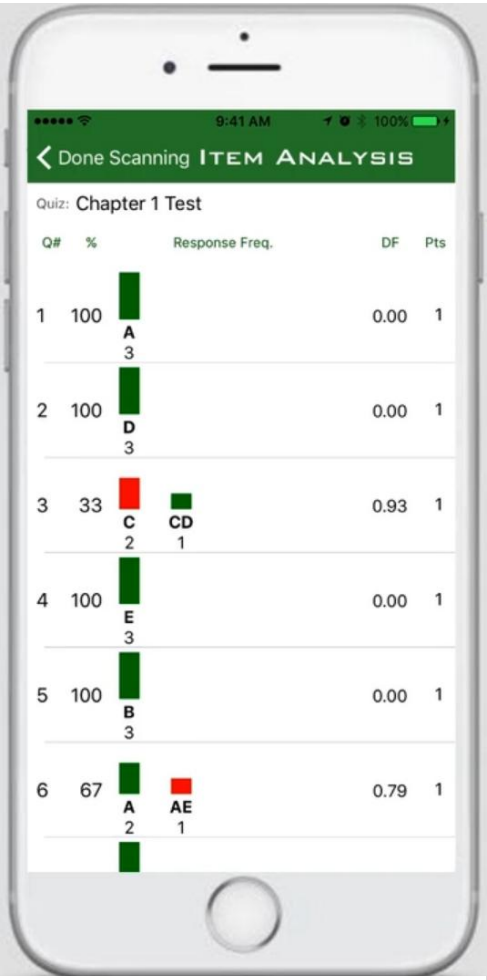

FIGURE 2- 3: ZipGrade test paper

2.2.2 GradeCam

GradeCam is a versatile assessment platform that allows teachers to scan and grade various types of assignments using any camera-enabled device (webcams, mobile phones, or document cameras). Unlike legacy OMR systems, GradeCam works on plain paper, eliminating the need for expensive specialized forms. Beyond simple bubble sheets, GradeCam supports numeric grids and rubric-based scoring.

GRADECAM

FIGURE 2- 4: GradeCam

The administrative dashboard of GradeCam, illustrated in **Figure 2-5**, provides a centralized hub for managing classrooms, scheduling exams, and reviewing graded rosters.

This figure showcases the GradeCam web dashboard interface. The sidebar navigation links to classes, assignments, and reports, while the primary container presents active exam files and student rosters.

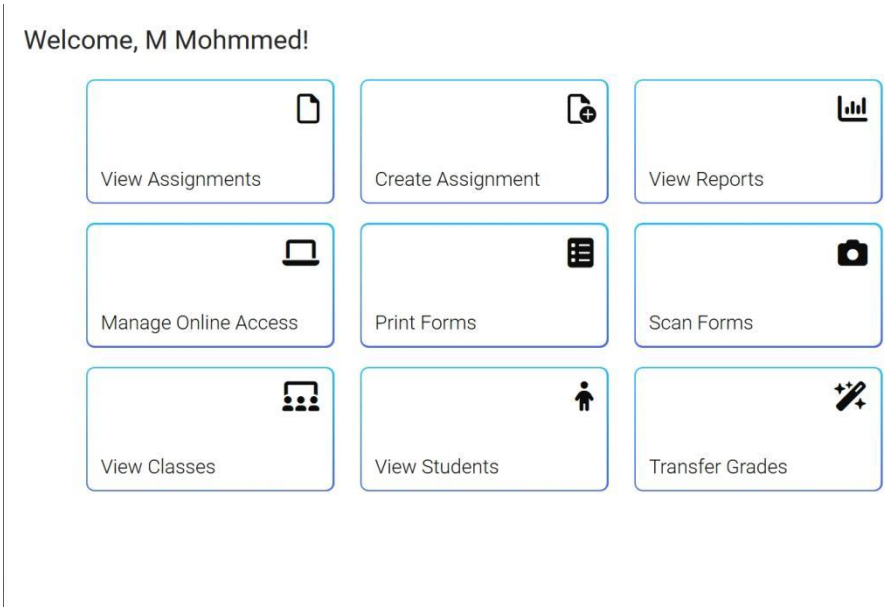

FIGURE 2- 5: GradeCam Home Page

To facilitate exam creation, GradeCam includes an online exam constructor, shown in **Figure 2-6**. Instructors can customize the question count, grid layout, and question formatting before printing.

This figure presents the GradeCam form creation interface, showing options for configuring bubble formats, adding student identification fields, and previewing the print-ready answer sheet layout.

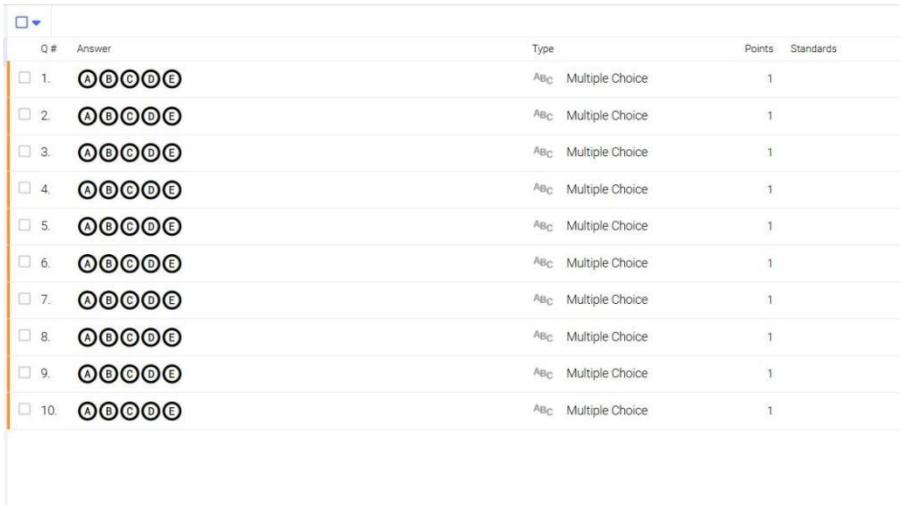

FIGURE 2- 6:GradeCam Create Exam Page

2.3 SMART OMR VS PREVIOUS WORKS(ZIPGRADE & GRADECAM)

By analyzing these specialized OMR systems, the OMR1 project identified a specific need for a lightweight, secure, and localization-friendly solution for individual instructors. While ZipGrade and GradeCam are powerful assessment tools, they are commercial platforms that require subscription fees and do not support regional layout customization.

Smart OMR (OMR1) differentiates itself by performing all bubble detection natively on-device, offering dynamic bilingual RTL interface adjustments, and utilizing source frameworks to remain less cost for educational institutions.

2.3.1 Architecture and Platform Flexibility

ZipGrade:Built primarily as a native mobile application (Swift for iOS and Kotlin/Java for Android). While it offers high mobile performance, it lacks native desktop clients, requiring users to use an auxiliary web portal for administrative management.

GradeCam: Developed as a cloud-centric web application with supplementary mobile app wraps. This allows it to run on browser-equipped devices, but makes performance highly dependent on browser processing limits.

SMART OMR : Developed using the Flutter cross-platform framework. A single codebase compiles into native binaries for Android, iOS, Windows, macOS, Linux, and Web, offering consistent performance and UI layouts across all targets.

2.3.2 Data Persistence and Synchronization Model

ZipGrade:Stores grading data locally in a mobile SQLite database for offline use, syncing with its cloud portal via manual triggers or when a mobile network is active.

GradeCam: Built on a cloud-first model. Scanning and grading operations depend heavily on connection to GradeCam's remote servers. A network disconnect halts grading workflows.

SMART OMR: Employs an offline-first data management paradigm. All student databases, exam setups, answer keys, and grading outcomes are stored locally in a local SQLite database (`sqflite`). Once a connection is detected, the built-in `SyncProvider` performs background synchronization to Cloud Firestore.

2.3.3 Image Processing and Computer Vision Libraries

ZipGrade: Utilizes a closed-source computer vision engine requiring specialized black corner markers printed on bubble sheets to calculate perspective transformation.

GradeCam:Employs proprietary WebAssembly algorithms to track plain-paper boundaries and identify marked circles, which can be sensitive to page wear and lighting.

SMART OMR: Uses the open-source OpenCV library via `opencv_dart`. It processes real-time camera frames and static image uploads, applying custom perspective correction, adaptive thresholding, and contour density calculations locally.

2.3.4 User Interface and Regional Localization

ZipGrade & GradeCam: Designed for Western academic systems. They lack support for Right-to-Left (RTL) text alignment or Arabic translations, presenting user experience issues in Middle Eastern regions.

SMART OMR: **Integrates full** Arabic-English localization**. Using dynamic locale resources (`app_ar.arb` / `app_en.arb`), the application alters user interface layouts (LTR vs. RTL), form fields, and typography automatically when switching languages.

2.3.5 Technical Comparison

The table below consolidates the architectural, operational, and user-facing differences between **ZipGrade**, **GradeCam**, and the **SMAR OMR** project:

Table 2- 1: Technical Comparison

Evaluation Vector / Parameter	ZipGrade	GradeCam	OMR (Smart OMR Project)
Framework & Core Codebase	Native Swift / Kotlin	HTML5 / JavaScript / WebAssembly	**Flutter (Dart compiler)**
Target OS Support	Android, iOS	Web-based (cross-browser)	**Android, iOS, Web, Windows, macOS, Linux**
Primary Persistence Layer	Hybrid (Local SQLite + Cloud)	Cloud Database (Remote Server)	**Local SQLite (Offline-First Paradigm)**
Cloud Synchronization	Manual / Active Sync	Active Network Dependent	**Automated Background Firestore Sync**
Computer Vision Engine	Closed-source library	Closed-source WebAssembly	**Open-source OpenCV (via opencv_dart)**
Camera Scanning Modes	Static viewfinder capture	Live document camera track	**Dual Modes (Live Stream & Photo Upload)**
RTL Alignment & Arabic UI	Unsupported (LTR only)	Unsupported (LTR only)	**Native Dynamic RTL / Bilingual Support**
Roster File Formats	Manual / CSV upload	Manual / CSV upload	**Excel (.xlsx) & CSV Bulk Import**
Report Export Capabilities	CSV, PDF (Cloud only)	CSV, Custom web tables	**Excel (.xlsx) Spreadsheets & Local PDFs**
User Data Isolation	Account-level cloud sync	Account-level cloud sync	**On-Device SQLite Isolation by User ID**

CHAPTER 3: METHODOLOGIES

3.1 OVERVIEW

This chapter outlines the methodologies applied in the design and implementation of the system. It provides detailed explanations of the frameworks, programming languages, development tools, algorithms, and OMR scanning techniques used. These methodologies ensure system reliability, security, and performance.

The system is designed to provide a seamless interface for creating exams, scanning OMR sheets, and managing results. Each component of the system, from front-end to back-end, is carefully structured to maintain usability while ensuring the integrity and confidentiality of user data.

Key objectives of this chapter

- Present the programming and development frameworks used.
- Explain the OMR scanning methodology for capturing and processing answer sheets.
- Describe algorithms and their roles in ensuring accuracy and efficiency.
- Detail back-end methodologies, including data storage and result processing.

3.2 UI (FRONT-END) METHODOLOGIES

3.2.1 Flutter Framework

The user interface (UI) is implemented using the Flutter framework, which allows for cross-platform development with a single codebase.

Dynamic UI Rendering: Flutter widgets enable dynamic rendering of exam creation forms, OMR scanning interfaces, and result dashboards.

Responsive Design: The UI is designed to adapt to various screen sizes, ensuring compatibility across mobile, web, and desktop platforms.

Localization Support: Integrated localization features allow the app to support multiple languages, enhancing accessibility for diverse users.

FIGURE 3- 1:Flutter Framework

3.2.2 Features Of Flutter

The Flutter framework provides developers with the following features;

- Hot-Reload features.
- Cross-Platform Development.
- The framework is both modern and responsive.
- It uses the Dart programming language, which is easy to grab.
- A large selection of widgets.
- High-performance application.
- User interfaces that are both stunning and fluid.

3.2.3 Dart Programming Language

Dart is the core language for both front-end logic and back-end processing.

- State Management: Manages user interactions and application state using Provider or Riverpod.
- UI Logic: Handles dynamic updates to the UI, such as displaying scanned results or updating exam details.

- Integration with Libraries: Facilitates the use of libraries for localization, charting, and animations.

FIGURE 3- 2: Dart Language

3.2.4 The Features Of Dart

- Dart is fast, predictable, and produces native code.
- Dart allows writing all Flutter code in Dart, which speeds up Flutter app development.
- Dart supports smooth animations and transitions for Flutter apps.
- Dart bridges between functional and object-oriented programming without the use of locks.
- Flutter apps written in Dart avoid the need for a separate layout language. making the code easier to visualize and maintain.
- Dart employs a garbage collection and allocation scheme that is effective for modern UI frameworks.
- Dart libraries and tools are well-documented and user-friendly.
- Dart is excellent for client-side and server-side development.

3.2.5 The Coding Characteristics In Dart

- The main() method is where a Dart class begins execution.
- Most data types have null as their default value in Dart.
- Dart classes only allow for single-level inheritance. Therefore, a class can have one superclass but can implement multiple interfaces.
- Control statements such as if, loops like for, while, do-while, and control flow keywords like switch-case, break, and continue follow the same flow structure.

- Control statements such as if, loops like for, while, do-while, and control flow keywords like switch-case, break, and continue follow the same flow structure.
- Dart supports the creation of abstract classes and interfaces.
- All data types, including numbers, are objects. If left uninitialized, their default value is null instead of 0.
- The return type of a method is not required to be specified.
- Any numeric element (real or integer) is declared by the type num.
- The super method is only called at the end of the constructor of a subclass.

3.2.6 Dart Libraries

Dart comes with a set of core libraries that developers can use to perform common tasks. The following is a list of Dart's core libraries:

- Built-in types and collections for basic data manipulation.
- Tools for responsive design like flutter_screenutil for adaptive layouts.
- Local databases like Hive and SQLite for data storage and retrieval.
- Cloud databases such as Firebase Firestore and Supabase for real-time data syncing.
- Notification support with flutter_local_notifications and Firebase Messaging.
- File handling using file_picker and path_provider.
- Utilities for numbers, dates, and tables with Intl and DataGrid tools.
- HTTP requests and API integration via the http library.
- Mathematical utilities and random number generators with dart:math.

3.2.7 Dart Is Used In Flutter

Dart is undoubtedly becoming the preferred language for Flutter developers. With attractive animations, delightful motions, high-fidelity, and amazing performance-Interactive-Uls, game development with Flutter can be done quickly and easily. Flutter is the gaming industry's portable solution. It helps us write and compile user interface codes in Dart so that we can create beautiful apps for iOS, Android, and the web. It also has some gaming features, such as single-code game development, single, double, and multiple gaming services.

Flutter work with Dart. The Flutter engine, Foundation library, and widgets are all part of the Flutter framework written in the Dart programming language. Flutter's approach to development differs from others in that it uses declarative Ul writing. There is a need to start from the end here, which means that before beginning to develop an element, the user must have a complete picture of what kind of Ul it will be. Many developers consider this UI writing to be more concise, but it does present some challenges for developers at first. Flutter's main idea is that developers can create an entire user interface by simply combining various widgets. The application interface comprises nested widgets that can be any type of object. This applies to everything from buttons to padding, and the developer can drastically customize the app by combining widgets. Widgets can interact with one another and respond to external changes in the state using built-in functions. Therefore, widgets are important parts of the user interface that adhere to the design guidelines for Android, iOS, and traditional web apps. Flutter also allows developers to view widgets in a reactive manner. Flutter isn't the first to do this, but it is the only mobile SDK that does so without using a JavaScript bridge. Dart also includes a repository of software packages that can enhance the capabilities of applications. For example, it provides a number of packages that make it easier for developers to connect to Firebase and build several applications.

3.3 PREREQUISITES

Before you can begin Dart programming for Android, you will need certain tools installed. Ensure that you download the Android SDK Bundle, which includes the Android SDK and Integrated Development Environment (IDE). After you download the bundle, unzip the contents and double-click the SDK Manager.exe file. Once the installation is complete, from the Start menu, start VS Code IDE that was bundled with the SDK.

3.3.1 Android Studio

Android Studio to design the app using Dart and Flutter.

Android Studio is the official IDE (Integrated Development Environment) for Android app development.

FIGURE 3- 3: Android Studio

3.3.2 Features of Android Studio

The following features are provided in the current stable version of Android Studio:

- Gradle-based build support.
- Android-specific refactoring and quick fixes.
- Lint tools to catch performance, usability, version compatibility, and other problems.
- Pro Guard integration and app-signing capabilities.
- Template-based wizards to create common Android designs and components.

A rich layout editor that allows users to drag-and-drop Ul components, option to preview layouts on multiple screen configurations. Support for building Android Wear apps.
Built-in support for Google Cloud Platform, enabling integration with Firebase Cloud Messaging (Earlier 'Google Cloud Messaging') and Google App Engine.

3.4 GITBASH

GitBash is an application that provides Git command line experience System. It is a command-line shell for enabling git with the command line in the system. A shell is a terminal application used to interface with an operating through written commands, and we used Git Bash To install the CLI of the technique with this command to download the libraries for framework.

3.4.1 GitBash Advantages

- It stores file changes more efficiently.
- Ensures file integrity better.

Figure 3- 4: Git +GitHub

3.5 DEVELOPMENT TOOLS

3.5.1 Visual Studio Code

Visual Studio Code is an open source cross-development platform that helps developers to build dynamic mobile applications and offers development teams full customization capabilities. It also offers other user interface to design dialogs, notifications, and menus for mobile app. App can be easily drag and drop in order to get a quick installation whereas emulators can be installed and run the app faster as it operates by imitating the design of the user device's architecture. Other than Java and Kotlin programming language, it is possible to implement other written program with C or C++ language by using the Android Native Development Kit.

FIGURE 3- 5 : Visual Studio Code

3.5.2 Visual Studio Code Advantages

- Cross-platform support
- Windows
- Linux
- Mac
- Light-weight
- Robust Architecture
- Intelli-Sense
- Freeware: Free of Cost probably the best feature of all for all the programmers out there, even more for the organizations.

3.6 FIREBASE

Firebase is a Backend-as-a-Service (BaaS) platform by Google that provides a suite of tools for building mobile and web applications. Among its offerings are two powerful NoSQL, cloud databases: Firestore and Realtime Database. These databases are designed to handle real-time data synchronization, scalability, and seamless integration with Firebase services, making them ideal for modern app development.

FIGURE 3- 6: Firebase DB

3.6.1 Key Features

Firebase Firestore

- **Advanced Querying:** Supports structured queries with collections and documents.
- **Real-time Syne:** Keeps data updated across devices in real-time.
- **Scalability:** Designed to handle large datasets and complex queries.
- **Security Rules:** Enables precise access control based on authentication.

Firebase Realtime Database

- **Real-time Data Syne:** Instant data synchronization across multiple clients.
- **Offline Support:** Automatic caching for offline functionality.
- **NoSQL Flexibility:** JSON-like format for dynamic and hierarchical data.
- **Cross-Platform Support:** Compatible with mobile, web, and backend platforms.

3.7 FIGMA UX/UI DESIGN

Figma is a powerful, cloud-based design tool widely used for creating user interfaces (UI) and user experiences (UX) in mobile and web applications. It offers a collaborative platform where designers can work in real-time, ensuring seamless teamwork and faster iteration cycles. For my graduation project, I utilized Figma to design the application's wireframes, prototypes, and final Ut layouts. Its intuitive interface, design components, and prototyping features made it an ideal choice for crafting a user-friendly and visually appealing application. Figma's cloud integration also enabled easy sharing and feedback collection from peers and supervisors.

FIGURE 3- 7: Figma

3.8 ALGORITHMS USED

The system implements algorithms to ensure the accuracy and efficiency of OMR sheet scanning and result processing.

3.8.1 OpenCV

An open-source leading library for “image processing and computer vision”. It is the backbone of the Optical Mark Recognition (OMR) technology within the application.

Primary Role: Responsible for “seeing” and interpreting shaded answers on scanned exam sheets.

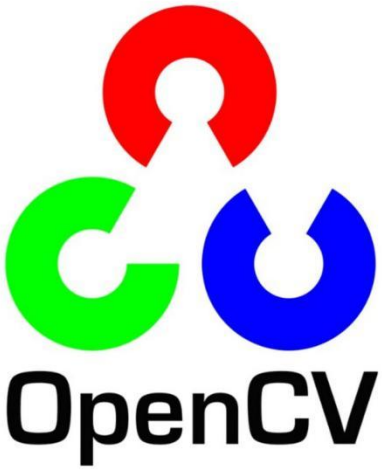

FIGURE 3- 8 :Open CV

Key Functions:

- **Image Processing:** Enhances image quality and converts it to grayscale and binary (black & white) to simplify analysis.
- **Bubble Detection:** Precisely locates each answer (bubble) on the form using edge and shape detection algorithms.
- **Classification:** Analyzes each bubble individually to distinguish between filled (selected answer) and empty states.
- **Advantage:** Provides the system with **high accuracy and speed** in processing visual data, ensuring reliable results.

3.8.1.1 OMR Detection Algorithm

- Detects and processes marked answers on scanned sheets.
- Validates the alignment and ensures accurate recognition of marked options.

3.8.1.2 Result Calculation Algorithm

- Compares scanned answers with the correct answer key.
- Calculates scores and generates detailed analytics for each student.

3.8.2 OMR Scanning Methodology

The OMR scanning module integrates image processing techniques to capture and analyze answer sheets.

Steps:

1. **Image Capture:**
   - The system accepts scanned images of OMR sheets via the app interface.
   - Supports live or real-time image uploads and batch processing.
2. **Answer Detection:**
   - Captured images are analyzed to detect marked answers.
   - Ensures alignment and rejects invalid or incomplete sheets.
3. **Data Storage:**
   - Each valid sheet is time stamped and stored in the database for traceability.
   - Results are linked to the corresponding exam and student records.
4. **Result Processing:**
   - The system calculates scores based on the detected answers and the provided answer key.
   - Generates detailed reports, including individual and class performance analytics.

3.9 BACK-END METHODOLOGIES

3.9.1 Firebase Integration

The back-end is implemented using Firebase, chosen for its real-time database capabilities and seamless integration with Flutter.

- Data Storage: Stores exam details, scanned results, and user information securely.

- Real-Time Updates: Enables instant updates to the UI when new data is added or modified.
- Scalability: Supports a growing number of users and exams without performance degradation.

3.9.2 Session Management

Secure session management is critical for tracking user authentication states:

- Firebase Authentication: Manages user login and ensures secure access to exam data.
- State Persistence: Maintains user state across multiple sessions, ensuring a seamless experience.
- Access Control: Restricts access to sensitive data based on user roles (e.g., teacher, student).

3.9.3 Result Processing and Analytics

The back-end processes scanned results and generates detailed analytics:

- Result Calculation: Compares scanned answers with the correct answer key and calculates scores.
- Performance Analytics: Provides insights into class performance, highlighting strengths and weaknesses.
- Data Visualization: Generates charts and graphs for easy interpretation of results.

3.9.4 Security Considerations

To enhance system security, additional methodologies were implemented:

- Data Encryption: Ensures sensitive data, such as exam results and user credentials, are encrypted.
- Access Control Policies: Prevents unauthorized access to exam data and results.
- Audit Logs: Maintains logs of all system activities for monitoring and troubleshooting.

CHAPTER 4: SYSTEM ANALYSIS AND REQUIREMENTS

4.1 INTRODUCTION

In this chapter, we are going to describe the methodology through using the System Development Life Cycle. Also, the use of technologies in many frames. It incorporates all aspects related to our project and allows us to ensure that the final result is on the highest standards.

4.2 SYSTEM DEVELOPMENT LIFE CYCLE (SDLC)

The System Development Life Cycle (SDLC) is a process used by the software industry to design, develop, and test high-quality software. The SDLC aims to produce high-quality software that meets or exceeds customer expectations and reaches completion within time and cost estimates.
(SDLC) is a process used by a system analyst to develop an information system, including requirements validation, training, and user stakeholder ownership. Any SDLC should result in a high-quality system that:

1. Meets or exceeds customer expectations
2. Reaches completion within time and cost estimates
3. Works effectively and efficiently in the current and planned Information Technology infrastructure
4. Is inexpensive to maintain and cost-effective to enhance

There are many methodologies for the development of information systems, such as The System Development Life Cycle (SDLC), which follows a project throughout from the initial ideas to the point where it is a functional system. It is a core part of the methodology used when defining a project.

The different steps will have different variations of this life cycle model. These stages of the System Development Life Cycle are divided into steps as shown in Figure 10, from definition to creation and modification of IT work products:

1. Planning Phase
2. Analysis Phase
3. Design Phase
4. Development Phase

association means to create. The three essential exercises associated with the analysis stage are as per the following:

- Gathering business requirement
- Creating process diagrams
- Performing a detailed analysis

Business requirement gathering is the most critical part of this dimension of SDLC. Business prerequisites are a concise arrangement of business functionalities that the framework needs to meet so as to be effective. Specialized subtleties, for example, the kinds of framework needs to meet so as to be effective.

4.2.1 Design Phase

Systems design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements. Systems design could be seen as the application of systems theory to product development.

4.2.2 Development Phase

In the development stage, every one of the documents from the previous stage is transformed into a genuine framework. The two essential exercises associated with the advancement stage are as per the following:

- Development of database and code
- Development of IT infrastructure

In the design stage, just the outline of the IT foundation is given, though in this stage the association really buys and introduces the separate programming and equipment parts to help the IT framework. Following this, the formation of the database and real code to start to finish the framework based on given determination.

4.2.3 Testing Phase

In the testing stage, every one of the bits of code is incorporated and sent in the testing condition. Testers at that point pursue Software Testing Life Cycle exercises to check the framework for mistakes, bugs, and deformities to confirm the framework's

- Deployment Phase
- Maintenance Phase

FIGURE 4- 1: System Development Life Cycle(SDLC)

4.2.4 Planning Phase

In the planning stage, venture objectives are resolved and an abnormal state plan for the proposed undertaking is built up. Planning is the most principal and basic authoritative stage. The three essential exercises associated with the planning stage are as per the following:

- Identification of the system for development
- Creation of project plan
- Feasibility assessment

4.2.5 Analysis Phase

In the analysis stage, end client business prerequisites are analyzed and venture objectives are changed over into the characterized framework works that the association means to create. The three essential exercises associated with the analysis stage are as per the following:

- Gathering business requirement

functionalities fill in the requirement charter or not. The two essential exercises associated with the testing stage are as per the following:

- Writing test cases
- Execution of test cases

Testing is a critical part of programming advancement life cycle. To give quality programming, an association must perform testing methodically. When experiments are composed, the analyzer executes them and contrasts the normal outcome and a genuine outcome so as to check the framework and guarantee it works effectively. Composing experiments and executing them physically is a concentrated assignment for any association, which can result in the accomplishment of any business whenever executed appropriately.

4.2.6 Deployment Phase

Amid this next stage, the framework is sent to a real-life (the client's) condition where the real client starts to work with the framework. All information and parts are then put in the production condition. This stage is also known as 'delivery.'

4.2.7 Maintenance Phase

In the maintenance stage, any vital improvements redress, and changes will be ensured the framework keeps on working, and stay refreshed to meet the business objectives. It is important to keep up and update the framework occasionally so it can adjust to future needs. The three essential exercises associated with the upkeep stage are as per the following:

- Support the system users
- System maintenance
- System changes and adjustment

These are the primary SDLC stages that you must know about.

4.3 UNIFIED MODELING LANGUAGE (UML)

The first thing to notice about UML is that there are many different schemes. The reason for this development is that the system is likely to be viewed from different angles according to its participants. Software development involves a number of people, and each one has a role - like:

- Beneficiaries

All of these individuals are interested in different aspects of the system, and each of them needs a different level of detail. UML attempts to provide a powerful expressive language for the system.

4.4 TYPES OF UML DIAGRAMS

4.4.1 Class Diagram

A UML class diagram is a visual tool that represents the structure of a system by showing its classes, attributes, methods, and the relationships between them. It helps everyone involved in a project-like developers and designers-understand how the system is organized and how its components interact.

The Class Diagram, shown in Figure 4-2, represents the static structure of the SMART OMR application, defining the classes, attributes, methods, and relationships.

Class Diagram Relationships

- Model Entities: `Student`, `Exam`, `Question`, and `Result` classes act as data models, containing attributes and JSON serialization methods (`toMap()`, `fromMap()`) for SQLite mapping.
- Relational Mapping: The `Exam` class holds a one-to-many relationship with the `Question` class, reflecting that an exam comprises multiple questions with specific correct choices and points.
- Core Handlers: `DatabaseHelper` implements singleton patterns to manage local database operations. It interacts with all model classes, executing CRUD operations and database transactions.
- State Providers:`AuthProvider`, `StudentProvider`, and `SyncProvider` utilize Flutter's ChangeNotifier pattern to manage states and notify the UI of data updates.

This diagram outlines the object-oriented structure of the Flutter application. It defines the models (`Student`, `Exam`, `Question`, `Result`), their attributes, and relationships (e.g., the 1-to-many relationship between Exams and Questions). It also details the operations of the database helper (`DatabaseHelper`) and state providers (`AuthProvider`, `SyncProvider`, `StudentProvider`).

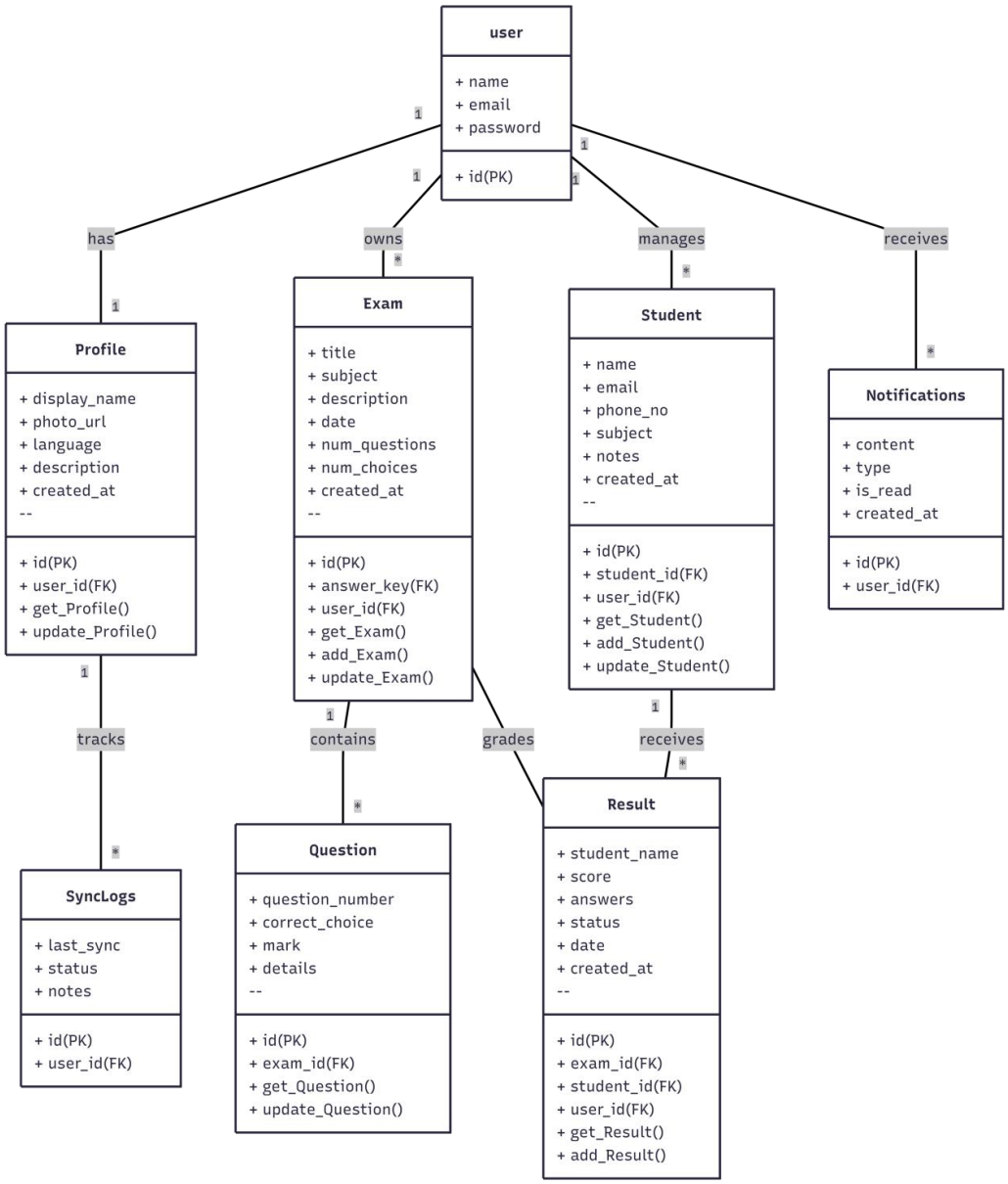

FIGURE 4- 2: Class Diagram

4.4.2 Behavioral UML Diagrams

Behavioral UML diagrams are visual representations that depict the dynamic aspects of a system, illustrating how objects interact and behave over time in response to events.

4.4.3 Use Case Diagram

A Use Case Diagram in Unified Modeling Language (UML) is a visual representation that illustrates the interactions between users (actors) and a system. It captures the functional requirements of a system, showing how different users engage with various use cases, or specific functionalities, within the system. Use case diagrams provide a high-level overview of a system's behavior, making them useful for stakeholders, developers, and analysts to understand how a system is intended to operate from the user's perspective, and how different processes relate to one another. They are crucial for defining system scope and requirements.

The Use Case Diagram, illustrated in Figure 4-3, depicts the functional boundaries of the system, showcasing the interactions between the primary actor (the Teacher/Instructor) and the system's core features.

Use Case Analysis

The Use Case Diagram is used to establish the system boundaries and define user interactions:

Primary Actor: The Teacher/Instructor acts as the primary operator, interacting directly with the client user interface to manage academic data.

Core Functional Boundaries: The actor initiates use cases clustered around four key modules:

- User Identity Management: Includes Register Account and Login User. These use cases verify security access and instantiate the unique user session.
- Roster Configuration: Includes Import Student List (which leverages external Excel sheets) and Manage Student Profiles.
- Exam Setup: Includes Create Exam Layout and Define Answer Key, which establish correct coordinates and marks per question.
- OMR Grading Engine: Includes Scan Answer Sheet (which extends into Live Camera Viewfinder Scan or Gallery Image Upload) and Generate Score Records.

- Data Analytics and Export: Includes View Class Statistics (means, grade ranges) and Export Reports (which extends into generating printable local PDFs or raw Excel files).

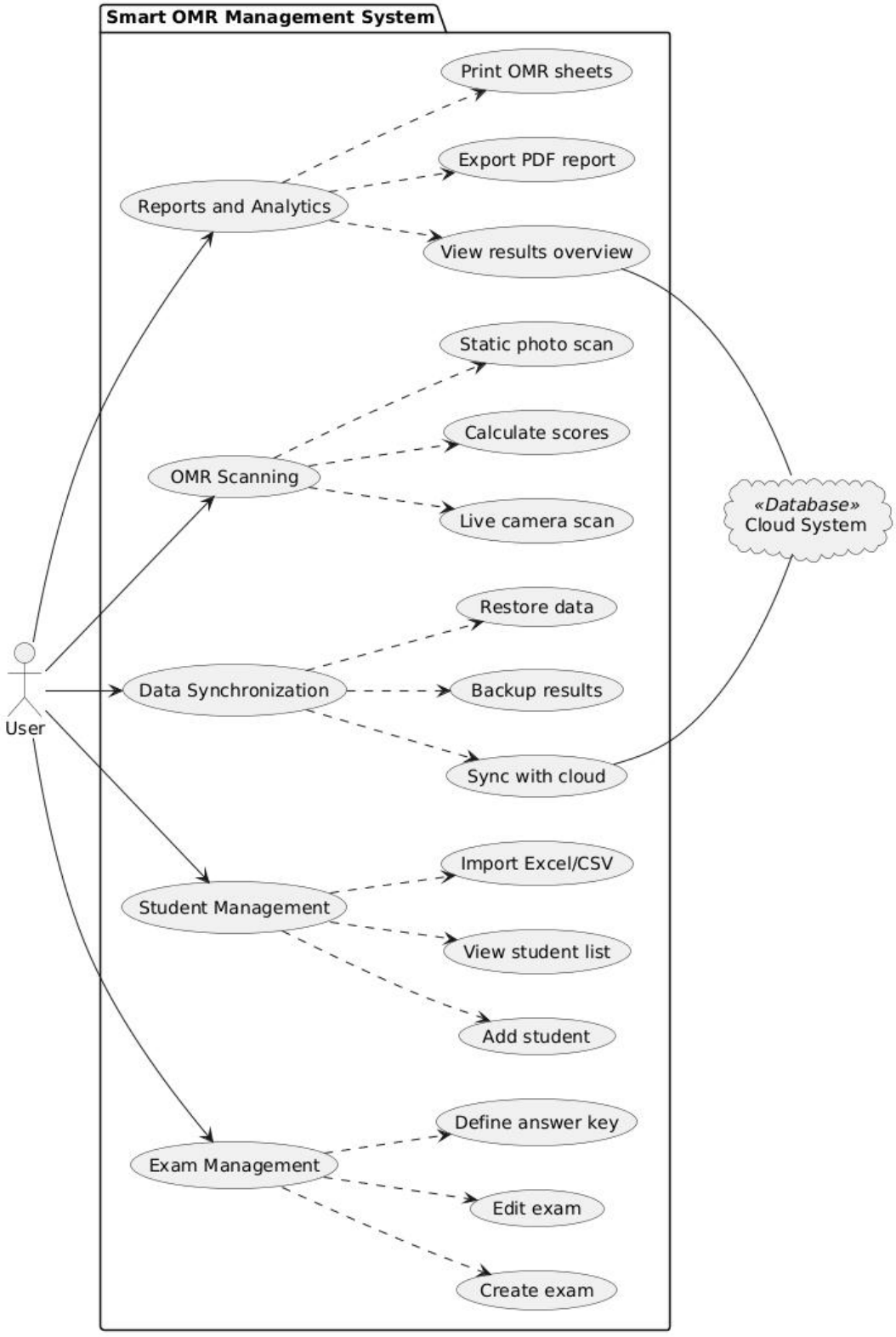

FIGURE 4- 3: Use Case

4.4.4 Sequence Diagram

Sequence diagrams are a type of UML (Unified Modeling Language) diagram that visually represent the interactions between objects or components in a system over time. They focus on the order and timing of messages or events exchanged between different system elements. The diagram captures how objects communicate with each other through a series of messages, providing a clear view of the sequence of operations or processes.

1. Sequence Diagram: Authentication & Cloud Sync

As shown in Figure 4-4, the chronological sequence begins when the user inputs credentials into the UI layout (`WelcomeScreen` or `CreateAccountScreen`). The screen calls the `AuthProvider` asynchronously.

The provider calls Firebase Authentication services over HTTP. Upon validation, Firebase returns the authenticated session data containing the unique User Identifier (`UID`). The `AuthProvider` updates its state, which triggers a SQLite local database update via `DatabaseHelper` to apply `user_id` filtering constraints on all subsequent data fetches.

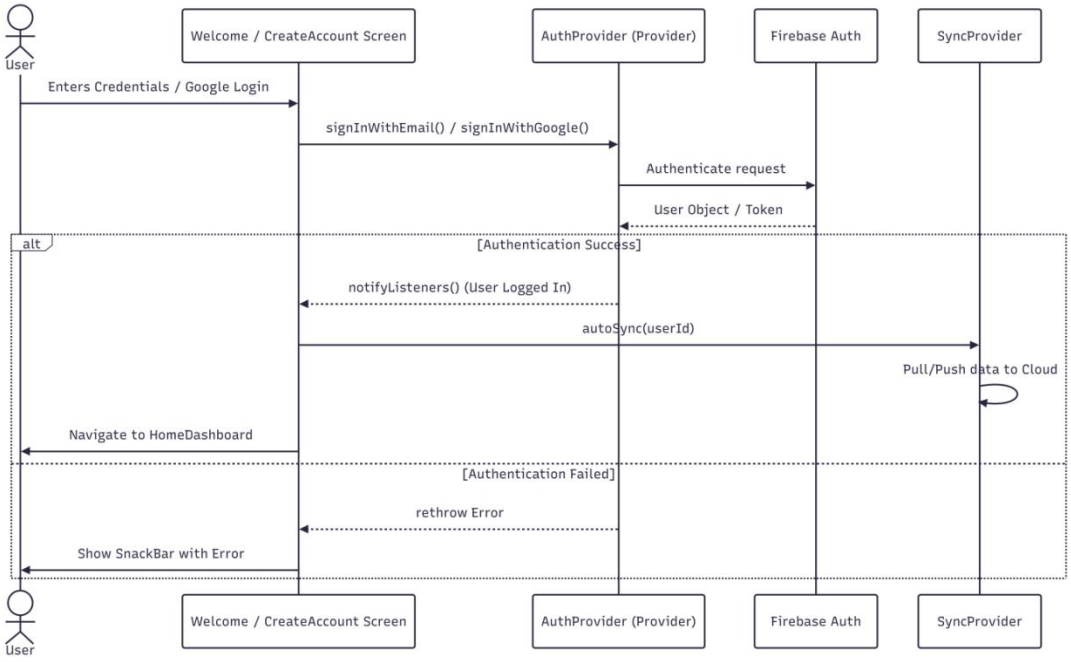

FIGURE 4- 4: Sequence Diagram Sign In and Login

2. Sequence Diagram OMR Live Scanning Pipeline

Illustrated in Figure 4-5, during active camera grading, the `LiveScanScreen` starts a camera frame capture loop. When a paper is aligned, the camera controller streams raw image bytes. To prevent lag on the main thread, the UI calls a background Dart Isolate, passing the image path.
The isolate instantiates the `OMRScanner` class (`omr_pipeline.dart`) to run OpenCV conversions (adaptive thresholding, contour extraction, perspective warp, coordinate clustering, and shade checks). The isolate returns the parsed student ID and answers map back to the main thread. The UI screen calls `DatabaseHelper` to write results and updates the UI.

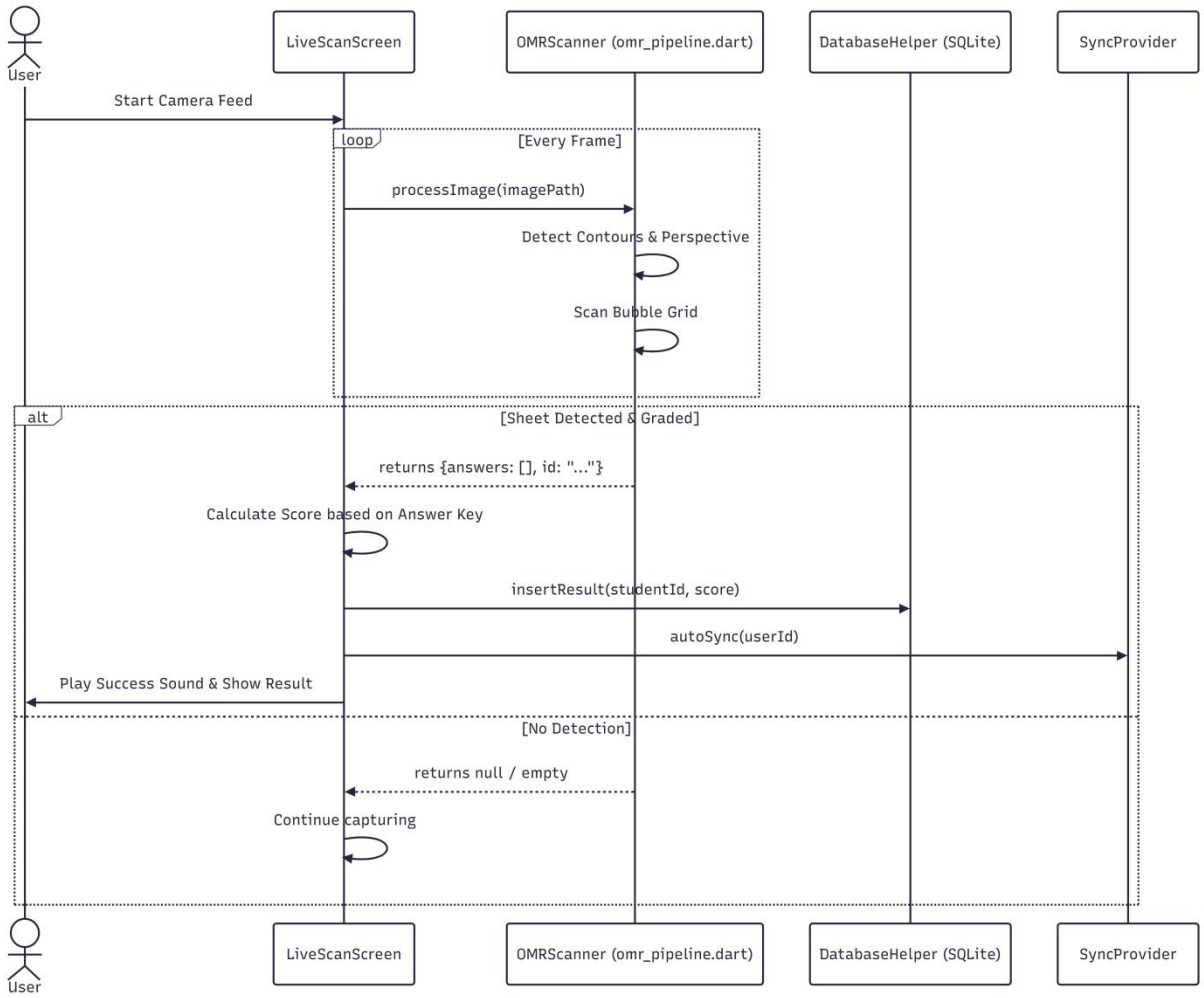

FIGURE 4- 5: Sequence Diagram for Live Scan

3. Sequence Diagram Statistical Report Generation

As mapped in Figure 4-6, when an instructor requests grading analytics, the results screen triggers a query command. The screen calls the `DatabaseHelper` to execute select queries from the local `results`, `students`, and `questions` tables.

The data arrays are returned to the UI controller, which calls the `ReportGenerator` class. The utility class structures the raw matrices and calls the native `pdf` and `excel` libraries to compile files. Once generated, the file handles are sent to `OpenFile` and saved to the local file system.

This diagram tracks the sequence for exporting grading metrics. It details the query flow from the results screen to the local database, the data aggregation by the report generator utility (`report_generator.dart`), and the execution of file writes for PDF and Excel documents.

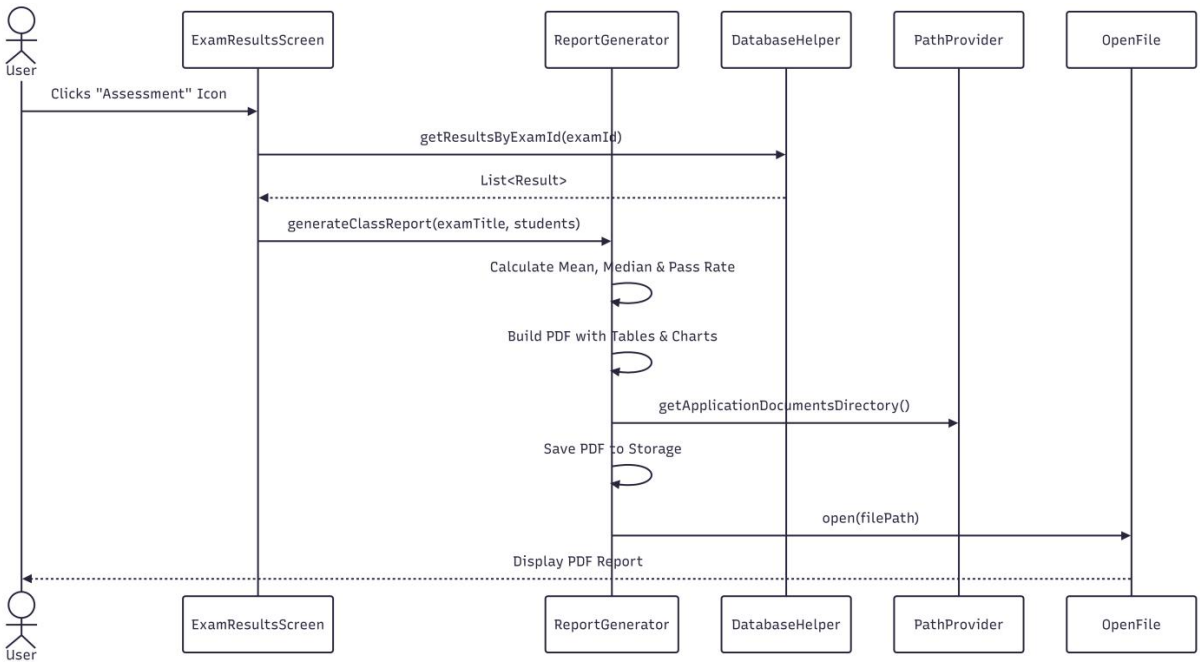

FIGURE 4- 6: Sequence Diagram for Report Generation

4. **Sequence Diagram: Student List Import**

As shown in Figure 4-7, the bulk registration sequence starts when the instructor clicks "Import List" on the student settings screen. The UI triggers `FilePicker` to retrieve target spreadsheets. The selected file handle is passed to the `StudentProvider` class.

The provider calls the `excel` library to load the binary stream and parse student details. Once processed, the provider opens an SQLite transaction block via `DatabaseHelper`. The transactions write multiple rows into the `students` table, and a success callback refreshes the active UI list.

This diagram maps the chronological sequence of roster imports. It shows the file selection process via the file picker, the parsing of cells by the `excel` package, and the transaction-protected batch insertions into the local database.

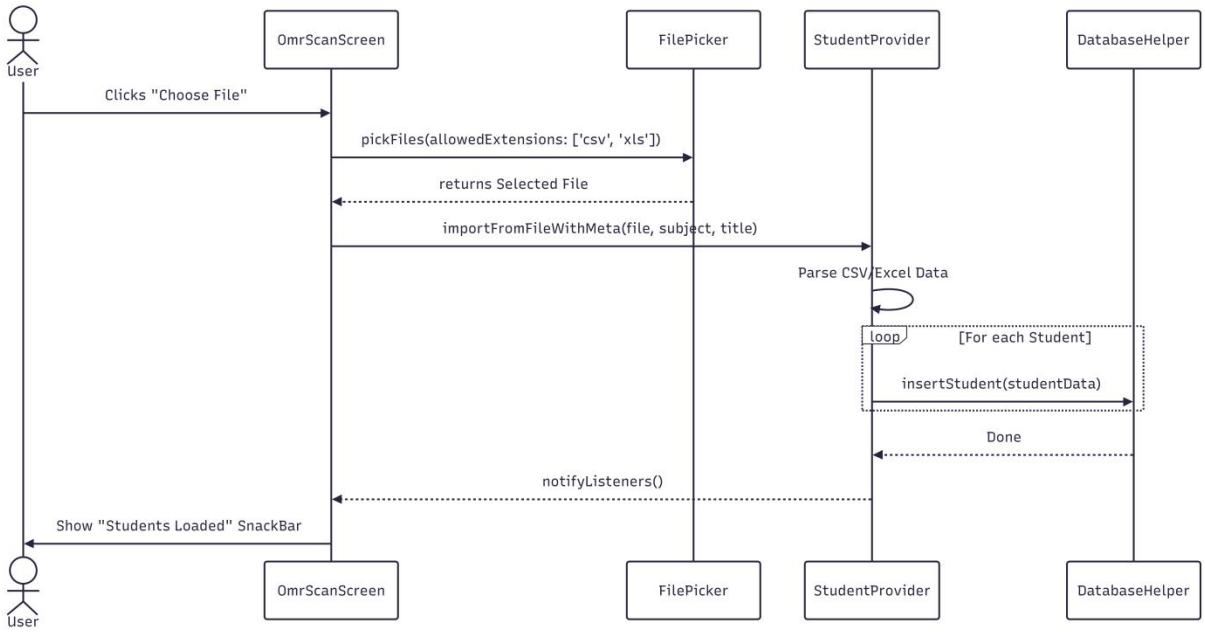

FIGURE 4- 7: Sequence Diagram Student List Import

4.4.5 Activity Diagram

Activity diagrams show the steps involved in how a system works, helping us understand the flow of control. They display the order in which activities happen and whether they occur one after the other (sequential) or at the same time (concurrent). These diagrams help explain what triggers certain actions or events in a system.
Activity diagrams represent the step-by-step operational workflows and decision-making logic of the system.

1. Activity Diagram for Login and sign In

This diagram maps the decision-making flow during user access. It details branches checking for account existence, validating inputs, handling Firebase authentication errors, and redirecting users to the main dashboard.

- Initial State: The user launches the application and arrives at the Welcome Screen.
- Paths & Decisions: The system checks if the user selects Login or Register.
- For registration, the user submits email, username, and password. The system validates email formatting and password strength. If validation fails, it loops back; otherwise, it registers the account.
- For login, the system sends credentials to Firebase Auth. If they are incorrect, it displays an error popup and returns to the input state.
- Terminal State: Upon successful authentication, the event loops to initialize the `SyncProvider` background sync and redirects the user to the Main Dashboard.

This diagram illustrates the user authentication lifecycle during system access. It details the sequential steps taken to evaluate inputs, verify account existence, and intelligently handle Firebase authentication errors, culminating in a seamless redirection to the main dashboard.
Figure 4-8 outlines the comprehensive sequence of credential flows, validation checks, and routing mechanisms.

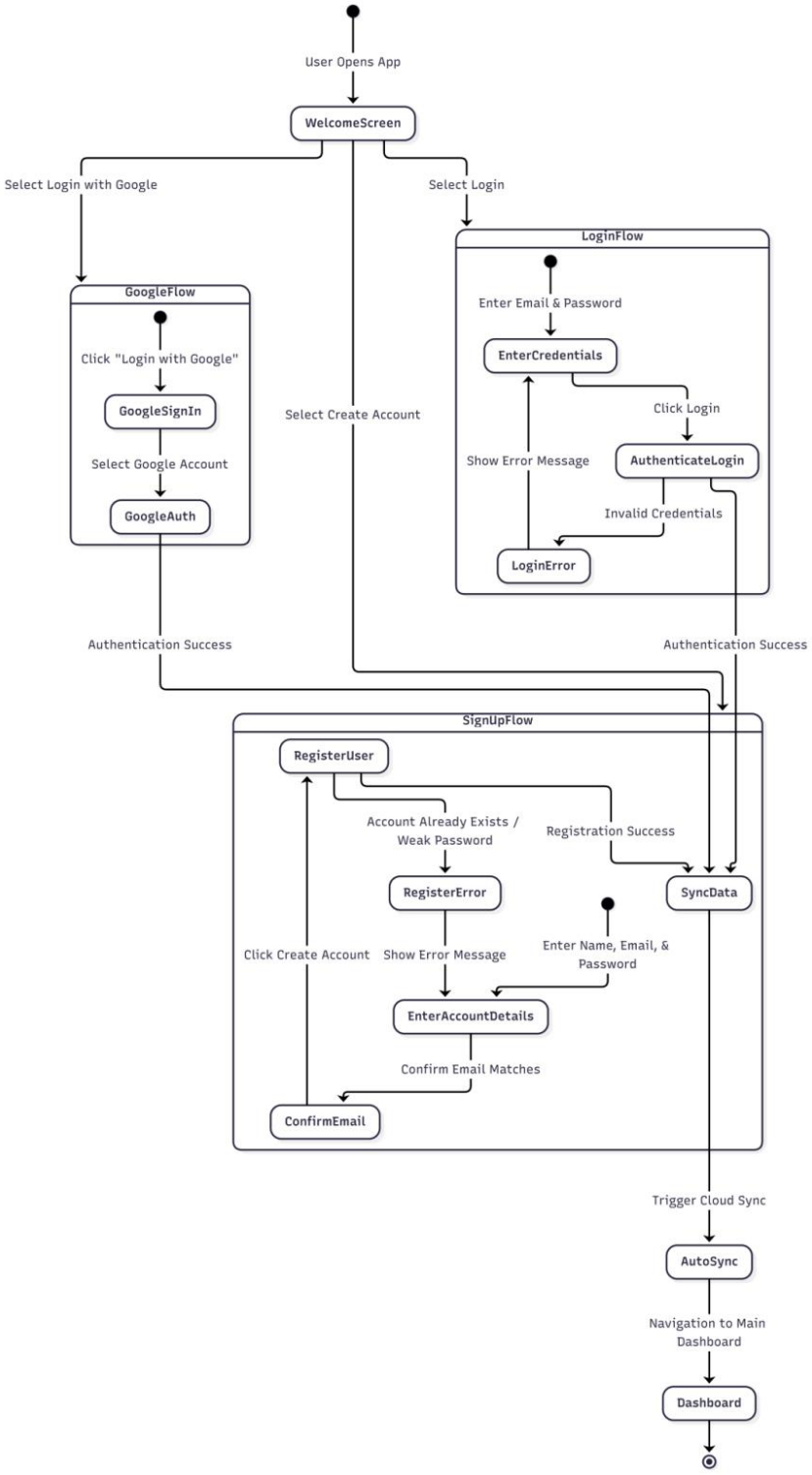

FIGURE 4- 8: Activity Diagram for Login and Sign In

2. **Activity Diagram for Create Exam**

Figure 4-9 shows the workflow for configuring exam records.

- Initial State:The instructor clicks "Create Exam" on the dashboard.
- Actions & Inputs:The user inputs metadata (exam title, subject, question count, and choices per question). The system checks if the inputs are valid.
- Answer Key Matrix: The system renders a dynamic, interactive grid. The instructor inputs correct options and sets marks per question.
- Terminal State: The instructor saves the exam. The system writes the exam layout and question records to SQLite, returning the user to the exam dashboard.

This diagram outlines the step-by-step workflow of exam creation. It traces the input of metadata, the configuration of the answer key grid, point weighting distributions, and the database insertion process.

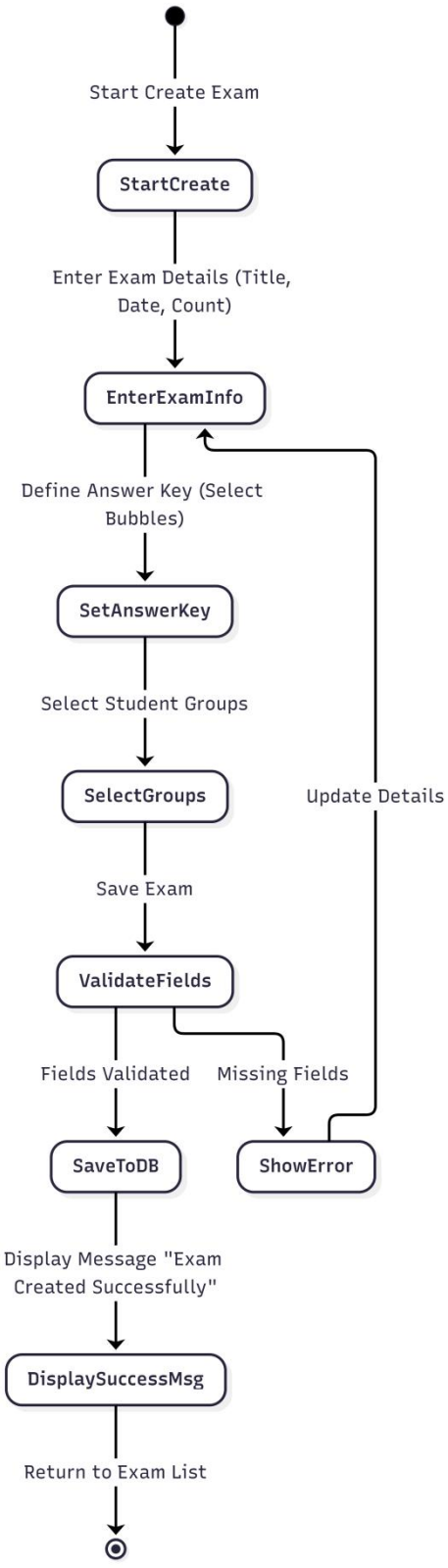

FIGURE4.9. Activity Diagram for Create Exam

1. **Activity Diagram for Scanning and Live Scanning**

Figure 4-10 details the computer vision logic from image acquisition to grade allocation.

- Initial State: The instructor initiates the camera grading module.
- Processing Steps: The camera captures a frame

The image is scaled, converted to grayscale, and blurred. Adaptive thresholding and morphological closing isolate shapes.

- Decision Nodes: The system searches for the Student ID and Answer boxes.
- If boundaries are not found, it flags an alignment error and loops to capture the next frame.
- If found, the system warps the perspective, groups bubble coordinates, and measures pixel densities.
- If multiple marks are found, it flags a multiple selection error. If a single dark mark is found, it logs the bubble value.
- Terminal State: The parsed ID and options are matched with database records, grading scores are saved to SQLite, and a success chime is triggered.

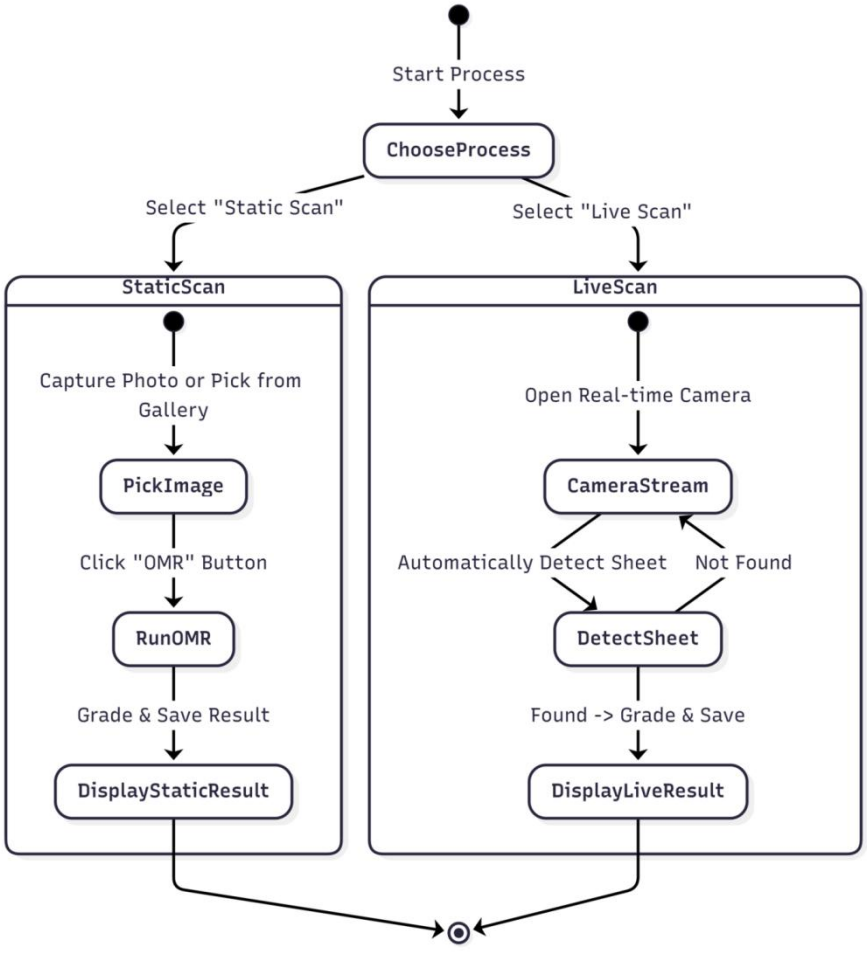

FIGURE 4.10: Activity Diagram for Scanning and Live Scanning

2. **Activity Diagram for Report Generation**

Figure 4-11 shows the process flow for compiling grades and exporting spreadsheets.

- Initial State: The instructor selects an exam on the results panel.
- Operational Flow: The database helper retrieves score records. The system computes average grades, question error distributions, and ranges.
- Decision Nodes: The instructor selects the export target (PDF report or Excel sheet).
- For PDF, the system compiles the layout and prompts the target system to select a directory to save the file.
- For Excel, the system builds columns and writes rows to the selected path.
- Terminal State: The report is saved, and a success notification is displayed.

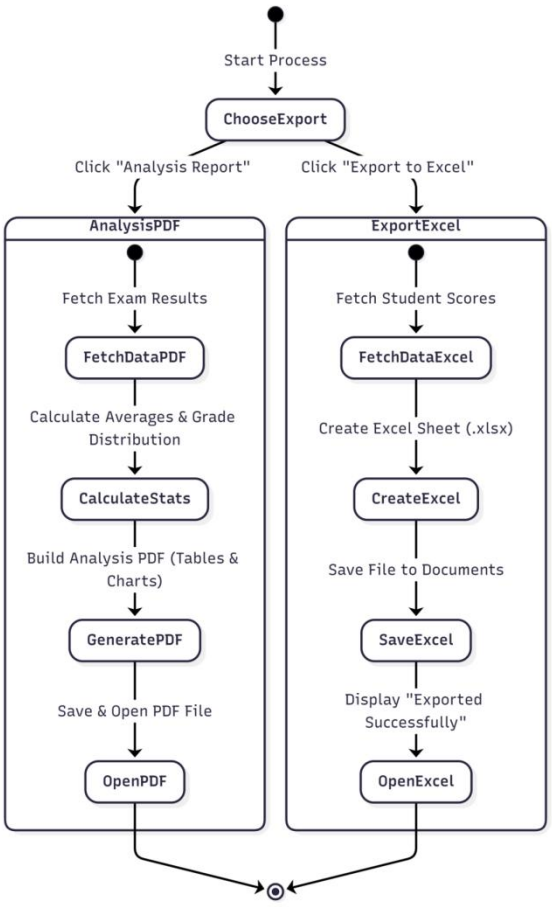

FIGURE 4.11: Activity Diagram for Report Generation

4.4.6 Databases

1. Students table

Table 4- 1: Students Database Table

Field	Datatype	Null / Not Null	Key / Extra
id	INTEGER	Not Null	PK (Auto-Increment)
student_id	TEXT	Not Null	UNIQUE
name	TEXT	Not Null	
subject	TEXT	Null	
notes	TEXT	Null	
user_id	TEXT	Null	FK

2. Exams table

Table 4- 2: Exams Database Table

Field	Datatype	Null / Not Null	Key / Extra
id	INTEGER	Not Null	PK (Auto-Increment)
title	TEXT	Not Null	
subject	TEXT	Not Null	
date	TEXT	Not Null	
num_questions	INTEGER	Not Null	
num_choices	INTEGER	Not Null	
answer_key	TEXT	Not Null	JSON
user_id	TEXT	Null	FK

3. Questions table

Table 4- 3: Questions Database Table

Field	Datatype	Null / Not Null	Key / Extra
id	INTEGER	Not Null	PK (Auto-Increment)
exam_id	INTEGER	Not Null	FK (exams)
question_number	INTEGER	Not Null	
correct_choice	INTEGER	Not Null	
mark	INTEGER	Not Null	

4. Results table

Table 4- 4: Results Database Table

Field	Datatype	Null / Not Null	Key / Extra
id	INTEGER	Not Null	PK (Auto-Increment)
exam_id	INTEGER	Not Null	FK (exams)
student_id	TEXT	Not Null	FK (students)
student_name	TEXT	Not Null	Snapshot
score	INTEGER	Not Null	
answers	TEXT	Not Null	JSON
date	TEXT	Not Null	
user_id	TEXT	Null	FK

CHAPTER 5: DESIGN AND IMPLEMENTATION

5.1 USER INTERFACE (UIS)

The user interface (UI) of the application is designed to provide a seamless and intuitive experience for managing exams, scanning OMR sheets, and viewing results. Below is a detailed description of all the screens in the project:

5.1.1 WELCOME SCREEN

As shown in Figure 5-1, the Welcome Screen is the primary secure entry point designed to streamline user authentication with minimal cognitive load.

UI and Layout Components

- Authentication Card (Central Container): An elevated white container that centralizes login controls to focus user attention.
- Credential Input Fields: Email Field: Features a leading envelope icon and input filtering for valid formats.
- Password Field: Features a leading lock icon and a trailing eye toggle to mask/unmask text.
- Action & Authentication Triggers:
- Primary Login Button: A solid blue CTA button to submit credentials.
- Dividing Separator: An "or" text splitting traditional and federated login flows.
- Federated Login: A "Login with Google" button providing a fast Single Sign-On (SSO) alternative.
- Account Creation Link (Footer): A "Create Account" hyperlink redirecting users to the sign-up page.

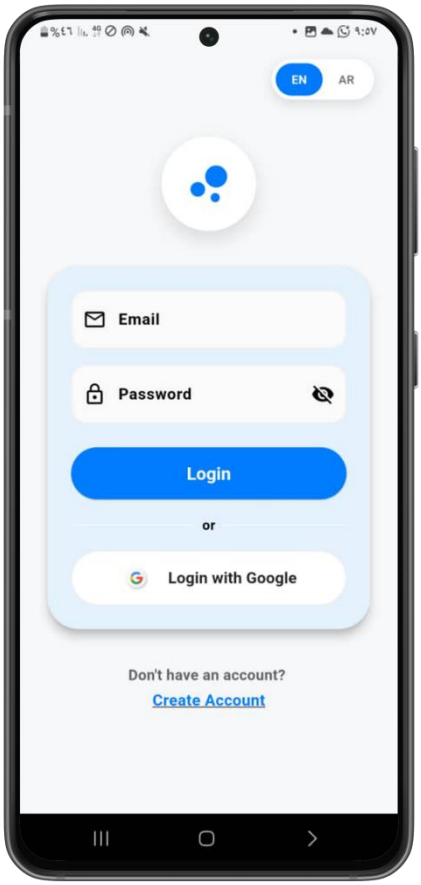

FIGURE 5- 1 : Create Account Screen

5.1.2 CREATE ACCOUNT SCREEN

As shown in Figure 5-2, the Create Account Screen provides a structured form for new user registration, focusing on real-time validation and immediate user feedback.

UI and Layout Components

- Email & Confirm Email Fields: Dual inputs with mail icons designed to prevent typographical errors through cross-checking.
- Password Field: Features a lock icon, masked keystrokes (****), and an eye-visibility toggle.
- User Name Field: Features a silhouette icon for entering the desired display name.
- Action Button: A solid blue "Create Account" CTA button that triggers validation and cloud registration routines.
- Email Mismatch Validation: If the "Confirm Email" field does not match the "Email" field, the label turns red and displays an "Emails do not match" error, blocking form submission.
- Password Strength Checks: Validates registration logic locally (e.g., minimum 8 characters, uppercase, numbers, and special characters) before proceeding.
- Secure Data Transmission:Transmits all registration payload data securely over encrypted HTTPS connections.
- Automated Cloud Synchronization: Upon successful registration, the auth service creates a unique UID, syncs profile details (username, email, date) to the cloud database, generates an active session token, and redirects the user to the dashboard.

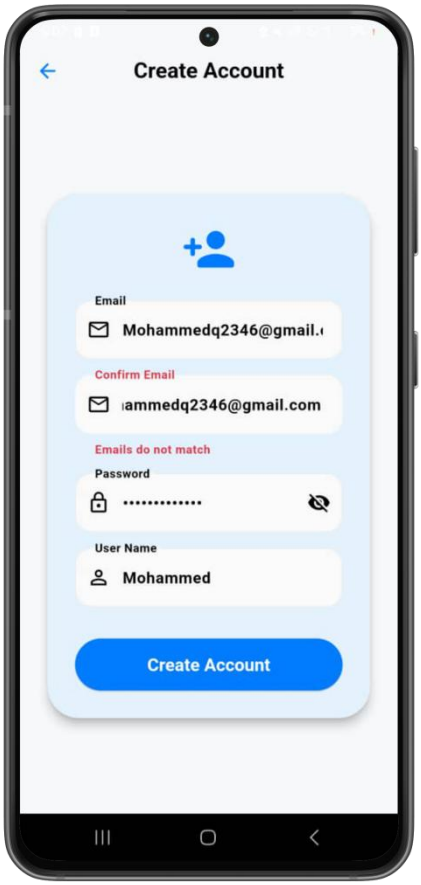

FIGURE 5- 2 : Create Account Screen

5.1.3 HOME DASHBOARD SCREEN

The Home Dashboard Screen serves as the central operation hub for academic evaluators to manage examinations, execute OMR scans, and track student performance through an easily accessible layout.

UI and Layout Components

- Personalized Welcome Banner:A large card displaying a personalized greeting to the user along with the current session date.
- Dynamic Statistical Summary Cards (Horizontal Scroll): Displays real-time metrics fetched directly from the cloud, including Total Exams registered, Scanned OMR Sheets processed, and other analytics.
- Core Actions Quick-Access Grid: four prominent interactive buttons:
  - Create Exam: To define a new test, customize questions, and set up answer keys.
  - Scan OMR Sheet: To instantly launch the device's camera for automated scanning.
  - View Results: To open the analytics dashboard and review scores, statistics, and graded sheets.
- Students: To manage classes, student directories, and identification numbers.
- Integrated Bottom Navigation Bar:Provides immediate access to the app's primary modules (Home, Exams, Results, Account).
- Central Floating Action Button (FAB): A prominent scanner shortcut centered in the navigation bar, allowing users to initiate a camera scan from any screen.

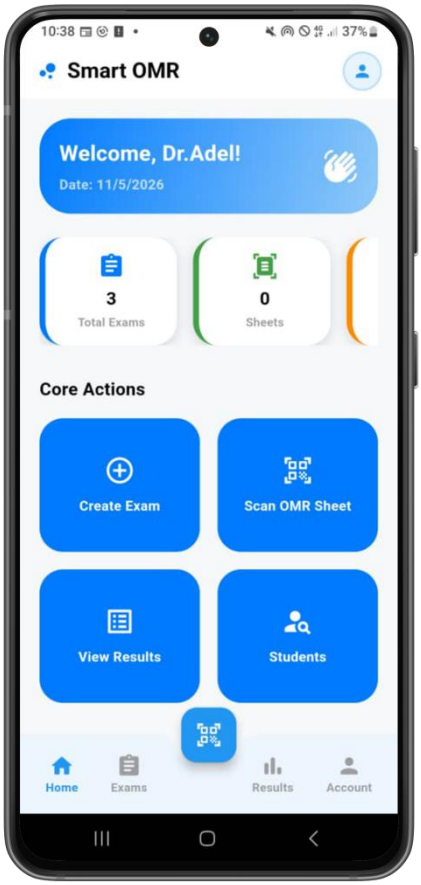

FIGURE 5- 3 :Home Dashboard Screen

5.1.4 PROFILE & ACCOUNT SETTINGS SCREEN

As shown in Figure 5-4, the Profile Screen serves as the user configuration center, packaging user data, localization preferences, and support links into clear visual sections.

UI and Layout Components

- User Profile Header: A centered circular avatar with an edit badge, displaying the teacher's name ("Dr.Adel") and email ("adel@gmail.com").
- Grouped Settings Cards: Rounded white containers featuring scannable tile rows:
- Account Settings: Includes a Language Tile ("English" >) and a Notifications Tile
- Support: Includes a Help Center Tile and an About Smart OMR Tile.
- Logout Button: A distinct full-width soft red button with a logout icon to delineate session destruction.
- Navigation State: The "Account" tab is highlighted on the bottom navigation bar alongside the persistent central scanner FAB.

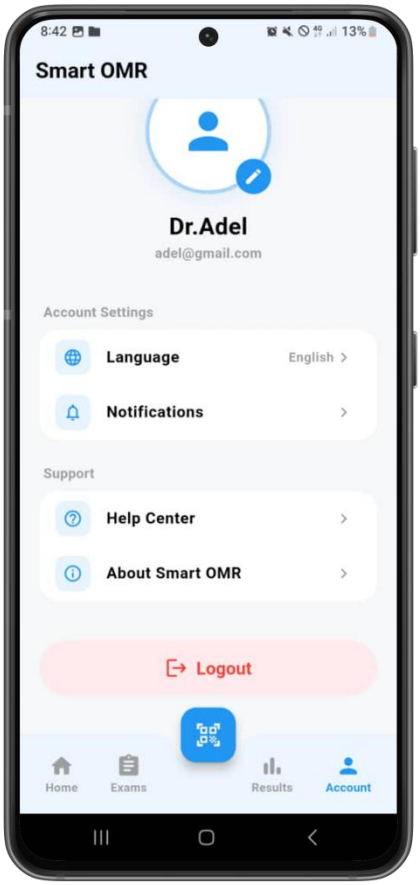

FIGURE 5- 4: Profile Screen

Functional Flow

- Localization Switcher: Tapping the language tile toggles locales (English vs. العربية), dynamically updating the provider to handle text translation and layout direction mirroring (LTR to RTL).
- Session Termination: Clicking "Logout" clears active JWT session tokens and redirects the application shell back to the Welcome Screen.

Technical Implementation

- State Binding: Consumes AuthProvider to dynamically bind displayName, email, and profile image URLs.
- Locale Management: Listens to a global LocaleProvider that notifies the application root widget to trigger a full view rebuild upon language updates.

5.1.5 Create Exam Screen

The Create Exam Screen is a multi-step wizard divided into two main visual phases to define an exam's structure and its answer key ,this **shown in figure 5.5.**

Metadata & Layout Configuration

- Exam Identification: Includes an Exam Title text field and a Subject dropdown selector to categorize assessments for grouped analytics.
- OMR Layout Controls: Uses touch-friendly circular buttons (+ and -) to configure structural properties: Number of Questions, Number of Choices (e.g., A, B, C, D), and Student ID Digits.
- Navigation: A blue "Next" button validates parameters and transitions to the next phase.

Answer Key & Grading Setup

- Sub-Navigation & Modes: Includes a back arrow (\leftarrow) to adjust layout parameters and an Entry Mode Toggle to switch between "Manual Entry" and "Auto Entry".
- Dynamic Question List: Compiles a scrollable list based on the question count:
- Interactive Key Bubbles: Tapping a bubble highlights it in blue with a checkmark (\checkmark) to set the correct answer.
- Custom Score Allocation ("Mark"): A numeric input box for each row to adjust question weightings (default is 1).

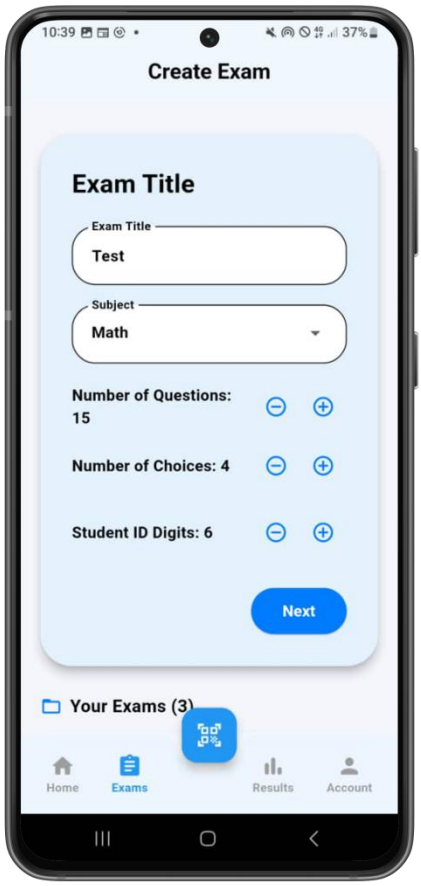

FIGURE 5- 5 :Create Exam Screen

Technical and Printing Integration

- Sheet Generation: A layout engine parses configuration parameters to automatically map precise coordinates for physical OMR bubbles.

- Automated PDF Compilation: Generates a print-ready A4 PDF with registration marks (black anchor boxes) in the corners, enabling the computer vision algorithm to accurately calibrate and locate answers.

5.1.6 ANSWER KEY SETUP SCREEN

As shown in Figure 5-6, this phase of the Create Exam workflow allows the educator to configure the correct answer keys and adjust score weightings for the generated questions.

UI and Layout Components

- Sub-Navigation Header: Features a left-aligned back arrow (\leftarrow) next to the "Answer Key" title, allowing users to return to the layout configuration phase without losing state.
- Entry Mode Toggle: A segmented control featuring two modes: "Manual Entry" (active, highlighted in blue) and "Auto Entry" (inactive, greyed out), offering flexibility between manual selection and automated key importing.
- Dynamic Grid Layout:
- Custom Score Field ("Mark"): Positioned on the far right of each row, a numeric input box allows the educator to define specific point weightings for each question (default set to 1).

FIGURE 5- 6 : Answer Key Screen

- Persistent Bottom Navigation: Displays the application's global navigation menu (Home, Exams, Results, Account) overlayed with the central floating action button (FAB) for instant camera access.

5.1.7 EDIT EXAM SCREEN

As **shown in Figure 5-7**, the Edit Exam Screen provides an interface to modify an existing exam's metadata and fine-tune its answer key post-creation.

UI and Layout Components

- Top Application Bar: Includes a back arrow (\leftarrow), "Edit Exam" title, and a top-right "Save" text button with a floppy disk icon.
- Exam Details Card: Contains editable Exam Title and Subject fields, alongside a dynamic summary badge (e.g., "20 questions • 4 choices").
- Answer Key Section:A scrollable list of questions inside rounded cards, each featuring:
- Question Indicator Badge: Light-blue box highlighting the question number (e.g., Q1, Q2) for scannability.
- Answer Dropdown Selector: An outline dropdown to select the correct choice (A, B, C, D).
- Mark Input Field: An outline numeric text input for question weighting (default is 1).

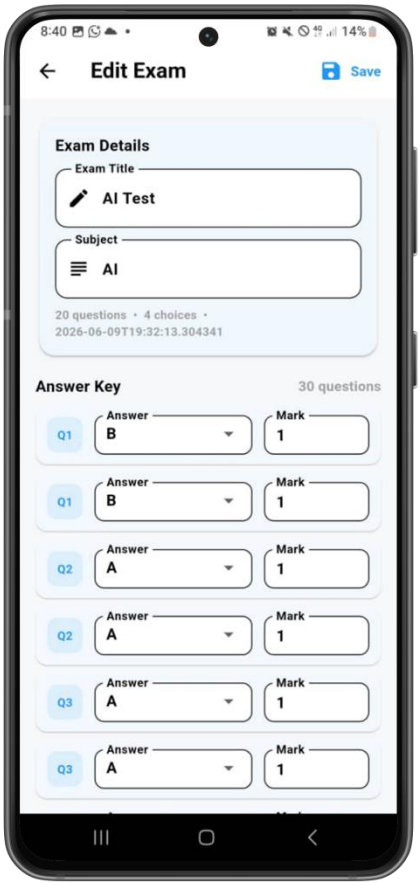

FIGURE 5- 7 :Edit Exam Screen

Functional Flow and State Validation

- Input Constraints: Dropdowns match original creation parameters, while the "Mark" field enforces numeric filtering to prevent calculation errors.
- State Isolation: Edits are isolated within the screen's local state, allowing users to modify fields and cancel via the back arrow without altering the database.

Technical Implementation

- Database & Cloud Sync:Clicking "Save" triggers an async routine updating the model via SQLite (updateExam / updateQuestion), notifies SyncProvider for cloud mirroring, displays a success snackbar, and pops the screen.

5.1.8 EDIT EXAMS LIST SCREEN

As shown in Figure 5-8, the Exams List Screen displays a directory of all registered test templates, providing entry points to edit, manage, or delete assessments.

UI and Layout Components

- Top Application Bar: Contains a back arrow (\leftarrow) and a "Create Exam" title.
- Scrollable Directory: Vertical rounded cards showing a clipboard icon, bold Exam Title, and a dynamic metadata line (e.g., "AI • 20 Q • 2026-06-09").
- Card Action Row: Includes a light-blue Edit Button (pencil icon) and a light-red Delete Button (trash icon).
- Feedback & Navigation: Features an overlay success snackbar ("✓ Exam updated successfully"), with the "Exams" tab highlighted on the bottom navigation bar alongside the central scanner FAB.

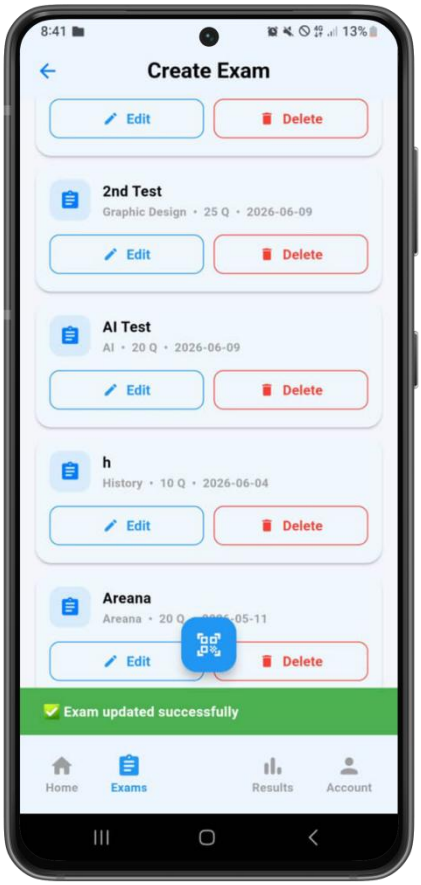

FIGURE 5- 8 : Edit exams List Screen

Functional Flow

- Dynamic Refresh: Reloads local database records and displays the success snackbar automatically upon returning from the editing sub-screen.
- Cascading Delete: Prompts a confirmation modal to avoid data loss before purging the template and its questions.

Technical Implementation

- Database Queries: Executes getAllExams(userId) via DatabaseHelper during layout assembly to fetch and sort local templates by date.
- Data Sync: Asynchronously processes foreign-key cascading deletes, updates UI state observers, and triggers SyncProvider for automated cloud backups.

5.1.9 OMR Sheet Template(Print-Ready PDF)

A Figure 5-9 shows the custom A4 PDF document containing coordinate-aligned bubble matrices designed for automated camera grading via computer vision.

Document Layout & Visual Structure

- Page-Level Alignment Markers: Solid black horizontal bars (40 \times 8 points) at the four outer corners of the page to align the physical paper with the device camera viewfinder.
- Header & Demographics: Displays the exam title, layout parameters (A4, 50 questions, 6-digit ID), and blank lines for handwritten student info.
- Student ID & Instructions Box: A bordered rectangle containing a 6-column grid (0–9) for the student ID, flanked by a testing instructions box. It features four solid black square anchors (12 \times 12 points) at its inside corners.
- Answer Section Container: A large bordered rectangle at the bottom housing 50 questions, dynamically split into 5 columns of 10 rows (Options: A, B, C). It also features four solid black square corner anchors for grid calibration.

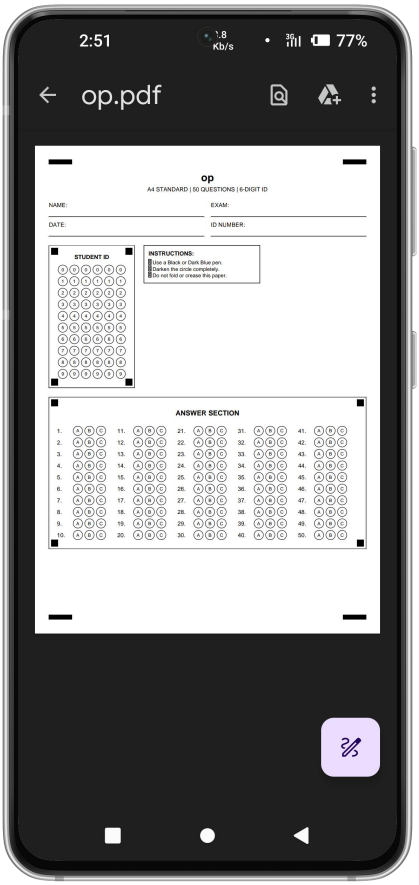

FIGURE 5- 9 :OMR Sheet Template

5.1.10 OMR Scan Screen

The OMR Scan Screen in **figure 5-10** is the core operational interface for digitizing and grading sheets, supporting single captures, bulk uploads, or live video processing.

UI and Layout Components

- Top Navigation & Live Trigger: Features a back arrow (\leftarrow) and a red "Live Scan" button that activates the live camera overlay for automated real-time video stream analysis.
- Template & Roster Management:
- Exam Template Selector: A dropdown to map scanned sheets to a specific exam blueprint.
- Student Roster Card:Manages student database files (CSV/Excel), displaying a green document icon, file name, and total parsed student count.
- Capture Viewfinder:A preview box with instructions to align physical sheets inside the frame for optimal corner detection.
- Action Panel (Bottom Row):
- Scan Button (Blue): Captures the frame and triggers the single-image grading algorithm.
- Upload Image Button (White):Opens the gallery to process pre-captured photos.
- OMR Button (Purple): Initiates the OMR extraction pipeline to convert pixel density into numeric grades.

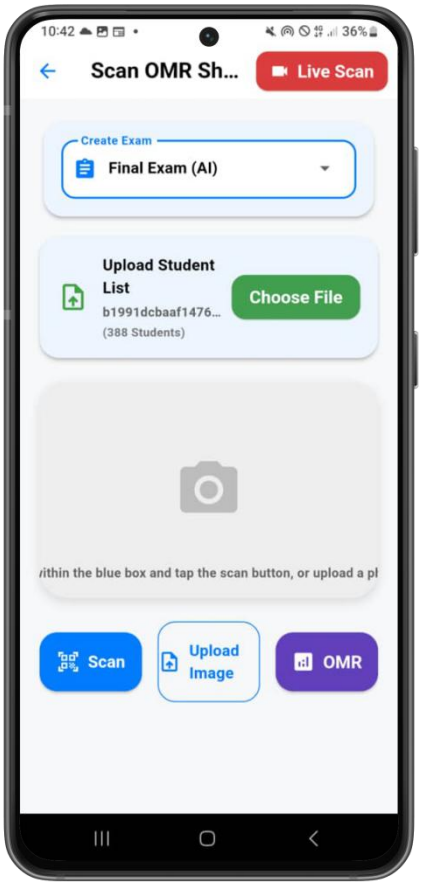

FIGURE 5- 10 : OMR Scan Screen

5.1.11 LIVE SCAN SCREEN

As shown in Figure 5-11, the Live Scan Screen features a continuous camera preview loop designed for automated, real-time OMR sheet processing without manual shutter triggers.

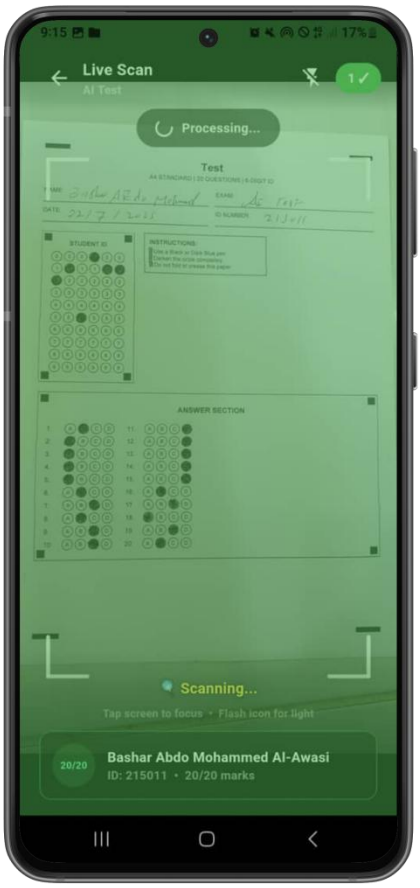

FIGURE 5- 11: Live Scan Screen

UI and Layout Components

- Top Navigation & Status Bar: Includes a back arrow (\leftarrow), active template context, a flash toggle for low-light tracking, and a green Batch Count Indicator ("[Count] ✓").
- Viewfinder & Alignment Guides: A high-definition preview overlayed with a tap-to-focus target and a semi-transparent guide that turns green once corner registration marks are detected.
- Bottom Status Panel:Features an animated state banner (e.g., confirming, prompts) and a floating Recent Result Card showing a color-coded score avatar alongside the student's metadata.

Functional Flow and Double-Confirmation

- Anti-Flicker Processing: Captures frames every 2.5 seconds. The computer vision engine must match the exact same Student ID across two consecutive frames to prevent misreads.
- Success Feedback: Upon verification, the UI flashes green, triggers an audio beep, persists the grade locally, and enters a 3-second cooldown state to allow sheet swapping.

Technical Implementation

- Multi-Threading: Offloads image processing tasks to a background worker (Flutter Isolate) via compute(), keeping the live viewport running at a fluid 60 FPS.
- Data & Cache Sync: Writes scores immediately to SQLite, triggers a background sync to the cloud, and instantly purges processed frames from local storage cache.

5.1.12 RESULTS OVERVIEW SCREEN

As shown in Figure 5-12, the Results Overview Screen serves as the primary data repository and grading dashboard of the application. It lists all created assessments, showing their scanning status, and acts as the portal to access detailed gradebooks, search records, and export reports. The detailed layout, functional navigation, and data management workflows are described below:

User Interface (UI) and Layout Components

- KPI Summary Cards (Horizontal Carousel):
- Recent Scans Directory List:
- A section titled "Recent Scans" lists the individual directories for active exams. Each list item functions as an interactive folder:
- Folder Icon: A blue document icon on the left signifies an organized exam grading folder.
- Status Badge: A light-blue, rounded capsule badge on the right displays the number of successfully graded student sheets (showing "0 Sheets" for the current empty folders).
- Active Navigation State:
- The "Results" tab in the bottom navigation bar is highlighted in blue, indicating that the user.

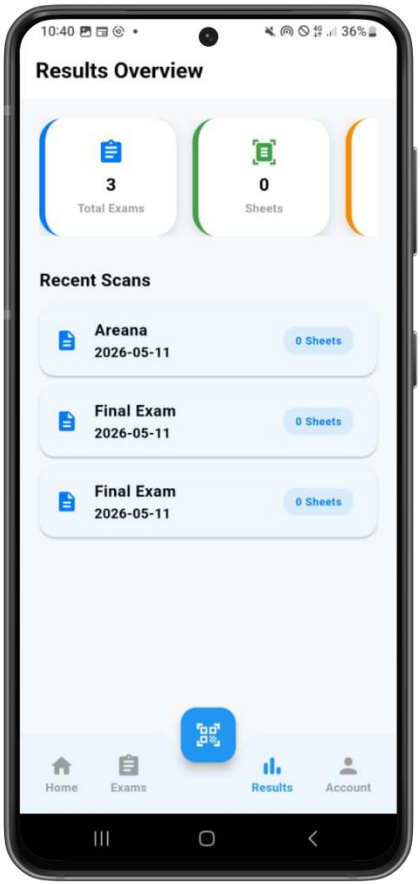

FIGURE 5- 12 :Result Overview Screen

5.1.13 CLASS ANALYSIS PDF REPORT

As shown in Figure 5-13, the Class Analysis PDF Report is a dynamically generated A4 document compiled into the system viewer to provide actionable student performance metrics.

Document Layout and Visual Structure

- Header & Info Row: Includes a bold blue title banner, generation timestamp, logo, and an info band displaying Exam Title, Subject, and Total Students.
- KPI Metrics Cards: Three bordered cards displaying Average Score (blue), Pass Rate(green), and Fail Rate (red).
- Analytical Visualizations: Grade Distribution Chart: Renders blue bars for standard letter grade tiers (A, B, C, D, F).
- Accuracy Analysis Table: A grid tracking question numbers, correct counts, accuracy, and color-coded difficulty ratings.

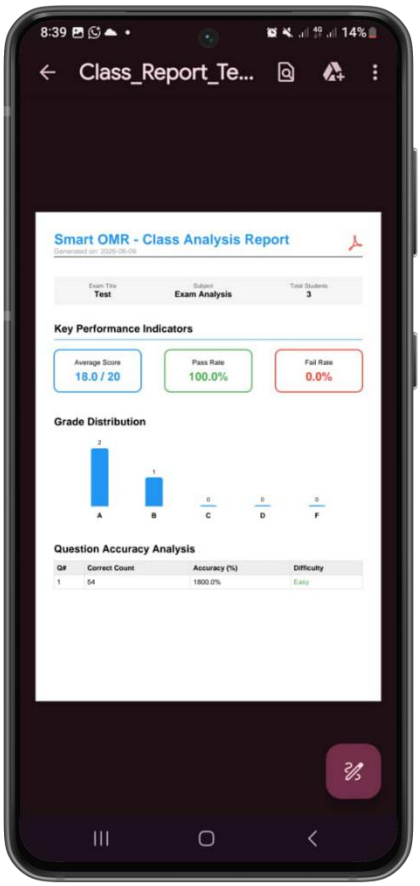

FIGURE 5- 13: Class Analysis PDF Report

Functional Flow and Analytics

- Grade Categorization: Filters passing students at \ge 50\% and splits scores into standard letter grade buckets.
- Difficulty Mapping: Automatically computes difficulty based on student accuracy: Easy* (\ge 70\%, green), Medium (40\% - 69\%, orange), and Hard (< 40\%, red).

Technical Implementation

- PDF Compilation: Uses the Dart pdf/widgets.dart library to programmatically render layout trees onto an A4 canvas.
- Storage & Sharing: Writes bytes locally via path_provider and utilizes open_file to send an operating system intent, launching the file in the default viewer for printing or cloud archiving.

5.1.14 Student Result Screen

The Student Result module provides a dual-level view: first, a directory interface to organize student groups and classes (Figure 5-14), and second, a highly detailed diagnostic page showing an individual student's exam score and question-by-question response breakdown (Figure 5-15). The details of these two interfaces are described below:

Student Directory and Roster Management

As shown in Figure 5-14, the Students Directory Screen acts as the administrative portal for managing student rosters and class cohorts:

- A prominent search field reading "Search by name" allows the educator to quickly locate individual student records within the system.
- Categorized Class Folders: Students are grouped under custom category headings (e.g., "IS DESIGN" and "All") to maintain class organization:
- A Floating Action Button (FAB) labeled "+ Import" is placed at the bottom right. This button allows educators to upload class rosters directly from external spreadsheet files (CSV or Excel) to populate student profiles in bulk.

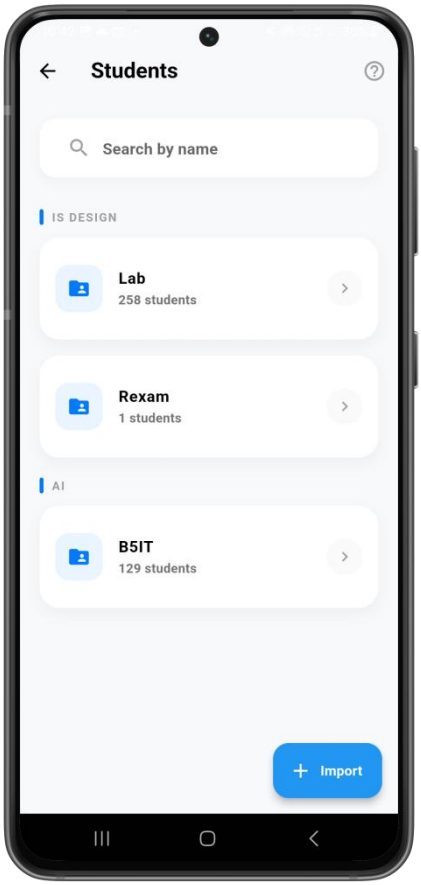

FIGURE 5- 14 : Student Result Screen

Individual Diagnostic Student Result

As shown in Figure 5-15, tapping on a student's name inside a graded exam folder opens the Individual Student Result view (titled "Result: Math Final"). This screen provides a clear, transparent breakdown of the student's performance:

- "Answers" section, which uses distinct color-coding to give students and teachers immediate visual feedback on performance:

- Correct Responses (Green Rows): . It displays the question number and the student's answer (e.g., "Q 1: Your answer: A").
- Incorrect Responses (Red Rows):
- Correct Key Correction: Crucially, to aid learning and verify grading accuracy, the incorrect row (e.g., "Q 2: Your answer: B")

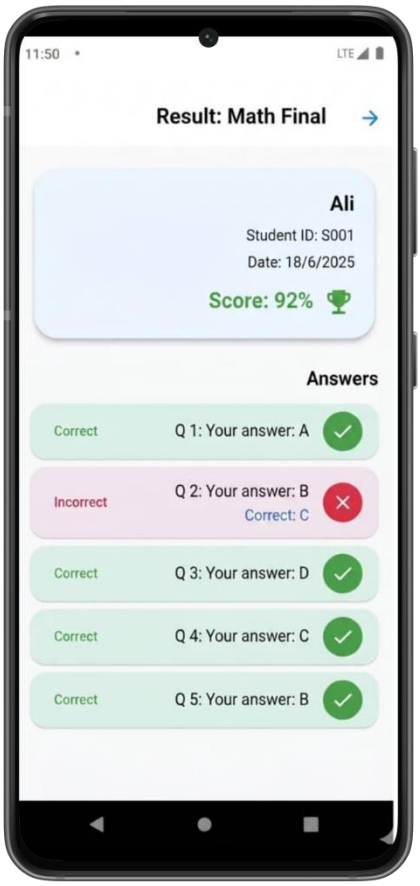

FIGURE 5- 15: Subject Results Screen

CONCLSION

This project has demonstrated the design and implementation of an Optical Mark Recognition (OMR) exam grading application using the Flutter framework and OpenCV. The system simplifies the grading workflow, allowing instructors to scan sheets, manage student rosters, and generate performance analytics using standard mobile hardware.

In contrast to older drafts of this report, the current system implements a robust offline-first data persistence layer. By using an embedded SQLite database (`sqflite`), the application can create exams, store student lists, grade papers, and calculate statistics entirely offline. Once internet connectivity is restored, the synchronization module (`SyncProvider`) automatically backs up data to Firebase Cloud Firestore. This dual-database setup ensures continuous operation in classrooms with poor network access while providing cloud backups.

Future Enhancements

- ✧ Advanced Neural Network Classification: Integrating TensorFlow Lite to identify handwritten student numbers, reducing identification errors.
- ✧ Flexible Page Layout Parser: Allowing instructors to design custom bubble sheets with variable dimensions, row gaps, and section blocks.
- ✧ Advanced Statistical Models: Integrating predictive models to identify class comprehension trends and suggest curriculum focus areas.

REFERENCES

1. Google Flutter Documentation, “Flutter Developer Reference Guides,” 2025. [Online]. Available: https://docs.flutter.dev
2. OpenCV Developer Foundation, “OpenCV Library Reference Manuals,” 2025. [Online]. Available: https://docs.opencv.org
3. SQLite Development Group, “SQLite SQL Syntax and Database Engine Operations,” 2025. [Online]. Available: https://sqlite.org/docs.html
4. S. Kumar and A. Arora, “Optical mark recognition using image processing techniques,” International Journal of Computer Applications, vol. 184, no. 32, pp. 15–21, 2022.
5. R. Patel and D. Bhatt, “Automated evaluation of multiple-choice questions using OMR technology,” Journal of Academic Software Engineering, vol. 12, no. 4, pp. 110–117, 2021.
6. M. M. Rahman and M. S. Hossain, “Mobile application development using Flutter and Dart: A case study,” Journal of Systems and Software, vol. 177, no. 22, pp. 25–31, 2020.
7. H. El-Sayed and A. Ahmed, “A survey on automated exam grading systems using OMR and machine learning techniques,” Journal of Computer Science and Information Technology, vol. 9, no. 2, pp. 45–58, 2021.

APPENDIX

Welcome screen code

```
Import ‘package:flutter/material.dart’;
Import ‘package:flutter_gen/gen_l10n/app_localizations.dart’;
Import ‘package:provider/provider.dart’;
Import ‘../locale_provider.dart’;
Import ‘../providers/auth_provider.dart’;
Import ‘../providers/sync_provider.dart’;
Import ‘create_account_screen.dart’;

Class WelcomeScreen extends StatefulWidget {
  Const WelcomeScreen({super.key});

  @override
  State<WelcomeScreen> createState() => _WelcomeScreenState();
}

Class _WelcomeScreenState extends State<WelcomeScreen> {
  Final TextEditingController _emailController = TextEditingController();
  Final TextEditingController _passwordController = TextEditingController();
  Final GlobalKey<FormState> _formKey = GlobalKey<FormState>();

  Bool _obscurePassword = true;
  String? _emailError;

  @override
  Void initState() {
    Super.initState();
    // Clear error when user types
    _emailController.addListener(() {
      If (_emailError != null) {
        setState(() => _emailError = null);
      }
    });
  }

  @override
```

```
  Void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    Super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    Final loc = AppLocalizations.of(context)!;
    Final localeProvider = Provider.of<LocaleProvider>(context, listen: false);
    Final authProvider = Provider.of<AuthProvider>(context);
    Final syncProvider = Provider.of<SyncProvider>(context, listen: false);
    Final isArabic = Localizations.localeOf(context).languageCode == 'ar';

    Return Scaffold(
      Body: Stack(
        Children: [
          Center(
            Child: SingleChildScrollView(
              Padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 32),
              Child: Form(
                Key: _formKey,
                Child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    const SizedBox(height: 16),
                    // Logo
                    Container(
                      Margin: const EdgeInsets.only(bottom: 32),
                      Decoration: const BoxDecoration(
                        boxShadow: [BoxShadow(color: Colors.black12, blurRadius: 16, offset:
Offset(0, 8))],
                        shape: BoxShape.circle,
                      ),
                      Child: CircleAvatar(
                        Radius: 48,
                        backgroundColor: Colors.white,
                        child: Icon(Icons.bubble_chart, size: 56, color:
Theme.of(context).primaryColor),
                      ),
                    ),
```

```
                    // Login Card
                    Card(
                      Shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(32)),
                      Elevation: 8,
                      Child: Padding(
                        Padding: const EdgeInsets.all(24),
                        Child: Column(
                          crossAxisAlignment: isArabic ? CrossAxisAlignment.end :
CrossAxisAlignment.start,
                          children: [
                            TextFormField(
                              Controller: _emailController,
                              textAlign: isArabic ? TextAlign.right : TextAlign.left,
                              decoration: InputDecoration(
                                labelText: loc.emailLabel,
                                prefixIcon: const Icon(Icons.email_outlined),
                                filled: true,
                                fillColor: Colors.grey[50],
                                border: OutlineInputBorder(borderRadius:
BorderRadius.circular(16), borderSide: BorderSide.none),
                                enabledBorder: OutlineInputBorder(borderRadius:
BorderRadius.circular(16), borderSide: const BorderSide(color: Color(0xFFEEEEEE))),
                                errorText: _emailError,
                              ),
                              keyboardType: TextInputType.emailAddress,
                              validator: (value) => (value == null || value.isEmpty) ?
loc.emailLabel : null,
                            ),
                            Const SizedBox(height: 16),
                            TextFormField(
                              Controller: _passwordController,
                              textAlign: isArabic ? TextAlign.right : TextAlign.left,
                              decoration: InputDecoration(
                                labelText: loc.passwordLabel,
                                prefixIcon: const Icon(Icons.lock_outline),
                                suffixIcon: IconButton(
                                  icon: Icon(_obscurePassword ? Icons.visibility_off :
Icons.visibility),
                                  onPressed: () => setState(() => _obscurePassword
= !_obscurePassword),
                                ),
```

```
                                Filled: true,
                                fillColor: Colors.grey[50],
                                border: OutlineInputBorder(borderRadius:
BorderRadius.circular(16), borderSide: BorderSide.none),
                                enabledBorder: OutlineInputBorder(borderRadius:
BorderRadius.circular(16), borderSide: const BorderSide(color: Color(0xFFEEEEEE))),
                              ),
                              obscureText: _obscurePassword,
                              validator: (value) => (value == null || value.isEmpty) ?
loc.passwordLabel : null,
                            ),
                            Const SizedBox(height: 24),
                            SizedBox(
                              Width: double.infinity,
                              Child: ElevatedButton(
                                onPressed: authProvider.isLoading
                                  ? null
                                  : () async {
                                      If (_formKey.currentState!.validate()) {
                                        Try {
                                          Await
authProvider.signInWithEmail(_emailController.text.trim(), _passwordController.text.trim());
                                          syncProvider.autoSync(authProvider.userId);
                                        } catch € {
                                          String errorMsg = e.toString();
                                          If (errorMsg.contains('user-not-found') ||
errorMsg.contains('invalid-credential')) {
                                            setState(() => _emailError = isArabic ? "الايميل او كلمة
المرور غير صحيحة" : "Incorrect email or password");
                                          } else {

ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(errorMsg), backgroundColor:
Colors.redAccent, behavior: SnackBarBehavior.floating));
                                          }
                                        }
                                      }
                                    },
                                Child: authProvider.isLoading
                                  ? const SizedBox(height: 20, width: 20, child:
CircularProgressIndicator(color: Colors.white, strokeWidth: 2))
                                  : Text(loc.loginButton),
```

```
                              ),
                            ),
                            Const SizedBox(height: 16),
                            Row(
                              Children: [
                                Const Expanded(child: Divider()),
                                Padding(padding: const EdgeInsets.symmetric(horizontal: 8),
child: Text(loc.orLabel)),
                                Const Expanded(child: Divider()),
                              ],
                            ),
                            Const SizedBox(height: 16),
                            SizedBox(
                              Width: double.infinity,
                              Child: OutlinedButton(
                                onPressed: authProvider.isLoading
                                  ? null
                                  : () async {
                                      Try {
                                        Await authProvider.signInWithGoogle();
                                        syncProvider.autoSync(authProvider.userId);
                                      } catch € {

ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(e.toString())));
                                      }
                                    },
                                Style: OutlinedButton.styleFrom(
                                  Shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(32)),
                                  Side: const BorderSide(color: Color(0xFFEEEEEE)),
                                  Padding: const EdgeInsets.symmetric(vertical: 12),
                                  backgroundColor: Colors.white,
                                ),
                                Child: Row(
                                  mainAxisAlignment: MainAxisAlignment.center,
                                  children: [
                                    Image.asset('assets/images/google_logo.png', height: 32),
                                    Const SizedBox(width: 16),
                                    Text(loc.loginWithGoogle, style: const TextStyle(color:
Colors.black87, fontWeight: FontWeight.w600, fontSize: 16)),
                                  ],
```

```
                              ),
                            ),
                          ),
                        ],
                      ),
                    ),
                  ),
                  Const SizedBox(height: 32),
                  Column(
                    Children: [
                      Text(loc.dontHaveAccount, style: TextStyle(fontSize: 15, color:
Colors.grey[700])),
                      Const SizedBox(height: 4),
                      GestureDetector(
                        onTap: () => Navigator.of(context).push(MaterialPageRoute(builder: (_)
=> const CreateAccountScreen())),
                        child: Text(
                          loc.createAccount,
                          style: TextStyle(color: Theme.of(context).primaryColor, fontWeight:
FontWeight.bold, fontSize: 16, decoration: TextDecoration.underline),
                        ),
                      ),
                    ],
                  ),
                ],
              ),
            ),
          ),
          // Language Selector
          Positioned(
            Top: MediaQuery.of(context).padding.top + 16,
            Right: 24,
            Child: Container(
              Decoration: BoxDecoration(
                Color: Colors.white,
                borderRadius: BorderRadius.circular(24),
                boxShadow: const [BoxShadow(color: Colors.black12, blurRadius: 10, offset:
Offset(0, 4))],
              ),
              Padding: const EdgeInsets.all(4),
```

```
              Child: Row(
                mainAxisSize: MainAxisSize.min,
                children: [
                  _LanguageOption(label: ‘EN’, isSelected: !isArabic, onTap: () =>
localeProvider.setLocale(const Locale(‘en’))),
                  _LanguageOption(label: ‘AR’, isSelected: isArabic, onTap: () =>
localeProvider.setLocale(const Locale(‘ar’))),
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }
}

Class _LanguageOption extends StatelessWidget {
  Final String label;
  Final bool isSelected;
  Final VoidCallback onTap;
  Const _LanguageOption({required this.label, required this.isSelected, required this.onTap});
  @override
  Widget build(BuildContext context) {
    Return GestureDetector(
      onTap: onTap,
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 200),
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
        decoration: BoxDecoration(
          color: isSelected ? Theme.of(context).primaryColor : Colors.transparent,
          borderRadius: BorderRadius.circular(18),
        ),
        Child: Text(label, style: TextStyle(color: isSelected ? Colors.white : Colors.grey[600],
fontWeight: FontWeight.bold, fontSize: 12)),
      ),
    );
  }
}
```

End welcome screen code

Create account screen code

```
Import ‘package:flutter/material.dart’;
Import ‘package:flutter_gen/gen_l10n/app_localizations.dart’;
Import ‘package:provider/provider.dart’;
Import ‘../providers/auth_provider.dart’;
Import ‘../providers/sync_provider.dart’;

Class CreateAccountScreen extends StatefulWidget {
  Const CreateAccountScreen({super.key});

  @override
  State<CreateAccountScreen> createState() => _CreateAccountScreenState();
}

Class _CreateAccountScreenState extends State<CreateAccountScreen> {
  Final _formKey = GlobalKey<FormState>();
  Final TextEditingController _emailController = TextEditingController();
  Final TextEditingController _confirmEmailController = TextEditingController();
  Final TextEditingController _passwordController = TextEditingController();
  Final TextEditingController _nameController = TextEditingController();

  Bool _obscurePassword = true;

  @override
  Void dispose() {
    _emailController.dispose();
    _confirmEmailController.dispose();
    _passwordController.dispose();
    _nameController.dispose();
    Super.dispose();
  }

  Future<void> _submit() async {
    If (_formKey.currentState!.validate()) {
      Final authProvider = Provider.of<AuthProvider>(context, listen: false);
      Final syncProvider = Provider.of<SyncProvider>(context, listen: false);
      Try {
        Await authProvider.signUpWithEmail(
          _emailController.text.trim(),
          _passwordController.text.trim(),
          _nameController.text.trim(),
```

```
      );

      // Immediately register the user document in Firestore
      Final uid = authProvider.userId;
      If (uid != null) {
        Try {
          Await syncProvider.ensureUserDocument(uid);
        } catch € {
          debugPrint('Could not write user doc on signup: $e');
          // Non-fatal - will retry on next sync
        }
      }

      // Kick off background sync for any existing data
      syncProvider.autoSync(authProvider.userId);

      if (mounted) {
        Navigator.of(context).pop();
      }
    } catch € {
      If (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            Content: Text(e.toString()),
            backgroundColor: Colors.redAccent,
            behavior: SnackBarBehavior.floating,
          ),
        );
      }
    }
  }
}

@override
Widget build(BuildContext context) {
  Final loc = AppLocalizations.of(context)!;
  Final authProvider = Provider.of<AuthProvider>(context);
  Final isArabic = Localizations.localeOf(context).languageCode == 'ar';

  Return Scaffold(
    backgroundColor: const Color(0xFFF7F8FA),
```

```
      appBar: AppBar(
        backgroundColor: Colors.transparent,
        elevation: 0,
        iconTheme: IconThemeData(color: Theme.of(context).primaryColor),
        title: Text(loc.createAccount, style: const TextStyle(color: Colors.black, fontWeight:
FontWeight.bold)),
        centerTitle: true,
      ),
      Body: Center(
        Child: SingleChildScrollView(
          Padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 20),
          Child: Card(
            Shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(32)),
            Elevation: 8,
            Child: Padding(
              Padding: const EdgeInsets.all(28),
              Child: Form(
                Key: _formKey,
                Child: Column(
                  mainAxisSize: MainAxisSize.min,
                  crossAxisAlignment: isArabic ? CrossAxisAlignment.end :
CrossAxisAlignment.start,
                  children: [
                    Center(child: Icon(Icons.person_add_rounded, size: 56, color:
Theme.of(context).primaryColor)),
                    Const SizedBox(height: 24),

                    TextFormField(
                      Controller: _emailController,
                      textAlign: isArabic ? TextAlign.right : TextAlign.left,
                      decoration: InputDecoration(
                        labelText: loc.emailLabel,
                        prefixIcon: const Icon(Icons.email_outlined),
                        filled: true,
                        fillColor: Colors.grey[50],
                        border: OutlineInputBorder(borderRadius: BorderRadius.circular(16),
borderSide: BorderSide.none),
                        enabledBorder: OutlineInputBorder(borderRadius:
BorderRadius.circular(16), borderSide: const BorderSide(color: Color(0xFFEEEEEE))),
                      ),
                      keyboardType: TextInputType.emailAddress,
```

```
                      validator: (value) {
                        if (value == null || value.isEmpty) return loc.emailLabel;
                        final emailRegex = RegExp(r'^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$');
                        if (!emailRegex.hasMatch(value)) return isArabic ? " : "ادخل ايميل صحيح
"Enter a valid email";
                        return null;
                      },
                    ),
                    Const SizedBox(height: 16),

                    TextFormField(
                      Controller: _confirmEmailController,
                      textAlign: isArabic ? TextAlign.right : TextAlign.left,
                      decoration: InputDecoration(
                        labelText: isArabic ? "" : "تأكيد الايميلConfirm Email",
                        prefixIcon: const Icon(Icons.email_outlined),
                        filled: true,
                        fillColor: Colors.grey[50],
                        border: OutlineInputBorder(borderRadius: BorderRadius.circular(16),
borderSide: BorderSide.none),
                        enabledBorder: OutlineInputBorder(borderRadius:
BorderRadius.circular(16), borderSide: const BorderSide(color: Color(0xFFEEEEEE))),
                      ),
                      keyboardType: TextInputType.emailAddress,
                      validator: (value) {
                        if (value != _emailController.text) return isArabic ? " : "الايميلات غير متطابقة
"Emails do not match";
                        return null;
                      },
                    ),
                    Const SizedBox(height: 16),

                    TextFormField(
                      Controller: _passwordController,
                      textAlign: isArabic ? TextAlign.right : TextAlign.left,
                      decoration: InputDecoration(
                        labelText: loc.passwordLabel,
                        prefixIcon: const Icon(Icons.lock_outline),
                        suffixIcon: IconButton(
                          icon: Icon(_obscurePassword ? Icons.visibility_off :
Icons.visibility),
```

```
                          onPressed: () => setState(() => _obscurePassword = !_obscurePassword),
                        ),
                        Filled: true,
                        fillColor: Colors.grey[50],
                        border: OutlineInputBorder(borderRadius: BorderRadius.circular(16),
borderSide: BorderSide.none),
                        enabledBorder: OutlineInputBorder(borderRadius:
BorderRadius.circular(16), borderSide: const BorderSide(color: Color(0xFFEEEEEE))),
                      ),
                      obscureText: _obscurePassword,
                      validator: (value) => value != null && value.length >= 6 ? null :
(isArabic ? "كلمة المرور قصيرة" : 'Password too short'),
                    ),
                    Const SizedBox(height: 16),

                    TextFormField(
                      Controller: _nameController,
                      textAlign: isArabic ? TextAlign.right : TextAlign.left,
                      decoration: InputDecoration(
                        labelText: loc.userName,
                        prefixIcon: const Icon(Icons.person_outline),
                        filled: true,
                        fillColor: Colors.grey[50],
                        border: OutlineInputBorder(borderRadius: BorderRadius.circular(16),
borderSide: BorderSide.none),
                        enabledBorder: OutlineInputBorder(borderRadius:
BorderRadius.circular(16), borderSide: const BorderSide(color: Color(0xFFEEEEEE))),
                      ),
                      Validator: (value) => value != null && value.trim().isNotEmpty ? null :
loc.enterValidUserName,
                    ),
                    Const SizedBox(height: 32),

                    SizedBox(
                      Width: double.infinity,
                      Child: ElevatedButton(
                        onPressed: authProvider.isLoading ? null : _submit,
                        child: authProvider.isLoading
                            ? const SizedBox(height: 20, width: 20, child:
CircularProgressIndicator(color: Colors.white, strokeWidth: 2))
```

```
                          : Text(loc.createAccount, style: const TextStyle(fontSize: 18,
fontWeight: FontWeight.bold)),
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

End create account screen code

Home dashboard screen

```
import 'package:flutter/material.dart';
import 'package:omr1/screens/create_exam_screen.dart';
import 'package:omr1/screens/omr_scan_screen.dart';
import 'package:omr1/screens/results_overview_screen.dart';
import 'package:flutter_gen/gen_l10n/app_localizations.dart';
import 'package:provider/provider.dart';
import 'package:omr1/locale_provider.dart';
import '../db/database_helper.dart';
import '../providers/auth_provider.dart';
import '../providers/sync_provider.dart';

class HomeDashboardScreen extends StatefulWidget {
  final Function(int)? onTabChange;
  const HomeDashboardScreen({super.key, this.onTabChange});

  @override
  State<HomeDashboardScreen> createState() => HomeDashboardScreenState();
}

class HomeDashboardScreenState extends State<HomeDashboardScreen> {
  Future<Map<String, dynamic>>? _statsFuture;
  Future<List<Map<String, dynamic>>>? _recentScansFuture;
```

```
@override
void initState() {
  super.initState();
  refreshData();
  // Auto-sync on startup
  WidgetsBinding.instance.addPostFrameCallback((_) {
    final auth = Provider.of<AuthProvider>(context, listen: false);
    final sync = Provider.of<SyncProvider>(context, listen: false);
    sync.autoSync(auth.userId);
  });
}

Future<void> refreshData({bool triggerSync = false}) async {
  final auth = Provider.of<AuthProvider>(context, listen: false);
  setState(() {
    _statsFuture = DatabaseHelper().getDashboardStats(auth.userId);
    _recentScansFuture = DatabaseHelper().getRecentScans(5, auth.userId);
  });

  if (triggerSync) {
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (mounted) {
        Provider.of<SyncProvider>(context, listen: false).autoSync(auth.userId);
      }
    });
  }
}

@override
Widget build(BuildContext context) {
  final loc = AppLocalizations.of(context)!;
  final auth = Provider.of<AuthProvider>(context);
  final userName = auth.user?.displayName ?? loc.teacher;
  final today = DateTime.now();
  final dateString = "${today.day}/${today.month}/${today.year}";

  return Scaffold(
    backgroundColor: const Color(0xFFF7F8FA),
    appBar: AppBar(
      backgroundColor: Colors.white,
      elevation: 0,
```

```
  title: Row(
    children: [
      Icon(Icons.bubble_chart, color: Theme.of(context).primaryColor),
      const SizedBox(width: 8),
      Text(loc.smartOmr,
          style: const TextStyle(
              color: Colors.black, fontWeight: FontWeight.bold)),
    ],
  ),
  actions: [
    _buildProfileMenu(context, auth),
    const SizedBox(width: 8),
  ],
  iconTheme: const IconThemeData(color: Colors.black),
),
body: SafeArea(
  child: RefreshIndicator(
    onRefresh: () async {
      await refreshData(triggerSync: true);
    },
    child: ListView(
      padding: const EdgeInsets.all(20),
      children: [
        // Header
        Container(
          padding: const EdgeInsets.all(20),
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(32),
            gradient: const LinearGradient(
              colors: [Color(0xFF007BFF), Color(0xFF90CAF9)],
              begin: Alignment.topLeft,
              end: Alignment.bottomRight,
            ),
          ),
          child: Row(
            children: [
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text("${loc.homeScreenTitle}, $userName!",
```

```
                style: const TextStyle(
                    fontSize: 22,
                    fontWeight: FontWeight.bold,
                    color: Colors.white)),
            const SizedBox(height: 6),
            Text("${loc.date}: $dateString",
                style: const TextStyle(color: Colors.white70)),
          ],
        ),
      ),
      const Icon(Icons.waving_hand,
          color: Colors.white, size: 36),
    ],
  ),
),
const SizedBox(height: 24),
// Summary Cards
FutureBuilder<Map<String, dynamic>>(
  future: _statsFuture,
  builder: (context, snapshot) {
    final stats = snapshot.data ??
        {'totalExams': 0, 'totalSheets': 0, 'avgScore': 0.0};
    return SizedBox(
      height: 110,
      child: ListView(
        scrollDirection: Axis.horizontal,
        children: [
          _SummaryCardModern(
            icon: Icons.assignment,
            value: '${stats['totalExams']}',
            label: loc.totalExams,
            color: const Color(0xFF007BFF),
          ),
          _SummaryCardModern(
            icon: Icons.document_scanner,
            value: '${stats['totalSheets']}',
            label: loc.sheets,
            color: const Color(0xFF43A047),
          ),
          _SummaryCardModern(
            icon: Icons.bar_chart,
```

```
            value:
                '${(stats['avgScore'] as double).toStringAsFixed(1)}%',
            label: loc.avgScore,
            color: const Color(0xFFFB8C00),
          ),
        ],
      ),
    );
  },
),
const SizedBox(height: 32),
// Quick Actions
Text(loc.coreActions,
    style: const TextStyle(
        fontSize: 18, fontWeight: FontWeight.bold)),
const SizedBox(height: 16),
GridView.count(
  crossAxisCount: 2,
  shrinkWrap: true,
  mainAxisSpacing: 18,
  crossAxisSpacing: 18,
  childAspectRatio: 1.2,
  physics: const NeverScrollableScrollPhysics(),
  children: [
    _QuickActionModern(
      icon: Icons.add_circle_outline,
      label: loc.createExam,
      onTap: () async {
        if (widget.onTabChange != null) {
          widget.onTabChange!(1);
        } else {
          await Navigator.of(context).push(
            MaterialPageRoute(
                builder: (_) => const CreateExamScreen()),
          );
          refreshData();
        }
      },
    ),
    _QuickActionModern(
      icon: Icons.qr_code_scanner,
```

```
      label: loc.scanOmrSheet,
      onTap: () async {
        await Navigator.of(context).push(
          MaterialPageRoute(
              builder: (_) => const OmrScanScreen()),
        );
        refreshData();
      },
    ),
    _QuickActionModern(
      icon: Icons.list_alt,
      label: loc.resultsButton,
      onTap: () {
        if (widget.onTabChange != null) {
          widget.onTabChange!(2);
        } else {
          Navigator.of(context)
              .push(
                MaterialPageRoute(
                    builder: (_) =>
                        const ResultsOverviewScreen()),
              )
              .then((_) => refreshData());
        }
      },
    ),
    _QuickActionModern(
      icon: Icons.person_search,
      label: loc.students,
      onTap: () {
        Navigator.of(context)
            .pushNamed('/students')
            .then((_) => refreshData());
      },
    ),
  ],
),
const SizedBox(height: 32),
// Recent Activity
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
```

```
  children: [
    Text(loc.recentScans,
        style: const TextStyle(
            fontSize: 18, fontWeight: FontWeight.bold)),
    TextButton(
      onPressed: () {},
      child: Text(loc.viewAll,
          style: TextStyle(
              color: Theme.of(context).primaryColor,
              fontWeight: FontWeight.bold)),
    ),
  ],
),
const SizedBox(height: 12),
FutureBuilder<List<Map<String, dynamic>>>(
  future: _recentScansFuture,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(
          child: Padding(
        padding: EdgeInsets.all(20.0),
        child: CircularProgressIndicator(),
      ));
    }
    final scans = snapshot.data ?? [];
    if (scans.isEmpty) {
      return Container(
        padding: const EdgeInsets.symmetric(vertical: 20),
        child: Center(
          child: Text(
            loc.noScansYet,
            style: const TextStyle(
                color: Colors.grey, fontStyle: FontStyle.italic),
          ),
        ),
      );
    }
    return Column(
      children: scans.map((scan) {
        return _RecentExamCard(
          title: scan['exam_title'] ?? loc.unknownExam,
```

```
                    date: scan['date']?.toString().split('T').first ?? '',
                    status: loc.graded,
                    studentCount: scan['student_count'] ?? 0,
                  );
                }).toList(),
              );
            },
          ),
        ],
      ),
    ),
  ),
  );
}

Widget _buildProfileMenu(BuildContext context, AuthProvider auth) {
  final loc = AppLocalizations.of(context)!;
  final localeProvider = Provider.of<LocaleProvider>(context, listen: false);
  final currentLocale = localeProvider.locale ?? const Locale('en');

  return PopupMenuButton<String>(
    offset: const Offset(0, 50),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
    icon: Container(
      decoration: BoxDecoration(
        shape: BoxShape.circle,
        border: Border.all(color: Colors.blue.withOpacity(0.3), width: 2),
      ),
      child: CircleAvatar(
        radius: 18,
        backgroundColor: Colors.blue.withOpacity(0.1),
        backgroundImage: auth.user?.photoURL != null
            ? NetworkImage(auth.user!.photoURL!)
            : null,
        child: auth.user?.photoURL == null
            ? const Icon(Icons.person, size: 20, color: Colors.blue)
            : null,
      ),
    ),
    onSelected: (value) async {
      if (value == 'logout') {
```

```
    auth.signOut();
  } else if (value == 'en') {
    localeProvider.setLocale(const Locale('en'));
  } else if (value == 'ar') {
    localeProvider.setLocale(const Locale('ar'));
  }
},
itemBuilder: (context) => [
  PopupMenuItem(
    enabled: false,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          auth.user?.displayName ?? 'Teacher',
          style: const TextStyle(
              fontWeight: FontWeight.bold,
              color: Colors.black,
              fontSize: 16),
        ),
        Text(
          auth.user?.email ?? '',
          style: const TextStyle(color: Colors.grey, fontSize: 12),
        ),
        const Divider(),
      ],
    ),
  ),
  PopupMenuItem(
    value: 'en',
    child: Row(
      children: [
        const Text('  ', style: TextStyle(fontSize: 18)),
        const SizedBox(width: 12),
        const Text('English'),
        const Spacer(),
        if (currentLocale.languageCode == 'en')
          const Icon(Icons.check, color: Colors.blue, size: 18),
      ],
    ),
  ),
```

```
        PopupMenuItem(
          value: 'ar',
          child: Row(
            children: [
              const Text('  ', style: TextStyle(fontSize: 18)),
              const SizedBox(width: 12),
              const Text(',('العربية
              const Spacer(),
              if (currentLocale.languageCode == 'ar')
                const Icon(Icons.check, color: Colors.blue, size: 18),
            ],
          ),
        ),
        const PopupMenuDivider(),
        PopupMenuItem(
          value: 'logout',
          child: Row(
            children: [
              Icon(Icons.logout_rounded, color: Colors.red.shade400, size: 20),
              const SizedBox(width: 12),
              Text(loc.logout, style: TextStyle(color: Colors.red.shade400)),
            ],
          ),
        ),
      ],
    );
  }
}

class _SummaryCardModern extends StatelessWidget {
  final IconData icon;
  final String value;
  final String label;
  final Color color;
  const _SummaryCardModern(
      {required this.icon,
      required this.value,
      required this.label,
      required this.color});

  @override
```

```
Widget build(BuildContext context) {
  return Container(
    width: 140,
    margin: const EdgeInsets.only(right: 16),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(24),
      boxShadow: const [
        BoxShadow(
          color: Colors.black12,
          blurRadius: 10,
          offset: Offset(0, 4),
        ),
      ],
      border: Border(
        left: BorderSide(color: color, width: 6),
      ),
    ),
    padding: const EdgeInsets.symmetric(vertical: 8, horizontal: 10),
    clipBehavior: Clip.hardEdge,
    child: Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        mainAxisAlignment: MainAxisAlignment.center,
        crossAxisAlignment: CrossAxisAlignment.center,
        children: [
          Icon(icon, color: color, size: 28),
          const SizedBox(height: 6),
          Text(value,
              style:
                  const TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
          const SizedBox(height: 2),
          Text(label,
              style: const TextStyle(
                  fontSize: 12,
                  color: Colors.grey,
                  fontWeight: FontWeight.w500)),
        ],
      ),
    ),
  );
```

```
  }
}

class _QuickActionModern extends StatelessWidget {
  final IconData icon;
  final String label;
  final VoidCallback onTap;
  const _QuickActionModern(
      {required this.icon, required this.label, required this.onTap});

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Theme.of(context).primaryColor,
      borderRadius: BorderRadius.circular(24),
      child: InkWell(
        borderRadius: BorderRadius.circular(24),
        onTap: onTap,
        child: Container(
          height: 100,
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Icon(icon, color: Colors.white, size: 32),
              const SizedBox(height: 10),
              Text(label,
                  style: const TextStyle(
                      color: Colors.white, fontWeight: FontWeight.w600)),
            ],
          ),
        ),
      ),
    );
  }
}

class _RecentExamCard extends StatelessWidget {
  final String title;
  final String date;
  final String status;
  final int studentCount;
```

```
  const _RecentExamCard(
      {required this.title,
      required this.date,
      required this.status,
      required this.studentCount});

  @override
  Widget build(BuildContext context) {
    final loc = AppLocalizations.of(context)!;
    Color statusColor =
        status == 'Graded' ? const Color(0xFF43A047) : const Color(0xFFFB8C00);
    return Card(
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
      margin: const EdgeInsets.only(bottom: 12),
      elevation: 2,
      child: ListTile(
        leading: Icon(Icons.description, color: Theme.of(context).primaryColor),
        title: Text(title, style: const TextStyle(fontWeight: FontWeight.w600)),
        subtitle: Text(date),
        trailing: Container(
          padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
          decoration: BoxDecoration(
            color: Colors.blue.withOpacity(0.1),
            borderRadius: BorderRadius.circular(12),
          ),
          child: Text(
            loc.sheetsCountLabel(studentCount),
            style: const TextStyle(
                color: Colors.blue, fontSize: 12, fontWeight: FontWeight.bold),
          ),import 'package:flutter/material.dart';
Import 'package:omr1/utils/omr_pdf_print.dart';
Import 'package:flutter_gen/gen_l10n/app_localizations.dart'; // Localization import
Import 'package:document_file_save_plus/document_file_save_plus.dart';
Import 'package:file_picker/file_picker.dart';
Import 'dart:io';
Import 'dart:typed_data';
Import 'package:open_file/open_file.dart';
Import 'package:omr1/db/database_helper.dart';
Import 'package:omr1/models/exam.dart';
Import 'package:omr1/models/question.dart';
Import 'package:provider/provider.dart';
```

```
Import ‘package:omr1/providers/auth_provider.dart’;
Import ‘package:omr1/providers/sync_provider.dart’;
Import ‘edit_exam_screen.dart’;

Class CreateExamScreen extends StatefulWidget {
  Final Function(int)? onTabChange;
  Const CreateExamScreen({super.key, this.onTabChange});

  @override
  State<CreateExamScreen> createState() => CreateExamScreenState();
}

Class CreateExamScreenState extends State<CreateExamScreen> {
  Final _formKey = GlobalKey<FormState>();
  Final GlobalKey<ScaffoldMessengerState> _scaffoldMessengerKey =
      GlobalKey<ScaffoldMessengerState>();
  Int _step = 0;
  String? _title;
  String? _subject;
  Int _numQuestions = 10;
  Int _numChoices = 5;
  Int _idDigits = 6; // Default student ID digits
  List<String?> _answerKey = List.filled(10, null);
  Bool _manualEntry = true;
  List<TextEditingController> _marksControllers =
      List.generate(10, (_) => TextEditingController(text: ‘1’));

  List<String> get _choiceLabels =>
      List.generate(_numChoices, (i) => String.fromCharCode(65 + i));

  // Existing exams list
  List<Exam> _existingExams = [];

  @override
  Void didChangeDependencies() {
    Super.didChangeDependencies();
    loadExistingExams();
  }

  Future<void> loadExistingExams() async {
    Final auth = Provider.of<AuthProvider>(context, listen: false);
```

```
  Final exams = await DatabaseHelper().getAllExams(auth.userId);
  If (mounted) setState(() => _existingExams = exams);
}

@override
Void dispose() {
  For (final c in _marksControllers) {
    c.dispose();
  }
  Super.dispose();
}

@override
Widget build(BuildContext context) {
  Final loc = AppLocalizations.of(context)!; // Access localizations
  Return ScaffoldMessenger(
    Key: _scaffoldMessengerKey,
    Child: PopScope(
      canPop: Navigator.of(context).canPop(),
      onPopInvoked: (didPop) {
        if (didPop) return;
        if (widget.onTabChange != null) {
          widget.onTabChange!(0);
        }
      },
      Child: Scaffold(
      backgroundColor: const Color(0xFFF4F6FB),
      appBar: AppBar(
        backgroundColor: Colors.white,
        elevation: 2,
        iconTheme: const IconThemeData(color: Color(0xFF007BFF)),
        title: Text(loc.createExam,
            style: const TextStyle(
                color: Colors.black, fontWeight: FontWeight.bold)),
        centerTitle: true,
        leading: !Navigator.of(context).canPop()
            ? IconButton(
                Icon: const Icon(Icons.arrow_back),
                onPressed: () {
                  if (widget.onTabChange != null) {
                    widget.onTabChange!(0);
```

```
                  }
                },
              )
            : null,
      ),
      Body: SafeArea(
        Child: SingleChildScrollView(
          Padding: EdgeInsets.only(
              Bottom: MediaQuery.of(context).viewInsets.bottom),
          Child: Center(
            Child: Container(
              Constraints: const BoxConstraints(maxWidth: 700),
              Padding:
                  Const EdgeInsets.symmetric(vertical: 32, horizontal: 16),
              Child: Column(
                Children: [
                  Card(
                    Elevation: 8,
                    Shape: RoundedRectangleBorder(
                        borderRadius: BorderRadius.circular(32)),
                    child: Padding(
                      padding: const EdgeInsets.all(32),
                      child: _step == 0
                          ? _buildExamDetails(loc)
                          : _buildAnswerKey(loc),
                    ),
                  ),
                  Const SizedBox(height: 24),
                  // ── Existing Exams List ──
                  If (_existingExams.isNotEmpty) _buildExistingExamsList(loc),
                ],
              ),
            ),
          ),
        ),
      ),
    ),
  );
}
```

```
Widget _buildExamDetails(AppLocalizations loc) {
  Return Form(
    Key: _formKey,
    Child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      mainAxisSize: MainAxisSize.min,
      children: [
        Text(loc.examTitle,
            Style:
                Const TextStyle(fontSize: 26, fontWeight: FontWeight.bold)),
        Const SizedBox(height: 24),
        TextFormField(
          Decoration: InputDecoration(
            labelText: loc.examTitle,
            border:
                OutlineInputBorder(borderRadius: BorderRadius.circular(24)),
          ),
          onChanged: (v) => _title = v,
          validator: (v) =>
              v == null || v.trim().isEmpty ? loc.examTitle : null,
        ),
        Const SizedBox(height: 20),
        DropdownButtonFormField<String>(
          Decoration: InputDecoration(
            labelText: loc.subject,
            border:
                OutlineInputBorder(borderRadius: BorderRadius.circular(24)),
          ),
          Items: [
            DropdownMenuItem(value: 'Math', child: Text(loc.math)),
            DropdownMenuItem(value: 'Science', child: Text(loc.science)),
            DropdownMenuItem(value: 'History', child: Text(loc.history)),
            DropdownMenuItem(value: 'English', child: Text(loc.english)),
            If (_subject != null &&
                !_subject!.isEmpty &&
                !_isDuplicateSubject(_subject!))
              DropdownMenuItem(value: _subject, child: Text(_subject!)),
            DropdownMenuItem(
                Value: 'add_new', child: Text('+ ${loc.subject}')),
          ],
          Value: _subject,
```

```
  onChanged: (v) async {
    if (v == 'add_new') {
      final newSubject = await showDialog<String>(
        context: context,
        builder: (context) {
          String? tempSubject;
          Return AlertDialog(
            Shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(24)),
            title: Text(loc.subject),
            content: TextField(
              autofocus: true,
              decoration: InputDecoration(labelText: loc.subject),
              onChanged: (val) => tempSubject = val,
            ),
            Actions: [
              TextButton(
                onPressed: () => Navigator.of(context).pop(),
                child: Text(loc.back),
              ),
              ElevatedButton(
                onPressed: () =>
                    Navigator.of(context).pop(tempSubject),
                Child: Text(loc.next),
              ),
            ],
          );
        },
      );
      If (newSubject != null && newSubject.trim().isNotEmpty) {
        setState(() {
          _subject = newSubject.trim();
        });
      }
    } else {
      setState(() => _subject = v);
    }
  },
  Validator: (v) => v == null || v.isEmpty ? loc.subject : null,
),
Const SizedBox(height: 20),
```

```
Row(
  Children: [
    Expanded(
      Child: Text('${loc.numQuestions}: $_numQuestions',
          Style: const TextStyle(fontSize: 16)),
    ),
    IconButton(
      Icon:
          Icon(Icons.remove_circle_outline, color: Color(0xFF007BFF)),
      onPressed: _numQuestions > 1
          ? () => setState(() {
                _numQuestions--;
                If (_answerKey.length > _numQuestions) {
                  _answerKey = _answerKey.sublist(0, _numQuestions);
                }
                If (_marksControllers.length > _numQuestions) {
                  _marksControllers =
                      _marksControllers.sublist(0, _numQuestions);
                }
              })
          : null,
    ),
    IconButton(
      Icon: Icon(Icons.add_circle_outline, color: Color(0xFF007BFF)),
      onPressed: () => setState(() {
        _numQuestions++;
        If (_answerKey.length < _numQuestions) {
          _answerKey = List<String?>.from(_answerKey)..add(null);
        }
        If (_marksControllers.length < _numQuestions) {
          _marksControllers.add(TextEditingController(text: '1'));
        }
      }),
    ),
  ],
),
Const SizedBox(height: 12),
Row(
  Children: [
    Expanded(
      Child: Text('${loc.numChoices}: $_numChoices',
```

```
            Style: const TextStyle(fontSize: 16)),
      ),
      IconButton(
        Icon:
            Icon(Icons.remove_circle_outline, color: Color(0xFF007BFF)),
        onPressed: _numChoices > 2
            ? () => setState(() {
                  _numChoices--;
                })
            : null,
      ),
      IconButton(
        Icon: Icon(Icons.add_circle_outline, color: Color(0xFF007BFF)),
        onPressed: _numChoices < 8
            ? () => setState(() {
                  _numChoices++;
                })
            : null,
      ),
    ],
  ),
  Const SizedBox(height: 20),
  Row(
    Children: [
      Expanded(
        Child: Text('${loc.studentIdDigits}: $_idDigits',
            Style: const TextStyle(fontSize: 16)),
      ),
      IconButton(
        Icon:
            Icon(Icons.remove_circle_outline, color: Color(0xFF007BFF)),
        onPressed: _idDigits > 3
            ? () => setState(() {
                  _idDigits--;
                })
            : null,
      ),
      IconButton(
        Icon: Icon(Icons.add_circle_outline, color: Color(0xFF007BFF)),
        onPressed: _idDigits < 12
            ? () => setState(() {
```

```
                        _idDigits++;
                      })
                  : null,
            ),
          ],
        ),
        Const SizedBox(height: 32),
        Align(
          Alignment: Alignment.centerRight,
          Child: ElevatedButton(
            onPressed: () {
              if (_formKey.currentState?.validate() ?? false) {
                setState(() => _step = 1);
              }
            },
            Style: ElevatedButton.styleFrom(
              backgroundColor: const Color(0xFF007BFF),
              shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(24)),
              padding:
                  const EdgeInsets.symmetric(horizontal: 32, vertical: 14),
            ),
            Child: Text(loc.next, style: const TextStyle(fontSize: 16)),
          ),
        ),
      ],
    ),
  );
}

Widget _buildAnswerKey(AppLocalizations loc) {
  Return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    mainAxisSize: MainAxisSize.min,
    children: [
      Row(
        Children: [
          IconButton(
            Icon: const Icon(Icons.arrow_back, color: Color(0xFF007BFF)),
            onPressed: () => setState(() => _step = 0),
          ),
```

```
    Const SizedBox(width: 8),
    Text(loc.answerKey,
        Style:
            Const TextStyle(fontSize: 26, fontWeight: FontWeight.bold)),
  ],
),
Const SizedBox(height: 24),
Row(
  Children: [
    Expanded(
      Child: ChoiceChip(
        Label: Text(loc.manualEntry, overflow: TextOverflow.ellipsis),
        Selected: _manualEntry,
        onSelected: (v) => setState(() => _manualEntry = true),
        selectedColor: const Color(0xFF007BFF),
        labelStyle: TextStyle(
            color: _manualEntry ? Colors.white : Colors.black),
      ),
    ),
    Const SizedBox(width: 12),
    Expanded(
      Child: ChoiceChip(
        Label: Text(loc.autoEntry, overflow: TextOverflow.ellipsis),
        Selected: !_manualEntry,
        onSelected: (v) => setState(() => _manualEntry = false),
        selectedColor: const Color(0xFF007BFF),
        labelStyle: TextStyle(
            color: !_manualEntry ? Colors.white : Colors.black),
      ),
    ),
  ],
),
Const SizedBox(height: 16),
If (_manualEntry) …[
  ListView.builder(
    shrinkWrap: true,
    physics: const NeverScrollableScrollPhysics(),
    itemCount: _numQuestions,
    itemBuilder: (context, i) {
      return Padding(
        padding: const EdgeInsets.symmetric(vertical: 4),
```

```
  child: Row(
    children: [
      Text('Q${I + 1}',
          Style: const TextStyle(fontWeight: FontWeight.w600)),
      Const SizedBox(width: 12),
      Expanded(
        Child: Wrap(
          Spacing: 8,
          Children: _choiceLabels.map((opt) {
            Final selected = _answerKey[i] == opt;
            Return ChoiceChip(
              Label: Text(opt),
              Selected: selected,
              selectedColor: const Color(0xFF007BFF),
              labelStyle: TextStyle(
                  color: selected ? Colors.white : Colors.black),
              onSelected: (_) =>
                  setState(() => _answerKey[i] = opt),
            );
          }).toList(),
        ),
      ),
      Const SizedBox(width: 12),
      SizedBox(
        Width: 60,
        Child: TextField(
          Controller: _marksControllers[i],
          keyboardType: TextInputType.number,
          decoration: InputDecoration(
            labelText: loc.mark,
            isDense: true,
            border: OutlineInputBorder(
                borderRadius: BorderRadius.circular(12)),
            contentPadding: const EdgeInsets.symmetric(
                horizontal: 8, vertical: 8),
          ),
        ),
      ),
    ],
  ),
);
```

```
    },
  ),
  If (_answerKey.any((a) => a == null))
    Padding(
      Padding: const EdgeInsets.only(top: 8.0),
      Child: Text(loc.answerKey,
          Style: const TextStyle(color: Colors.red)),
    ),
] else …[
  Container(
    Height: 120,
    Decoration: BoxDecoration(
      Color: Colors.grey[100],
      borderRadius: BorderRadius.circular(20),
      border: Border.all(color: Colors.grey[300]!),
    ),
    Child: Center(
      Child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Icon(Icons.upload_file,
              size: 36, color: Color(0xFF007BFF)),
          const SizedBox(height: 8),
          Text(loc.autoEntry,
              Style: TextStyle(color: Colors.grey[700])),
        ],
      ),
    ),
  ),
],
Const SizedBox(height: 32),
Align(
  Alignment: Alignment.centerRight,
  Child: ElevatedButton(
    onPressed: () async {
      if (_manualEntry && _answerKey.any((a) => a == null)) {
        setState(() {});
        return;
      }

      // Show loading dialog
```

```
showDialog(
  context: context,
  barrierDismissible: false,
  builder: (context) => const Center(child: CircularProgressIndicator()),
);

Try {
  // Save exam and questions to database
  Final db = DatabaseHelper();
  Final auth = Provider.of<AuthProvider>(context, listen: false);
  Final exam = Exam(
    Title: _title ?? ‘’,
    Subject: _subject ?? ‘’,
    Date: DateTime.now().toIso8601String(),
    numQuestions: _numQuestions,
    numChoices: _numChoices,
    answerKey: ‘’,
    userId: auth.userId,
  );
  Final examId = await db.insertExam(exam);
  For (int I = 0; I < _numQuestions; i++) {
    Final correctChoice = _choiceLabels.indexOf(_answerKey[i] ?? ‘’);
    Final mark = int.tryParse(_marksControllers[i].text) ?? 1;
    Final question = Question(
      examId: examId,
      questionNumber: I + 1,
      correctChoice: correctChoice,
      mark: mark,
    );
    Await db.insertQuestion(question);
  }

  // Generate and save PDF
  String fileName = ‘${_title ?? ‘exam’}.pdf’;
  Final pdfBytes = await printOmrExamPaper(
    Context: context,
    examTitle: _title ?? ‘’,
    idDigits: _idDigits,
    numQuestions: _numQuestions,
    numChoices: _numChoices,
  );
```

```
                If (pdfBytes.isNotEmpty) {
                  If (Theme.of(context).platform == TargetPlatform.android) {
                    Await DocumentFileSavePlus().saveFile(
                      Uint8List.fromList(pdfBytes),
                      fileName,
                      “application/pdf”,
                    );
                    _scaffoldMessengerKey.currentState?.showSnackBar(
                      SnackBar(content: Text(loc.pdfSaved)),
                    );
                  } else {
                    String? outputPath = await FilePicker.platform.saveFile(
                      dialogTitle: loc.chooseSaveLocation,
                      fileName: fileName,
                      type: FileType.custom,
                      allowedExtensions: [‘pdf’],
                    );
                    If (outputPath != null) {
                      Final file = File(outputPath);
                      Await file.writeAsBytes(pdfBytes);
                      _scaffoldMessengerKey.currentState?.showSnackBar(
                        SnackBar(
                            Content: Text(‘${loc.pdfSavedTo} $out
        ),
      ),
    );
  }
}
```

End home dashboard screen code

Create exam screen code

```
import 'package:flutter/material.dart';
import 'package:omr1/utils/omr_pdf_print.dart';
import 'package:flutter_gen/gen_l10n/app_localizations.dart'; // Localization import
import 'package:document_file_save_plus/document_file_save_plus.dart';
import 'package:file_picker/file_picker.dart';
import 'dart:io';
import 'dart:typed_data';
```

```
import 'package:open_file/open_file.dart';
import 'package:omr1/db/database_helper.dart';
import 'package:omr1/models/exam.dart';
import 'package:omr1/models/question.dart';
import 'package:provider/provider.dart';
import 'package:omr1/providers/auth_provider.dart';
import 'package:omr1/providers/sync_provider.dart';
import 'edit_exam_screen.dart';

class CreateExamScreen extends StatefulWidget {
  final Function(int)? onTabChange;
  const CreateExamScreen({super.key, this.onTabChange});

  @override
  State<CreateExamScreen> createState() => CreateExamScreenState();
}

class CreateExamScreenState extends State<CreateExamScreen> {
  final _formKey = GlobalKey<FormState>();
  final GlobalKey<ScaffoldMessengerState> _scaffoldMessengerKey =
      GlobalKey<ScaffoldMessengerState>();
  int _step = 0;
  String? _title;
  String? _subject;
  int _numQuestions = 10;
  int _numChoices = 5;
  int _idDigits = 6; // Default student ID digits
  List<String?> _answerKey = List.filled(10, null);
  bool _manualEntry = true;
  List<TextEditingController> _marksControllers =
      List.generate(10, (_) => TextEditingController(text: '1'));

  List<String> get _choiceLabels =>
      List.generate(_numChoices, (i) => String.fromCharCode(65 + i));

  // Existing exams list
  List<Exam> _existingExams = [];

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
```

```
  loadExistingExams();
}

Future<void> loadExistingExams() async {
  final auth = Provider.of<AuthProvider>(context, listen: false);
  final exams = await DatabaseHelper().getAllExams(auth.userId);
  if (mounted) setState(() => _existingExams = exams);
}

@override
void dispose() {
  for (final c in _marksControllers) {
    c.dispose();
  }
  super.dispose();
}

@override
Widget build(BuildContext context) {
  final loc = AppLocalizations.of(context)!; // Access localizations
  return ScaffoldMessenger(
    key: _scaffoldMessengerKey,
    child: PopScope(
      canPop: Navigator.of(context).canPop(),
      onPopInvoked: (didPop) {
        if (didPop) return;
        if (widget.onTabChange != null) {
          widget.onTabChange!(0);
        }
      },
      child: Scaffold(
      backgroundColor: const Color(0xFFF4F6FB),
      appBar: AppBar(
        backgroundColor: Colors.white,
        elevation: 2,
        iconTheme: const IconThemeData(color: Color(0xFF007BFF)),
        title: Text(loc.createExam,
            style: const TextStyle(
                color: Colors.black, fontWeight: FontWeight.bold)),
        centerTitle: true,
        leading: !Navigator.of(context).canPop()
```

```
      ? IconButton(
          icon: const Icon(Icons.arrow_back),
          onPressed: () {
            if (widget.onTabChange != null) {
              widget.onTabChange!(0);
            }
          },
        )
      : null,
),
body: SafeArea(
  child: SingleChildScrollView(
    padding: EdgeInsets.only(
        bottom: MediaQuery.of(context).viewInsets.bottom),
    child: Center(
      child: Container(
        constraints: const BoxConstraints(maxWidth: 700),
        padding:
            const EdgeInsets.symmetric(vertical: 32, horizontal: 16),
        child: Column(
          children: [
            Card(
              elevation: 8,
              shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(32)),
              child: Padding(
                padding: const EdgeInsets.all(32),
                child: _step == 0
                    ? _buildExamDetails(loc)
                    : _buildAnswerKey(loc),
              ),
            ),
            const SizedBox(height: 24),
            // ── Existing Exams List ──
            if (_existingExams.isNotEmpty) _buildExistingExamsList(loc),
          ],
        ),
      ),
    ),
  ),
),
```

```
      ),
    ),
  );
}

  Widget _buildExamDetails(AppLocalizations loc) {
    return Form(
      key: _formKey,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        mainAxisSize: MainAxisSize.min,
        children: [
          Text(loc.examTitle,
              style:
                  const TextStyle(fontSize: 26, fontWeight: FontWeight.bold)),
          const SizedBox(height: 24),
          TextFormField(
            decoration: InputDecoration(
              labelText: loc.examTitle,
              border:
                  OutlineInputBorder(borderRadius: BorderRadius.circular(24)),
            ),
            onChanged: (v) => _title = v,
            validator: (v) =>
                v == null || v.trim().isEmpty ? loc.examTitle : null,
          ),
          const SizedBox(height: 20),
          DropdownButtonFormField<String>(
            decoration: InputDecoration(
              labelText: loc.subject,
              border:
                  OutlineInputBorder(borderRadius: BorderRadius.circular(24)),
            ),
            items: [
              DropdownMenuItem(value: 'Math', child: Text(loc.math)),
              DropdownMenuItem(value: 'Science', child: Text(loc.science)),
              DropdownMenuItem(value: 'History', child: Text(loc.history)),
              DropdownMenuItem(value: 'English', child: Text(loc.english)),
              if (_subject != null &&
                  !_subject!.isEmpty &&
                  !_isDuplicateSubject(_subject!))
```

```
    DropdownMenuItem(value: _subject, child: Text(_subject!)),
  DropdownMenuItem(
      value: 'add_new', child: Text('+ ${loc.subject}')),
],
value: _subject,
onChanged: (v) async {
  if (v == 'add_new') {
    final newSubject = await showDialog<String>(
      context: context,
      builder: (context) {
        String? tempSubject;
        return AlertDialog(
          shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(24)),
          title: Text(loc.subject),
          content: TextField(
            autofocus: true,
            decoration: InputDecoration(labelText: loc.subject),
            onChanged: (val) => tempSubject = val,
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.of(context).pop(),
              child: Text(loc.back),
            ),
            ElevatedButton(
              onPressed: () =>
                  Navigator.of(context).pop(tempSubject),
              child: Text(loc.next),
            ),
          ],
        );
      },
    );
    if (newSubject != null && newSubject.trim().isNotEmpty) {
      setState(() {
        _subject = newSubject.trim();
      });
    }
  } else {
    setState(() => _subject = v);
```

```
    }
  },
  validator: (v) => v == null || v.isEmpty ? loc.subject : null,
),
const SizedBox(height: 20),
Row(
  children: [
    Expanded(
      child: Text('${loc.numQuestions}: $_numQuestions',
          style: const TextStyle(fontSize: 16)),
    ),
    IconButton(
      icon:
          Icon(Icons.remove_circle_outline, color: Color(0xFF007BFF)),
      onPressed: _numQuestions > 1
          ? () => setState(() {
                _numQuestions--;
                if (_answerKey.length > _numQuestions) {
                  _answerKey = _answerKey.sublist(0, _numQuestions);
                }
                if (_marksControllers.length > _numQuestions) {
                  _marksControllers =
                      _marksControllers.sublist(0, _numQuestions);
                }
              })
          : null,
    ),
    IconButton(
      icon: Icon(Icons.add_circle_outline, color: Color(0xFF007BFF)),
      onPressed: () => setState(() {
        _numQuestions++;
        if (_answerKey.length < _numQuestions) {
          _answerKey = List<String?>.from(_answerKey)..add(null);
        }
        if (_marksControllers.length < _numQuestions) {
          _marksControllers.add(TextEditingController(text: '1'));
        }
      }),
    ),
  ],
),
```

```
const SizedBox(height: 12),
Row(
  children: [
    Expanded(
      child: Text('${loc.numChoices}: $_numChoices',
          style: const TextStyle(fontSize: 16)),
    ),
    IconButton(
      icon:
          Icon(Icons.remove_circle_outline, color: Color(0xFF007BFF)),
      onPressed: _numChoices > 2
          ? () => setState(() {
                _numChoices--;
              })
          : null,
    ),
    IconButton(
      icon: Icon(Icons.add_circle_outline, color: Color(0xFF007BFF)),
      onPressed: _numChoices < 8
          ? () => setState(() {
                _numChoices++;
              })
          : null,
    ),
  ],
),
const SizedBox(height: 20),
Row(
  children: [
    Expanded(
      child: Text('${loc.studentIdDigits}: $_idDigits',
          style: const TextStyle(fontSize: 16)),
    ),
    IconButton(
      icon:
          Icon(Icons.remove_circle_outline, color: Color(0xFF007BFF)),
      onPressed: _idDigits > 3
          ? () => setState(() {
                _idDigits--;
              })
          : null,
```

```
            ),
            IconButton(
              icon: Icon(Icons.add_circle_outline, color: Color(0xFF007BFF)),
              onPressed: _idDigits < 12
                  ? () => setState(() {
                        _idDigits++;
                      })
                  : null,
            ),
          ],
        ),
        const SizedBox(height: 32),
        Align(
          alignment: Alignment.centerRight,
          child: ElevatedButton(
            onPressed: () {
              if (_formKey.currentState?.validate() ?? false) {
                setState(() => _step = 1);
              }
            },
            style: ElevatedButton.styleFrom(
              backgroundColor: const Color(0xFF007BFF),
              shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(24)),
              padding:
                  const EdgeInsets.symmetric(horizontal: 32, vertical: 14),
            ),
            child: Text(loc.next, style: const TextStyle(fontSize: 16)),
          ),
        ),
      ],
    ),
  );
}

Widget _buildAnswerKey(AppLocalizations loc) {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    mainAxisSize: MainAxisSize.min,
    children: [
      Row(
```

```
  children: [
    IconButton(
      icon: const Icon(Icons.arrow_back, color: Color(0xFF007BFF)),
      onPressed: () => setState(() => _step = 0),
    ),
    const SizedBox(width: 8),
    Text(loc.answerKey,
        style:
            const TextStyle(fontSize: 26, fontWeight: FontWeight.bold)),
  ],
),
const SizedBox(height: 24),
Row(
  children: [
    Expanded(
      child: ChoiceChip(
        label: Text(loc.manualEntry, overflow: TextOverflow.ellipsis),
        selected: _manualEntry,
        onSelected: (v) => setState(() => _manualEntry = true),
        selectedColor: const Color(0xFF007BFF),
        labelStyle: TextStyle(
            color: _manualEntry ? Colors.white : Colors.black),
      ),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: ChoiceChip(
        label: Text(loc.autoEntry, overflow: TextOverflow.ellipsis),
        selected: !_manualEntry,
        onSelected: (v) => setState(() => _manualEntry = false),
        selectedColor: const Color(0xFF007BFF),
        labelStyle: TextStyle(
            color: !_manualEntry ? Colors.white : Colors.black),
      ),
    ),
  ],
),
const SizedBox(height: 16),
if (_manualEntry) ...[
  ListView.builder(
    shrinkWrap: true,
```

```
physics: const NeverScrollableScrollPhysics(),
itemCount: _numQuestions,
itemBuilder: (context, i) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 4),
    child: Row(
      children: [
        Text('Q${i + 1}',
            style: const TextStyle(fontWeight: FontWeight.w600)),
        const SizedBox(width: 12),
        Expanded(
          child: Wrap(
            spacing: 8,
            children: _choiceLabels.map((opt) {
              final selected = _answerKey[i] == opt;
              return ChoiceChip(
                label: Text(opt),
                selected: selected,
                selectedColor: const Color(0xFF007BFF),
                labelStyle: TextStyle(
                    color: selected ? Colors.white : Colors.black),
                onSelected: (_) =>
                    setState(() => _answerKey[i] = opt),
              );
            }).toList(),
          ),
        ),
        const SizedBox(width: 12),
        SizedBox(
          width: 60,
          child: TextField(
            controller: _marksControllers[i],
            keyboardType: TextInputType.number,
            decoration: InputDecoration(
              labelText: loc.mark,
              isDense: true,
              border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(12)),
              contentPadding: const EdgeInsets.symmetric(
                  horizontal: 8, vertical: 8),
            ),
```

```
              ),
            ),
          ],
        ),
      );
    },
  ),
  if (_answerKey.any((a) => a == null))
    Padding(
      padding: const EdgeInsets.only(top: 8.0),
      child: Text(loc.answerKey,
          style: const TextStyle(color: Colors.red)),
    ),
] else ...[
  Container(
    height: 120,
    decoration: BoxDecoration(
      color: Colors.grey[100],
      borderRadius: BorderRadius.circular(20),
      border: Border.all(color: Colors.grey[300]!),
    ),
    child: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Icon(Icons.upload_file,
              size: 36, color: Color(0xFF007BFF)),
          const SizedBox(height: 8),
          Text(loc.autoEntry,
              style: TextStyle(color: Colors.grey[700])),
        ],
      ),
    ),
  ),
],
const SizedBox(height: 32),
Align(
  alignment: Alignment.centerRight,
  child: ElevatedButton(
    onPressed: () async {
      if (_manualEntry && _answerKey.any((a) => a == null)) {
```

```
  setState(() {});
  return;
}

// Show loading dialog
showDialog(
  context: context,
  barrierDismissible: false,
  builder: (context) => const Center(child: CircularProgressIndicator()),
);

try {
  // Save exam and questions to database
  final db = DatabaseHelper();
  final auth = Provider.of<AuthProvider>(context, listen: false);
  final exam = Exam(
    title: _title ?? '',
    subject: _subject ?? '',
    date: DateTime.now().toIso8601String(),
    numQuestions: _numQuestions,
    numChoices: _numChoices,
    answerKey: '',
    userId: auth.userId,
  );
  final examId = await db.insertExam(exam);
  for (int i = 0; i < _numQuestions; i++) {
    final correctChoice = _choiceLabels.indexOf(_answerKey[i] ?? '');
    final mark = int.tryParse(_marksControllers[i].text) ?? 1;
    final question = Question(
      examId: examId,
      questionNumber: i + 1,
      correctChoice: correctChoice,
      mark: mark,
    );
    await db.insertQuestion(question);
  }

  // Generate and save PDF
  String fileName = '${_title ?? 'exam'}.pdf';
  final pdfBytes = await printOmrExamPaper(
    context: context,
```

```
  examTitle: _title ?? '',
  idDigits: _idDigits,
  numQuestions: _numQuestions,
  numChoices: _numChoices,
);

if (pdfBytes.isNotEmpty) {
  if (Theme.of(context).platform == TargetPlatform.android) {
    await DocumentFileSavePlus().saveFile(
      Uint8List.fromList(pdfBytes),
      fileName,
      "application/pdf",
    );
    _scaffoldMessengerKey.currentState?.showSnackBar(
      SnackBar(content: Text(loc.pdfSaved)),
    );
  } else {
    String? outputPath = await FilePicker.platform.saveFile(
      dialogTitle: loc.chooseSaveLocation,
      fileName: fileName,
      type: FileType.custom,
      allowedExtensions: ['pdf'],
    );
    if (outputPath != null) {
      final file = File(outputPath);
      await file.writeAsBytes(pdfBytes);
      _scaffoldMessengerKey.currentState?.showSnackBar(
        SnackBar(
            content: Text('${loc.pdfSavedTo} $outputPath'),
            action: SnackBarAction(
                label: loc.view,
                onPressed: () => OpenFile.open(outputPath))),
      );
    }
  }
}

final syncProv = Provider.of<SyncProvider>(context, listen: false);
final userId = auth.userId;
// Directly push the new exam to Firestore immediately (awaited)
if (userId != null) {
```

```
                try {
                  await syncProv.syncExamDirectly(userId, examId);
                } catch (e) {
                  debugPrint('Firestore sync failed after exam creation: $e');
                  // Non-fatal — exam is saved locally; background sync will retry
                  _scaffoldMessengerKey.currentState?.showSnackBar(
                    SnackBar(
                      content: Text('Saved locally. Cloud sync failed: $e'),
                      backgroundColor: Colors.orange,
                      duration: const Duration(seconds: 5),
                    ),
                  );
                }
              }
              // Also kick off full background sync for other data
              syncProv.autoSync(userId);

              if (mounted) {
                Navigator.of(context).pop(); // Dismiss loading dialog

                _scaffoldMessengerKey.currentState?.showSnackBar(
                  SnackBar(content: Text(loc.examSaved)),
                );

                if (Navigator.of(context).canPop()) {
                  Navigator.of(context).pop(); // Return to previous screen
                } else {
                  // Reset form for the tab view
                  setState(() {
                    _step = 0;
                    _title = null;
                    _subject = null;
                    _numQuestions = 10;
                    _numChoices = 5;
                    _idDigits = 6;
                    _answerKey = List.filled(10, null);
                    _marksControllers = List.generate(10, (_) => TextEditingController(text:
'1'));
                  });
                  loadExistingExams();
                }
```

```
              }
            } catch (e) {
              if (mounted) Navigator.of(context).pop(); // Dismiss loading dialog
              _scaffoldMessengerKey.currentState?.showSnackBar(
                SnackBar(content: Text('Error: $e'), backgroundColor: Colors.red),
              );
            }
          },
          style: ElevatedButton.styleFrom(
            backgroundColor: const Color(0xFF007BFF),
            shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(24)),
            padding: const EdgeInsets.symmetric(horizontal: 32, vertical: 14),
          ),
          child: Text(loc.save, style: const TextStyle(fontSize: 16)),
        ),
      ),
    ],
  );
}

void _showA4PreviewDialog(BuildContext context) {
  showDialog(
    context: context,
    barrierColor: Colors.black.withOpacity(0.18),
    builder: (context) {
      return Dialog(
        shape:
            RoundedRectangleBorder(borderRadius: BorderRadius.circular(28)),
        insetPadding:
            const EdgeInsets.symmetric(horizontal: 12, vertical: 24),
        backgroundColor: Colors.transparent,
        child: Container(
          constraints: const BoxConstraints(maxWidth: 600, maxHeight: 800),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(28),
            boxShadow: [
              BoxShadow(
                color: Colors.black.withOpacity(0.10),
                blurRadius: 24,
```

```
      offset: const Offset(0, 8),
    ),
  ],
),
child: Column(
  mainAxisSize: MainAxisSize.min,
  children: [
    // Title Bar
    Padding(
      padding:
          const EdgeInsets.symmetric(horizontal: 24, vertical: 18),
      child: Row(
        children: [
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  _title ?? '',
                  style: const TextStyle(
                      fontSize: 20, fontWeight: FontWeight.bold),
                  overflow: TextOverflow.ellipsis,
                ),
                const SizedBox(height: 2),
                Text(
                  _subject ?? '',
                  style: const TextStyle(
                      fontSize: 15, color: Colors.black54),
                  overflow: TextOverflow.ellipsis,
                ),
              ],
            ),
          ),
          IconButton(
            icon: const Icon(Icons.close_rounded,
                size: 28, color: Colors.black54),
            splashRadius: 22,
            onPressed: () => Navigator.of(context).pop(),
          ),
        ],
      ),
```

```
),
const Divider(height: 1, thickness: 1),
// Main Content
Expanded(
  child: SingleChildScrollView(
    padding: const EdgeInsets.symmetric(
        horizontal: 18, vertical: 18),
    child: Center(
      child: Container(
        constraints: const BoxConstraints(maxWidth: 480),
        child: AspectRatio(
          aspectRatio: 210 / 297, // A4 aspect ratio
          child: Container(
            decoration: BoxDecoration(
              color: Colors.white,
              border: Border.all(
                  color: Colors.grey[300]!, width: 1.5),
              borderRadius: BorderRadius.circular(18),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withOpacity(0.04),
                  blurRadius: 8,
                  offset: const Offset(0, 2),
                ),
              ],
            ),
            padding: const EdgeInsets.symmetric(
                horizontal: 18, vertical: 18),
            child: SingleChildScrollView(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  // Student ID Section
                  LayoutBuilder(
                    builder: (context, constraints) {
                      return Builder(
                        builder: (context) {
                          final loc =
                              AppLocalizations.of(context)!;
                          double maxWidth =
                              constraints.maxWidth -
```

```
        90; // label + spacing
double minBubble = 14;
double maxBubble = 22;
double bubble =
    (_idDigits * (maxBubble + 8) >
            maxWidth)
        ? (maxWidth / _idDigits) - 8
        : maxBubble;
bubble = bubble.clamp(
    minBubble, maxBubble);
return Column(
  crossAxisAlignment:
      CrossAxisAlignment.start,
  children: [
    Row(
      crossAxisAlignment:
          CrossAxisAlignment.start,
      children: [
        Text(loc.studentId,
            style: const TextStyle(
                fontWeight:
                    FontWeight.w600)),
        const SizedBox(width: 12),
        Expanded(
          child: Wrap(
            spacing: 8,
            runSpacing: 2,
            children: List.generate(
                _idDigits,
                (d) => Column(
                      mainAxisSize:
                          MainAxisSize
                              .min,
                      children: [
                        for (int n =
                                0;
                            n < 10;
                            n++)
                          Padding(
                            padding: const EdgeInsets
                                .symmetric(
```

```
                                                    vertical:
                                                        1.2),
                                                child:
                                                    Container(
                                                  width:
                                                      bubble,
                                                  height:
                                                      bubble,
                                                  decoration:
                                                      BoxDecoration(
                                                    shape:
                                                        BoxShape.circle,
                                                    border: Border.all(
                                                     color: Colors.black26,
                                                        width: 1.1),
                                                    color:
                                                        Colors.white,
                                                  ),
                                                  child:
                                                      Center(
                                                    child: Text(
                                                        '$n',
                                              style: TextStyle(fontSize:
                                        bubble * 0.55, color: Colors.black54)),
                                                    ),
                                                  ),
                                                ),
                                              ],
                                            )),
                                      ),
                                    ),
                                  ],
                                ),
                              ],
                            );
                          },
                        );
                      },
                    ),
                    const SizedBox(height: 18),
                    // Questions Section
```

```
...List.generate(
    _numQuestions,
    (i) => Padding(
          padding: const EdgeInsets.symmetric(
              vertical: 7),
          child: Row(
            crossAxisAlignment:
                CrossAxisAlignment.center,
            children: [
              SizedBox(
                width: 36,
                child: Builder(
                  builder: (context) {
                    final loc =
                        AppLocalizations.of(
                            context)!;
                    return Text(
                        '${loc.question} ${i + 1}',
                        style: const TextStyle(
                            fontWeight:
                                FontWeight
                                    .w600));
                  },
                ),
              ),
              const SizedBox(width: 8),
              ..._choiceLabels.map((opt) =>
                  Padding(
                    padding: const EdgeInsets
                        .symmetric(
                        horizontal: 4),
                    child: Column(
                      children: [
                        Container(
                          width: 18,
                          height: 18,
                          decoration:
                              BoxDecoration(
                            shape: BoxShape
                                .circle,
                            border: Border.all(
```

```
                                                color: Colors
                                                    .black38,
                                                width: 1.2),
                                            color:
                                                Colors.white,
                                        ),
                                    ),
                                    Text(opt,
                                        style:
                                            const TextStyle(
                                                fontSize:
                                                    11)),
                                ],
                            ),
                        )),
                    const SizedBox(width: 10),
                    Builder(
                      builder: (context) {
                        final loc =
                            AppLocalizations.of(
                                context)!;
                        return Container(
                          padding: const EdgeInsets
                              .symmetric(
                              horizontal: 8,
                              vertical: 3),
                          decoration: BoxDecoration(
                            color: const Color(
                                    0xFF007BFF)
                                .withOpacity(0.09),
                            borderRadius:
                                BorderRadius
                                    .circular(7),
                          ),
                          child: Text(
'${loc.mark}: ${_marksControllers.length > i ? _marksControllers[i].text : '1'}',
                              style: const TextStyle(
                                  fontSize: 12,
                                  color: Color(
                                      0xFF007BFF))),
                        );
```

```
                                  },
                                ),
                              ],
                            ),
                          )),
                    ],
                  ),
                ),
              ),
            ),
          ),
        ),
      ),
      // Footer
      Container(
        padding:
            const EdgeInsets.symmetric(horizontal: 24, vertical: 18),
        decoration: BoxDecoration(
          color: Colors.white,
          borderRadius: const BorderRadius.vertical(
              bottom: Radius.circular(28)),
          boxShadow: [
            BoxShadow(
              color: Colors.black.withOpacity(0.03),
              blurRadius: 2,
              offset: const Offset(0, 1),
            ),
          ],
        ),
        child: Wrap(
          alignment: WrapAlignment.end,
          spacing: 12,
          runSpacing: 8,
          children: [
            Builder(
              builder: (context) {
                final loc = AppLocalizations.of(context)!;
                return OutlinedButton.icon(
                  icon: const Icon(Icons.picture_as_pdf,
                      color: Color(0xFF007BFF)),
```

```
label: Text(loc.exportPdf,
    style:
        const TextStyle(color: Color(0xFF007BFF))),
style: OutlinedButton.styleFrom(
  side: const BorderSide(color: Color(0xFF007BFF)),
  shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(18)),
  padding: const EdgeInsets.symmetric(
      horizontal: 22, vertical: 12),
),
onPressed: () async {
  String fileName = '${_title ?? 'exam'}.pdf';
  final pdfBytes = await printOmrExamPaper(
    context: context,
    examTitle: _title ?? '',
    idDigits: _idDigits,
    numQuestions: _numQuestions,
    numChoices: _numChoices,
  );
  if (pdfBytes.isEmpty) return;
  // Show dialog to let user edit filename before saving
  String? editedFileName = await showDialog<String>(
    context: context,
    builder: (dialogContext) {
      final controller =
          TextEditingController(text: fileName);
      return AlertDialog(
        shape: RoundedRectangleBorder(
            borderRadius:
                BorderRadius.circular(24)),
        title: Text(loc.exportPdf),
        content: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text(loc.chooseSaveLocation),
            const SizedBox(height: 12),
            TextField(
              controller: controller,
              autofocus: true,
              decoration: InputDecoration(
                labelText: loc.fileName,
```

```
                border: OutlineInputBorder(
                    borderRadius:
                        BorderRadius.circular(12)),
              ),
            ),
            const SizedBox(height: 8),
       // if (Theme.of(context).platform == TargetPlatform.android)
            //   Text(
            //     loc.androidSaveHint,
//     style: const TextStyle(fontSize: 12, color: Colors.grey),
            //     textAlign: TextAlign.center,
            //   ),
          ],
        ),
        actions: [
          TextButton(
            onPressed: () =>
                Navigator.of(dialogContext).pop(),
            child: Text(loc.cancel),
          ),
          ElevatedButton(
            onPressed: () {
              String name = controller.text.trim();
       // Remove invalid filename characters (Windows, macOS, Linux)
              name = name.replaceAll(
                  RegExp(r'[\\/:*?"<>|]'), '');
              if (!name
                  .toLowerCase()
                  .endsWith('.pdf')) {
                name += '.pdf';
              }
              if (name.isEmpty || name == '.pdf') {
                // Show error (simple way: shake or ignore)
                Navigator.of(dialogContext)
                    .pop(fileName); // fallback
              } else {
                Navigator.of(dialogContext)
                    .pop(name);
              }
            },
            child: Text(loc.save),
```

```
        ),
      ],
    );
  },
);
if (editedFileName == null) return;
fileName = editedFileName;
// Use DocumentFileSavePlus for Android, FilePicker for others
try {
  if (Theme.of(context).platform ==
      TargetPlatform.android) {
    await DocumentFileSavePlus().saveFile(
      Uint8List.fromList(pdfBytes),
      fileName,
      "application/pdf",
    );
    _scaffoldMessengerKey.currentState
        ?.showSnackBar(
      SnackBar(
        content: Text(loc.pdfSaved),
      ),
    );
  } else {
    String? outputPath =
        await FilePicker.platform.saveFile(
      dialogTitle: loc.chooseSaveLocation,
      fileName: fileName,
      type: FileType.custom,
      allowedExtensions: ['pdf'],
    );
    if (outputPath != null) {
      final file = File(outputPath);
      await file.writeAsBytes(pdfBytes);
      _scaffoldMessengerKey.currentState
          ?.showSnackBar(
        SnackBar(
            content: Text(
                '${loc.pdfSavedTo} $outputPath'),
            action: SnackBarAction(
                label: loc.view,
                onPressed: () =>
```

```
                                OpenFile.open(outputPath))),
                  );
                }
              }
            } catch (e) {
              _scaffoldMessengerKey.currentState
                  ?.showSnackBar(
                SnackBar(content: Text('Error: $e')),
              );
            }
          },
        );
      },
    ),
    Builder(
      builder: (context) {
        final loc = AppLocalizations.of(context)!;
        return TextButton.icon(
          icon: const Icon(Icons.save_rounded,
              color: Color(0xFF007BFF)),
          label: Text(loc.save,
              style:
                  const TextStyle(color: Color(0xFF007BFF))),
          style: TextButton.styleFrom(
            shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(18)),
            padding: const EdgeInsets.symmetric(
                horizontal: 22, vertical: 12),
          ),
          onPressed: () {
            Navigator.of(context).pop();
            showDialog(
              context: context,
              builder: (context) {
                final loc = AppLocalizations.of(context)!;
                return AlertDialog(
                  shape: RoundedRectangleBorder(
                      borderRadius:
                          BorderRadius.circular(24)),
                  title: Column(
                    mainAxisSize: MainAxisSize.min,
```

```
                                    children: [
                                      Icon(Icons.check_circle,
                                          color: Color(0xFF43A047), size: 48),
                                      const SizedBox(height: 8),
                                      Text(loc.examCreated),
                                    ],
                                  ),
                                  content: Text(loc.examSaved),
                                  actions: [
                                    TextButton(
                                      onPressed: () {
                                        Navigator.of(context).pop();
                                      },
                                      child: Text(loc.ok),
                                    ),
                                  ],
                                );
                              },
                            );
                          },
                        );
                      },
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      );
    },
  );
}

// ── Existing Exams List ──
Widget _buildExistingExamsList(AppLocalizations loc) {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Padding(
        padding: const EdgeInsets.symmetric(horizontal: 4),
```

```
  child: Row(
    children: [
      const Icon(Icons.folder_open, color: Color(0xFF007BFF), size: 22),
      const SizedBox(width: 8),
      Text('Your Exams (${_existingExams.length})',
          style: const TextStyle(
              fontSize: 18, fontWeight: FontWeight.bold)),
    ],
  ),
),
const SizedBox(height: 12),
..._existingExams.map((exam) => Card(
      shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(16)),
      elevation: 2,
      margin: const EdgeInsets.only(bottom: 10),
      child: Padding(
        padding: const EdgeInsets.all(14),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Container(
                  width: 42,
                  height: 42,
                  decoration: BoxDecoration(
                    color: const Color(0xFF007BFF).withOpacity(0.1),
                    borderRadius: BorderRadius.circular(12),
                  ),
                  child: const Center(
                    child: Icon(Icons.assignment,
                        color: Color(0xFF007BFF), size: 22),
                  ),
                ),
                const SizedBox(width: 12),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(exam.title,
```

```
              style: const TextStyle(
                  fontWeight: FontWeight.bold,
                  fontSize: 15)),
          const SizedBox(height: 2),
          Text(
            '${exam.subject}  •  ${exam.numQuestions} Q  •
                    ${exam.date.split('T').first}',
            style: const TextStyle(
                color: Colors.grey, fontSize: 12),
          ),
        ],
      ),
    ),
  ],
),
const SizedBox(height: 10),
Row(
  children: [
    Expanded(
      child: OutlinedButton.icon(
        onPressed: () async {
          final changed =
              await Navigator.of(context).push<bool>(
            MaterialPageRoute(
              builder: (_) => EditExamScreen(exam: exam),
            ),
          );
          if (changed == true) loadExistingExams();
        },
        icon: const Icon(Icons.edit, size: 16),
        label: const Text('Edit'),
        style: OutlinedButton.styleFrom(
          foregroundColor: Colors.blue,
          side: const BorderSide(color: Colors.blue),
          shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(10)),
          padding: const EdgeInsets.symmetric(vertical: 8),
        ),
      ),
    ),
    const SizedBox(width: 10),
```

```
Expanded(
  child: OutlinedButton.icon(
    onPressed: () async {
      final confirm = await showDialog<bool>(
        context: context,
        builder: (ctx) => AlertDialog(
          title: const Text('Delete Exam'),
          content: Text(
            'Delete "${exam.title}"?\n\n'
       'All questions and student results will be permanently removed.',
          ),
          actions: [
            TextButton(
              onPressed: () =>
                  Navigator.of(ctx).pop(false),
              child: const Text('Cancel'),
            ),
            ElevatedButton(
              onPressed: () =>
                  Navigator.of(ctx).pop(true),
              style: ElevatedButton.styleFrom(
                  backgroundColor: Colors.red),
              child: const Text('Delete'),
            ),
          ],
        ),
      );
      if (confirm == true) {
        final auth = Provider.of<AuthProvider>(context,
            listen: false);
        await DatabaseHelper()
            .deleteExam(exam.id!, auth.userId);
        loadExistingExams();
        if (mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(
              content: Text('Exam deleted'),
              backgroundColor: Colors.red,
            ),
          );
        }
```

```
                            }
                          },
                          icon: const Icon(Icons.delete, size: 16),
                          label: const Text('Delete'),
                          style: OutlinedButton.styleFrom(
                            foregroundColor: Colors.red,
                            side: const BorderSide(color: Colors.red),
                            shape: RoundedRectangleBorder(
                                borderRadius: BorderRadius.circular(10)),
                            padding: const EdgeInsets.symmetric(vertical: 8),
                          ),
                        ),
                      ),
                    ],
                  ),
                ],
              ),
            ),
          )),
      ],
    );
  }
}

class _StepCircle extends StatelessWidget {
  final bool active;
  final String label;
  const _StepCircle({required this.active, required this.label});

  @override
  Widget build(BuildContext context) {
    final loc = AppLocalizations.of(context)!; // Access localization
    // Only use ARB keys that exist. Fallback to label if not found.
    String displayLabel;
    if (label == '1') {
      displayLabel = loc.examTitle; // Use examTitle as step 1 label
    } else if (label == '2') {
      displayLabel = loc.answerKey;
    } else {
      displayLabel = label;
    }
```

```
    return Container(
      width: 32,
      height: 32,
      decoration: BoxDecoration(
        color: active ? const Color(0xFF007BFF) : Colors.grey[300],
        shape: BoxShape.circle,
      ),
      child: Center(
        child: Text(
          displayLabel,
          style: TextStyle(
            color: active ? Colors.white : Colors.black,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    );
  }
}

bool _isDuplicateSubject(String subject) {
  final predefinedSubjects = ['Math', 'Science', 'History', 'English'];
  return predefinedSubjects.contains(subject);
}
```

End create exam screen code

OMR Scan Screen code

```
Import ‘package:file_picker/file_picker.dart’;
Import ‘live_scan_screen.dart’;
Import ‘dart:io’;
Import ‘package:flutter/material.dart’;
Import ‘package:image_picker/image_picker.dart’;
Import ‘package:flutter_gen/gen_l10n/app_localizations.dart’;
Import ‘../utils/omr_pipeline.dart’;
Import ‘package:provider/provider.dart’;
Import ‘../providers/student_provider.dart’;
Import ‘../db/database_helper.dart’;
Import ‘../models/exam.dart’;
Import ‘../models/result.dart’;
Import ‘../models/question.dart’;
```

```
Import ‘dart:convert’;
Import ‘../providers/auth_provider.dart’;
Import ‘../providers/sync_provider.dart’;

Class OmrScanScreen extends StatefulWidget {
  Const OmrScanScreen({super.key});

  @override
  State<OmrScanScreen> createState() => _OmrScanScreenState();
}

Class _OmrScanScreenState extends State<OmrScanScreen> {
  String? _selectedListTitle;
  String? _selectedListSubject;
  String? _scanResult;
  File? _selectedImage;
  String? _tempIdResult;
  List<Exam> _exams = [];
  Exam? _selectedExam;

  @override
  Void initState() {
    Super.initState();
    _loadExams();
  }

  Future<void> _loadExams() async {
    Final auth = Provider.of<AuthProvider>(context, listen: false);
    Final exams = await DatabaseHelper().getAllExams(auth.userId);
    setState(() {
      _exams = exams;
    });
  }

  Future<void> _pickImageFromCamera() async {
    Final picker = ImagePicker();
    Final picked = await picker.pickImage(source: ImageSource.camera);
    If (picked != null) {
      setState(() {
        _selectedImage = File(picked.path);
      });
```

```
  }
}

Future<void> _pickImageFromGallery() async {
  Final picker = ImagePicker();
  Final picked = await picker.pickImage(source: ImageSource.gallery);
  If (picked != null) {
    setState(() {
      _selectedImage = File(picked.path);
    });
  }
}

/// Shows a bottom sheet with existing student lists + an import option.
Future<void> _showStudentListPicker(AppLocalizations loc) async {
  Final provider = Provider.of<StudentProvider>(context, listen: false);
  // Make sure all students are loaded first
  Await provider.load();
  Final groups = provider.getListGroups();

  If (!mounted) return;

  Final result = await showModalBottomSheet<Map<String, String>>(
    Context: context,
    isScrollControlled: true,
    backgroundColor: Colors.transparent,
    builder: (ctx) {
      return Container(
        constraints: BoxConstraints(
          maxHeight: MediaQuery.of(ctx).size.height * 0.65,
        ),
        Decoration: const BoxDecoration(
          Color: Colors.white,
          borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
        ),
        Child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            // Handle bar
            Padding(
              Padding: const EdgeInsets.only(top: 12, bottom: 4),
```

```
  Child: Container(
    Width: 40,
    Height: 4,
    Decoration: BoxDecoration(
      Color: Colors.grey[300],
      borderRadius: BorderRadius.circular(2),
    ),
  ),
),
// Header
Padding(
  Padding: const EdgeInsets.fromLTRB(20, 12, 20, 8),
  Child: Row(
    Children: [
      Container(
        Padding: const EdgeInsets.all(10),
        Decoration: BoxDecoration(
          Color: const Color(0xFF007BFF).withOpacity(0.1),
          Shape: BoxShape.circle,
        ),
        Child: const Icon(Icons.people_alt_rounded,
            Color: Color(0xFF007BFF), size: 24),
      ),
      Const SizedBox(width: 14),
      Expanded(
        Child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(loc.selectStudentList,
                Style: const TextStyle(
                    fontSize: 18, fontWeight: FontWeight.w800)),
            Text(loc.selectOrImportHint,
                Style: TextStyle(
                    Color: Colors.grey[500], fontSize: 12)),
          ],
        ),
      ),
      IconButton(
        Icon: const Icon(Icons.close, size: 20),
        onPressed: () => Navigator.pop(ctx),
      ),
```

```
    ],
  ),
),
Const Divider(height: 1),
// List of existing groups
If (groups.isEmpty)
  Padding(
    Padding: const EdgeInsets.symmetric(vertical: 32),
    Child: Column(
      Children: [
        Icon(Icons.folder_off_outlined,
            Size: 48, color: Colors.grey[300]),
        Const SizedBox(height: 12),
        Text(loc.noStudentsImported,
            Style: TextStyle(
                Color: Colors.grey[500],
                fontWeight: FontWeight.w600)),
        const SizedBox(height: 4),
        Text(loc.importNewListHint,
            Style: TextStyle(
                Color: Colors.grey[400], fontSize: 12)),
      ],
    ),
  )
Else
  Flexible(
    Child: ListView.separated(
      shrinkWrap: true,
      padding: const EdgeInsets.symmetric(
          horizontal: 16, vertical: 12),
      itemCount: groups.length,
      separatorBuilder: (_, __) => const SizedBox(height: 8),
      itemBuilder: (_, i) {
        final g = groups[i];
        final isSelected =
            _selectedListTitle == g[‘title’] &&
                _selectedListSubject == g[‘subject’];
        Return Material(
          Color: isSelected
              ? const Color(0xFF007BFF).withOpacity(0.08)
              : Colors.grey[50],
```

```
borderRadius: BorderRadius.circular(14),
child: InkWell(
  borderRadius: BorderRadius.circular(14),
  onTap: () {
    Navigator.of(ctx).pop({
      'title': g['title'] as String,
      'subject': g['subject'] as String,
    });
  },
  Child: Padding(
    Padding: const EdgeInsets.symmetric(
        Horizontal: 16, vertical: 14),
    Child: Row(
      Children: [
        Container(
          Padding: const EdgeInsets.all(10),
          Decoration: BoxDecoration(
            Color: isSelected
                ? const Color(0xFF007BFF)
                    .withOpacity(0.15)
                : Colors.blueGrey.withOpacity(0.08),
            borderRadius: BorderRadius.circular(12),
          ),
          Child: Icon(Icons.folder_shared,
              Color: isSelected
                  ? const Color(0xFF007BFF)
                  : Colors.blueGrey,
              Size: 22),
        ),
        Const SizedBox(width: 14),
        Expanded(
          Child: Column(
            crossAxisAlignment:
                CrossAxisAlignment.start,
            Children: [
              Text(g['title'] as String,
                  Style: TextStyle(
                      fontWeight: FontWeight.w700,
                      fontSize: 15,
                      color: isSelected
                          ? const Color(0xFF007BFF)
```

```
                                    : Colors.black87)),
                          Const SizedBox(height: 2),
                          Text(
                    '${g['subject']}  •  ${loc.studentsCount(g['count'] as int)}',
                              Style: TextStyle(
                                  Color: Colors.grey[500],
                                  fontSize: 12)),
                        ],
                      ),
                    ),
                    If (isSelected)
                      Const Icon(Icons.check_circle,
                          Color: Color(0xFF007BFF), size: 22)
                    Else
                      Icon(Icons.chevron_right,
                          Color: Colors.grey[400], size: 20),
                  ],
                ),
              ),
            ),
          );
        },
      ),
    ),
// Import new list button
Padding(
  Padding:
      Const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
  Child: SizedBox(
    Width: double.infinity,
    Child: OutlinedButton.icon(
      Icon: const Icon(Icons.add_circle_outline, size: 20),
      Label: Text(loc.importNewList,
          Style: const TextStyle(fontWeight: FontWeight.w700)),
      Style: OutlinedButton.styleFrom(
        foregroundColor: const Color(0xFF43A047),
        side: const BorderSide(color: Color(0xFF43A047)),
        shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(14)),
        padding: const EdgeInsets.symmetric(vertical: 14),
      ),
```

```
                  onPressed: () {
                    Navigator.of(ctx).pop({'action': 'import'});
                  },
                ),
              ),
            ),
            SizedBox(height: MediaQuery.of(ctx).padding.bottom + 8),
          ],
        ),
      );
    },
  );

  If (result == null) return;

  // User chose to import a new list
  If (result.containsKey('action') && result['action'] == 'import') {
    Await _importNewStudentList(loc);
    Return;
  }

  // User selected an existing list
  Final title = result['title']!;
  Final subject = result['subject']!;
  Await provider.loadByGroup(title, subject);
  If (mounted) {
    setState(() {
      _selectedListTitle = title;
      _selectedListSubject = subject;
    });
    ScaffoldMessenger.of(context).showSnackBar(SnackBar(
      Content: Text(loc.studentsLoaded(provider.students.length)),
      Behavior: SnackBarBehavior.floating,
      backgroundColor: const Color(0xFF007BFF),
    ));
  }
}

/// Import a new student list (title/subject dialog + file picker).
Future<void> _importNewStudentList(AppLocalizations loc) async {
```

```
Final meta = await showModalBottomSheet<Map<String, String>>(
  Context: context,
  isScrollControlled: true,
  backgroundColor: Colors.transparent,
  builder: (ctx) {
    final titleController = TextEditingController();
    final subjectController = TextEditingController();
    return Padding(
      padding: EdgeInsets.only(
          bottom: MediaQuery.of(ctx).viewInsets.bottom),
      child: Container(
        decoration: const BoxDecoration(
          color: Colors.white,
          borderRadius:
              BorderRadius.vertical(top: Radius.circular(28)),
        ),
        Padding: const EdgeInsets.fromLTRB(24, 12, 24, 28),
        Child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Container(
              Width: 40,
              Height: 4,
              Decoration: BoxDecoration(
                Color: Colors.grey[300],
                borderRadius: BorderRadius.circular(2),
              ),
            ),
            Const SizedBox(height: 20),
            Row(
              Children: [
                Container(
                  Padding: const EdgeInsets.all(10),
                  Decoration: BoxDecoration(
                    Color: const Color(0xFF43A047).withOpacity(0.1),
                    Shape: BoxShape.circle,
                  ),
                  Child: const Icon(Icons.drive_folder_upload,
                      Color: Color(0xFF43A047), size: 24),
                ),
                Const SizedBox(width: 14),
```

```
    Expanded(
      Child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(loc.newListInfo,
              Style: const TextStyle(
                  fontSize: 18,
                  fontWeight: FontWeight.w800)),
          Text(loc.importDetailsSubtitle,
              Style: TextStyle(
                  Color: Colors.grey[500],
                  fontSize: 12)),
        ],
      ),
    ),
    IconButton(
      Icon: const Icon(Icons.close, size: 20),
      onPressed: () => Navigator.pop(ctx),
    ),
  ],
),
Const SizedBox(height: 24),
TextField(
  Controller: titleController,
  Decoration: InputDecoration(
    labelText: loc.title,
    prefixIcon: const Icon(Icons.title),
    border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12)),
  ),
),
Const SizedBox(height: 14),
TextField(
  Controller: subjectController,
  Decoration: InputDecoration(
    labelText: loc.subject,
    prefixIcon: const Icon(Icons.book_outlined),
    border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12)),
  ),
),
```

```
            Const SizedBox(height: 24),
            SizedBox(
              Width: double.infinity,
              Child: ElevatedButton(
                onPressed: () {
                  final t = titleController.text.trim();
                  final s = subjectController.text.trim();
                  if (t.isNotEmpty && s.isNotEmpty) {
                    Navigator.of(ctx)
                        .pop({‘title’: t, ‘subject’: s});
                  }
                },
                Style: ElevatedButton.styleFrom(
                  backgroundColor: const Color(0xFF43A047),
                  shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(12)),
                  padding:
                      const EdgeInsets.symmetric(vertical: 14),
                ),
                Child: Text(loc.next,
                    Style: const TextStyle(
                        fontWeight: FontWeight.bold)),
              ),
            ),
          ],
        ),
      ),
    );
  },
);

If (meta == null) return;
Final String title = meta[‘title’]!;
Final String subject = meta[‘subject’]!;

Final fileResult = await FilePicker.platform.pickFiles(
  Type: FileType.custom,
  allowedExtensions: [‘csv’, ‘txt’, ‘xls’, ‘xlsx’, ‘pdf’],
  withData: true,
);
If (fileResult == null) return;
```

```
Final file = fileResult.files.single;
Final provider = Provider.of<StudentProvider>(context, listen: false);
Final auth = Provider.of<AuthProvider>(context, listen: false);

showDialog(
  context: context,
  barrierDismissible: false,
  builder: (_) => const Center(child: CircularProgressIndicator()),
);
Int importedCount = 0;
Try {
  If (file.path != null) {
    importedCount = await provider.importFromFileWithMeta(
        File(file.path!), subject, title);
  } else if (file.bytes != null) {
    Final tmp = await Directory.systemTemp.createTemp('omr_students');
    Final f = File('${tmp.path}/${file.name}');
    Await f.writeAsBytes(file.bytes!);
    importedCount = await provider.importFromFileWithMeta(f, subject, title);
  }
  // Automatically select the newly imported list
  Await provider.loadByGroup(title, subject);
  If (!mounted) return;
  setState(() {
    _selectedListTitle = title;
    _selectedListSubject = subject;
  });
  ScaffoldMessenger.of(context).showSnackBar(SnackBar(
    Content: Text(importedCount > 0
      ? loc.studentsLoaded(importedCount)
      : "No students found in the file structure."),
    Behavior: SnackBarBehavior.floating,
    backgroundColor: importedCount > 0 ? const Color(0xFF43A047) : Colors.orange[800],
  ));
} catch € {
  If (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(SnackBar(
        Content: Text(loc.importFailed(e.toString())),
        backgroundColor: Colors.red[700]));
  }
} finally {
```

```
    If (mounted) Navigator.of(context).pop();
    If (mounted) {
      Provider.of<SyncProvider>(context, listen: false)
          .autoSync(auth.userId);
      setState(() {});
    }
  }
}

Future<void> _runOmrScan() async {
  Final loc = AppLocalizations.of(context)!;
  If (_selectedExam == null) {
    ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(loc.selectExamFirst)));
    Return;
  }
  If (_selectedImage == null) {
    setState(() {
      _scanResult = loc.uploadImageFirst;
    });
    Return;
  }
  setState(() {
    _scanResult = null;
    _tempIdResult = null;
  });
  Try {
    Final imagePath = _selectedImage!.path;
    Final omr = OMRScanner();
    Final result = omr.processImage(imagePath);
    Final scannedAnswers = result['answers'] as List<int>;
    Final idDigits = result['id'] as List<int>;
    Final studentIdStr = idDigits.join();

    // Fetch correct answers for this exam
    Final db = DatabaseHelper();
    Final correctQuestions =
        Await db.getQuestionsByExamId(_selectedExam!.id!);

    Int totalMaxMark = 0;
    Int studentMark = 0;
```

```
      List<Map<String, dynamic>> answerData = [];

      For (int I = 0; I < scannedAnswers.length; i++) {
        Final qNum = I + 1;
        // Find matching correct question from DB
        Final correctQ = correctQuestions.firstWhere(
            (q) => q.questionNumber == qNum,
            orElse: () =>
```

End OMR Scan Screen code

Result overview screen code

```
import 'package:flutter/material.dart';
import 'package:omr1/db/database_helper.dart';
import 'package:omr1/models/exam.dart';
import 'dart:convert';
import 'package:flutter_gen/gen_l10n/app_localizations.dart';
import 'package:omr1/screens/exam_results_screen.dart';
import 'package:provider/provider.dart';
import '../providers/auth_provider.dart';

class ResultsOverviewScreen extends StatefulWidget {
  final Function(int)? onTabChange;
  const ResultsOverviewScreen({super.key, this.onTabChange});

  @override
  State<ResultsOverviewScreen> createState() => ResultsOverviewScreenState();
}

class ResultsOverviewScreenState extends State<ResultsOverviewScreen> {
  List<Map<String, dynamic>> _examSummaries = [];
  int _totalExams = 0;
  int _totalSheets = 0;
  double _avgScore = 0;
  bool _isLoading = true;

  @override
  void initState() {
    super.initState();
    loadData();
```

```
}

Future<void> loadData() async {
  final db = DatabaseHelper();
  final auth = Provider.of<AuthProvider>(context, listen: false);
  final exams = await db.getAllExams(auth.userId);
  _totalExams = exams.length;
  _totalSheets = 0;
  double totalScoreSum = 0;
  int totalResultCount = 0;

  List<Map<String, dynamic>> summaries = [];

  for (var exam in exams) {
    final results = await db.getResultsByExamId(exam.id!, auth.userId);
    _totalSheets += results.length;

    for (var r in results) {
      totalScoreSum += r.score;
      totalResultCount++;
    }

    // Use pre-stored student names for each result with a fallback lookup
    List<Map<String, dynamic>> studentResults = [];
    for (var r in results) {
      String displayName = r.studentName;

      // Auto-repair: If the stored name is "Unknown", try finding it now
      if (displayName == 'Unknown Student') {
        final student = await db.getStudentByStudentId(r.studentId, auth.userId);
        if (student != null) {
          displayName = student.name;
        }
      }

      studentResults.add({
        'db_id': r.id, // Needed for deletion
        'id': r.studentId,
        'name': displayName,
        'score': r.score,
        'answers': List<Map<String, dynamic>>.from(jsonDecode(r.answers)),
```

```
      });
    }

    summaries.add({
      'exam': exam,
      'count': results.length,
      'studentResults': studentResults,
    });
  }

  _avgScore = totalResultCount > 0 ? (totalScoreSum / totalResultCount) : 0;

  if (mounted) {
    setState(() {
      _examSummaries = summaries;
      _isLoading = false;
    });
  }
}

@override
Widget build(BuildContext context) {
  final loc = AppLocalizations.of(context)!;
  if (_isLoading) {
    return Scaffold(
      appBar: AppBar(title: Text(loc.resultsOverview)),
      body: const Center(child: CircularProgressIndicator()),
    );
  }
  return PopScope(
    canPop: Navigator.of(context).canPop(),
    onPopInvoked: (didPop) {
      if (didPop) return;
      if (widget.onTabChange != null) {
        widget.onTabChange!(0);
      }
    },
    child: Scaffold(
      backgroundColor: const Color(0xFFF7F8FA),
      appBar: AppBar(
        backgroundColor: Colors.white,
```

```
    elevation: 0,
    iconTheme: const IconThemeData(color: Color(0xFF007BFF)),
    title: Text(loc.resultsOverview,
        style: const TextStyle(
            color: Colors.black, fontWeight: FontWeight.bold)),
    leading: !Navigator.of(context).canPop()
        ? IconButton(
            icon: const Icon(Icons.arrow_back),
            onPressed: () {
              if (widget.onTabChange != null) {
                widget.onTabChange!(0);
              }
            },
          )
        : null,
  ),
body: SafeArea(
  child: ListView(
    padding: const EdgeInsets.all(20),
    children: [
      // Summary Cards
      SizedBox(
        height: 120, // Match card height
        child: SingleChildScrollView(
          scrollDirection: Axis.horizontal,
          child: Row(
            children: [
              _ResultSummaryCard(
                icon: Icons.assignment,
                value: _totalExams.toString(),
                label: loc.totalExams,
                color: const Color(0xFF007BFF),
              ),
              _ResultSummaryCard(
                icon: Icons.document_scanner,
                value: _totalSheets.toString(),
                label: loc.sheets,
                color: const Color(0xFF43A047),
              ),
              _ResultSummaryCard(
                icon: Icons.bar_chart,
```

```
                    value: '${_avgScore.toStringAsFixed(1)}%',
                    label: loc.avgScore,
                    color: const Color(0xFFFB8C00),
                  ),
                ],
              ),
            ),
          ),
          const SizedBox(height: 32),
          Text(loc.recentScans, style: const TextStyle(fontSize: 18, fontWeight:
FontWeight.bold)),
          const SizedBox(height: 12),
          if (_examSummaries.isEmpty)
            Center(child: Padding(
              padding: const EdgeInsets.all(20.0),
              child: Text(loc.noScansYet),
            )),
          ..._examSummaries.map((summary) {
            final Exam exam = summary['exam'];
            return _ResultListTile(
              title: exam.title,
              date: exam.date.split('T').first, // Simple date display
              score: '${summary['count']} ${loc.sheets}',
              onTap: () async {
                await Navigator.of(context).push(
                  MaterialPageRoute(
                    builder: (context) => ExamResultsScreen(
                      examId: exam.id!,
                      examTitle: exam.title,
                      date: exam.date.split('T').first,
                      students: List<Map<String, dynamic>>.from(summary['studentResults']),
                    ),
                  ),
                );
                loadData(); // Refresh overview after returning (in case a result was deleted)
              },
            );
          }).toList(),
        ],
      ),
    ),
```

```
    ),
  );
}
}

class _ResultSummaryCard extends StatelessWidget {
  final IconData icon;
  final String value;
  final String label;
  final Color color;
  const _ResultSummaryCard({required this.icon, required this.value, required this.label,
required this.color});

  @override
  Widget build(BuildContext context) {
    return Container(
      width: 140,
      margin: const EdgeInsets.only(right: 16),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(24),
        boxShadow: [
          BoxShadow(
            color: Colors.black12,
            blurRadius: 10,
            offset: Offset(0, 4),
          ),
        ],
        border: Border(
          left: BorderSide(color: color, width: 6),
        ),
      ),
      padding: const EdgeInsets.symmetric(vertical: 8, horizontal: 10),
      clipBehavior: Clip.hardEdge,
      child: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            Icon(icon, color: color, size: 28),
```

```
            const SizedBox(height: 6),
            Text(value, style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
            const SizedBox(height: 2),
            Text(label, style: TextStyle(fontSize: 12, color: Colors.grey, fontWeight:
FontWeight.w500)),
          ],
        ),
      ),
    );
  }
}

class _ResultListTile extends StatelessWidget {
  final String title;
  final String date;
  final String score;
  final VoidCallback onTap;
  const _ResultListTile({required this.title, required this.date, required this.score, required
this.onTap});

  @override
  Widget build(BuildContext context) {
    return Card(
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
      margin: const EdgeInsets.only(bottom: 10),
      elevation: 2,
      child: ListTile(
        leading: Icon(Icons.description, color: Theme.of(context).primaryColor),
        title: Text(title, style: TextStyle(fontWeight: FontWeight.w600)),
        subtitle: Text(date),
        trailing: Container(
          padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
          decoration: BoxDecoration(
            color: Color(0xFF007BFF).withOpacity(0.1),
            borderRadius: BorderRadius.circular(12),
          ),
          child: Text(score, style: TextStyle(color: Color(0xFF007BFF), fontWeight:
FontWeight.bold)),
        ),
        onTap: onTap,
      ),
```

```
    );
  }
}
```

End result overview screen code

student result screen code

```
import 'package:flutter/material.dart';
import 'package:flutter_gen/gen_l10n/app_localizations.dart';

class StudentResultScreen extends StatelessWidget {
  final String examTitle;
  final String studentId;
  final String studentName;
  final String date;
  final int score;
  final List<Map<String, dynamic>> answers;

  const StudentResultScreen({
    super.key,
    required this.examTitle,
    required this.studentId,
    required this.studentName,
    required this.date,
    required this.score,
    required this.answers,
  });

  @override
  Widget build(BuildContext context) {
    final loc = AppLocalizations.of(context)!; // Access localization
    return Scaffold(
      backgroundColor: const Color(0xFFF7F8FA),
      appBar: AppBar(
        backgroundColor: Colors.white,
        elevation: 0,
        iconTheme: const IconThemeData(color: Color(0xFF007BFF)),
```

```
        title: Text('${loc.result}: $examTitle', style: const TextStyle(color: Colors.black,
fontWeight: FontWeight.bold)), // Localized
      ),
      body: SafeArea(
        child: ListView(
          padding: const EdgeInsets.all(20),
          children: [
            Card(
              shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
              elevation: 4,
              child: Padding(
                padding: const EdgeInsets.all(20),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(studentName, style: const TextStyle(fontSize: 20, fontWeight:
FontWeight.bold)),
                    const SizedBox(height: 6),
                    Text('${loc.studentId}: $studentId', style: const TextStyle(color:
Colors.grey)), // Localized
                    const SizedBox(height: 6),
                    Text('${loc.date}: $date', style: const TextStyle(color: Colors.grey)), //
Localized
                    const SizedBox(height: 12),
                    Row(
                      children: [
                        const Icon(Icons.emoji_events, color: Color(0xFF43A047), size: 32),
                        const SizedBox(width: 10),
                        Builder(
                          builder: (context) {
                            int total = 0;
                            int obtained = 0;
                            for (var a in answers) {
                              int m = a['mark'] ?? 0;
                              total += m;
                              if (a['isCorrect'] == true) obtained += m;
                            }
                            return Text(
                              '${loc.score}: $obtained / $total',
                              style: const TextStyle(fontSize: 22, fontWeight: FontWeight.bold,
color: Color(0xFF43A047)),
```

```
                          );
                        },
                      ),
                    ],
                  ),
                ],
              ),
            ),
            const SizedBox(height: 24),
            Text(loc.answers, style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold)), //
Localized
            const SizedBox(height: 12),
            ...answers.map((a) => _AnswerTile(
              question: a['question'],
              selected: a['selected'],
              correct: a['correct'],
              isCorrect: a['isCorrect'],
            )),
          ],
        ),
      ),
    );
  }
}

class _AnswerTile extends StatelessWidget {
  final int question;
  final String selected;
  final String correct;
  final bool isCorrect;
  const _AnswerTile({required this.question, required this.selected, required this.correct,
required this.isCorrect});

  @override
  Widget build(BuildContext context) {
    final loc = AppLocalizations.of(context)!; // Access localization
    return Card(
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
      margin: const EdgeInsets.only(bottom: 8),
      color: isCorrect ? Color(0xFFE8F5E9) : Color(0xFFFFEBEE),
```

```
      child: ListTile(
        leading: CircleAvatar(
          backgroundColor: isCorrect ? Color(0xFF43A047) : Color(0xFFDC3545),
          child: Icon(isCorrect ? Icons.check : Icons.close, color: Colors.white),
        ),
        title: Text('${loc.question} $question: ${loc.yourAnswer}: $selected'), // Localized
        subtitle: !isCorrect ? Text('${loc.correct}: $correct', style: TextStyle(color:
Color(0xFF007BFF))) : null, // Localized
        trailing: isCorrect
            ? Text(loc.correct, style: TextStyle(color: Color(0xFF43A047), fontWeight:
FontWeight.bold))
            : Text(loc.incorrect, style: TextStyle(color: Color(0xFFDC3545), fontWeight:
FontWeight.bold)),
      ),
    );
  }
}
End student result screen

student group screen

import 'package:flutter/material.dart';
import 'package:flutter_gen/gen_l10n/app_localizations.dart';
import '../models/student.dart';

class StudentGroupScreen extends StatefulWidget {
  final String subject;
  final String title;
  final List<Student> students;

  const StudentGroupScreen({
    super.key,
    required this.subject,
    required this.title,
    required this.students,
  });

  @override
  State<StudentGroupScreen> createState() => _StudentGroupScreenState();
}
```

```
class _StudentGroupScreenState extends State<StudentGroupScreen> {
  late List<Student> _filteredStudents;
  final TextEditingController _searchController = TextEditingController();

  @override
  void initState() {
    super.initState();
    _filteredStudents = widget.students;
    _searchController.addListener(_onSearchChanged);
  }

  @override
  void dispose() {
    _searchController.dispose();
    super.dispose();
  }

  void _onSearchChanged() {
    final query = _searchController.text.toLowerCase();
    setState(() {
      _filteredStudents = widget.students.where((s) {
        return s.name.toLowerCase().contains(query) || s.studentId.contains(query);
      }).toList();
    });
  }

  @override
  Widget build(BuildContext context) {
    final loc = AppLocalizations.of(context)!;
    final theme = Theme.of(context);

    return Scaffold(
      backgroundColor: const Color(0xFFF8F9FB),
      appBar: AppBar(
        title: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(widget.title, style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 18,
color: Colors.black)),
            Text(widget.subject, style: const TextStyle(fontSize: 12, color: Colors.grey)),
          ],
```

```
        ),
        backgroundColor: Colors.white,
        elevation: 0,
        iconTheme: const IconThemeData(color: Colors.black),
      ),
      body: Column(
        children: [
          // Header Stats & Search
          Container(
            color: Colors.white,
            padding: const EdgeInsets.fromLTRB(20, 0, 20, 20),
            child: Column(
              children: [
                const SizedBox(height: 12),
                Container(
                  padding: const EdgeInsets.symmetric(horizontal: 16),
                  decoration: BoxDecoration(
                    color: const Color(0xFFF1F3F4),
                    borderRadius: BorderRadius.circular(12),
                  ),
                  child: TextField(
                    controller: _searchController,
                    decoration: InputDecoration(
                      hintText: loc.search,
                      border: InputBorder.none,
                      prefixIcon: const Icon(Icons.search, color: Colors.grey, size: 20),
                      contentPadding: const EdgeInsets.symmetric(vertical: 12),
                    ),
                  ),
                ),
                const SizedBox(height: 16),
                Row(
                  mainAxisAlignment: MainAxisAlignment.spaceBetween,
                  children: [
                    Text(
                      loc.studentsCount(_filteredStudents.length),
                      style: const TextStyle(fontWeight: FontWeight.bold, color:
Colors.blueGrey),
                    ),
                    const Icon(Icons.sort, color: Colors.grey, size: 20),
                  ],
```

```
              ),
            ],
          ),
        ),

        // Student List
        Expanded(
          child: ListView.builder(
            padding: const EdgeInsets.all(16),
            itemCount: _filteredStudents.length,
            itemBuilder: (context, i) {
              final s = _filteredStudents[i];
              final initial = s.name.isNotEmpty ? s.name[0].toUpperCase() : '?';

              return Container(
                margin: const EdgeInsets.only(bottom: 12),
                decoration: BoxDecoration(
                  color: Colors.white,
                  borderRadius: BorderRadius.circular(16),
                  boxShadow: [BoxShadow(color: Colors.black.withOpacity(0.02), blurRadius: 8,
offset: const Offset(0, 4))],
                ),
                child: ListTile(
                  contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
                  leading: CircleAvatar(
                    backgroundColor: theme.primaryColor.withOpacity(0.1),
                    child: Text(initial, style: TextStyle(color: theme.primaryColor,
fontWeight: FontWeight.bold)),
                  ),
                  title: Text(
                    s.name.isEmpty ? '—' : s.name,
                    style: const TextStyle(fontWeight: FontWeight.w600, fontSize: 15),
                  ),
                  subtitle: Text(
                    'ID: ${s.studentId}',
                    style: TextStyle(color: Colors.grey[600], fontSize: 13),
                  ),
                  trailing: Text(
                    '#${i + 1}',
                    style: TextStyle(color: Colors.grey[300], fontSize: 12, fontWeight:
FontWeight.bold),
```

```
                  ),
                ),
              );
            },
          ),
        ),
      ],
    ),
  );
  }
}End student group screen



edit exam screen

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../db/database_helper.dart';
import '../models/exam.dart';
import '../models/question.dart';
import '../providers/auth_provider.dart';
import '../providers/sync_provider.dart';

class EditExamScreen extends StatefulWidget {
  final Exam exam;
  const EditExamScreen({super.key, required this.exam});

  @override
  State<EditExamScreen> createState() => _EditExamScreenState();
}

class _EditExamScreenState extends State<EditExamScreen> {
  late TextEditingController _titleController;
  late TextEditingController _subjectController;
  List<Question> _questions = [];
  bool _isLoading = true;
  bool _isSaving = false;

  // Editable copies
  late List<int> _correctChoices;
```

```
late List<int> _marks;

@override
void initState() {
  super.initState();
  _titleController = TextEditingController(text: widget.exam.title);
  _subjectController = TextEditingController(text: widget.exam.subject);
  _loadQuestions();
}

@override
void dispose() {
  _titleController.dispose();
  _subjectController.dispose();
  super.dispose();
}

Future<void> _loadQuestions() async {
  final questions =
      await DatabaseHelper().getQuestionsByExamId(widget.exam.id!);
  setState(() {
    _questions = questions;
    _correctChoices = questions.map((q) => q.correctChoice).toList();
    _marks = questions.map((q) => q.mark).toList();
    _isLoading = false;
  });
}

Future<void> _saveChanges() async {
  if (_isSaving) return;
  setState(() => _isSaving = true);

  final db = DatabaseHelper();
  final auth = Provider.of<AuthProvider>(context, listen: false);

  // Update exam title/subject
  final updatedExam = Exam(
    id: widget.exam.id,
    title: _titleController.text.trim(),
    subject: _subjectController.text.trim(),
    date: widget.exam.date,
```

```
    numQuestions: widget.exam.numQuestions,
    numChoices: widget.exam.numChoices,
    answerKey: widget.exam.answerKey,
    userId: widget.exam.userId,
  );
  await db.updateExam(updatedExam);

  // Update each question
  for (int i = 0; i < _questions.length; i++) {
    final q = _questions[i];
    final updated = Question(
      id: q.id,
      examId: q.examId,
      questionNumber: q.questionNumber,
      correctChoice: _correctChoices[i],
      mark: _marks[i],
    );
    await db.updateQuestion(updated);
  }

  // Sync
  Provider.of<SyncProvider>(context, listen: false).autoSync(auth.userId);

  setState(() => _isSaving = false);

  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('✅ Exam updated successfully'),
        backgroundColor: Colors.green,
      ),
    );
    Navigator.of(context).pop(true); // true = changed
  }
}

String _choiceLabel(int index) {
  if (index < 0) return '–';
  return String.fromCharCode(65 + index);
}
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F8FA),
    appBar: AppBar(
      backgroundColor: Colors.white,
      foregroundColor: Colors.black,
      elevation: 0,
      title: const Text('Edit Exam',
          style: TextStyle(fontWeight: FontWeight.bold)),
      actions: [
        TextButton.icon(
          onPressed: _isSaving ? null : _saveChanges,
          icon: _isSaving
              ? const SizedBox(
                  width: 18,
                  height: 18,
                  child: CircularProgressIndicator(strokeWidth: 2),
                )
              : const Icon(Icons.save, color: Colors.blue),
          label: Text(
            _isSaving ? 'Saving...' : 'Save',
            style: const TextStyle(
                color: Colors.blue, fontWeight: FontWeight.bold),
          ),
        ),
      ],
    ),
    body: _isLoading
        ? const Center(child: CircularProgressIndicator())
        : ListView(
            padding: const EdgeInsets.all(16),
            children: [
              // ── Exam Info Card ──
              Card(
                shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(16)),
                elevation: 2,
                child: Padding(
                  padding: const EdgeInsets.all(16),
                  child: Column(
```

```
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      const Text('Exam Details',
                          style: TextStyle(
                              fontSize: 16, fontWeight: FontWeight.bold)),
                      const SizedBox(height: 12),
                      TextField(
                        controller: _titleController,
                        decoration: InputDecoration(
                          labelText: 'Exam Title',
                          border: OutlineInputBorder(
                              borderRadius: BorderRadius.circular(12)),
                          prefixIcon: const Icon(Icons.edit),
                        ),
                      ),
                      const SizedBox(height: 12),
                      TextField(
                        controller: _subjectController,
                        decoration: InputDecoration(
                          labelText: 'Subject',
                          border: OutlineInputBorder(
                              borderRadius: BorderRadius.circular(12)),
                          prefixIcon: const Icon(Icons.subject),
                        ),
                      ),
                      const SizedBox(height: 8),
                      Text(
                        '${widget.exam.numQuestions} questions  •  ${widget.exam.numChoices}
choices  •  ${widget.exam.date}',
                          style:
                              const TextStyle(color: Colors.grey, fontSize: 12),
                      ),
                    ],
                  ),
                ),
              ),

              const SizedBox(height: 16),

              // ── Questions Header ──
              Row(
```

```
  children: [
    const Text('Answer Key',
        style: TextStyle(
            fontSize: 16, fontWeight: FontWeight.bold)),
    const Spacer(),
    Text('${_questions.length} questions',
        style: const TextStyle(color: Colors.grey)),
  ],
),
const SizedBox(height: 8),

// ── Questions List ──
...List.generate(_questions.length, (i) {
  return Card(
    margin: const EdgeInsets.only(bottom: 8),
    shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12)),
    child: Padding(
      padding: const EdgeInsets.symmetric(
          horizontal: 12, vertical: 10),
      child: Row(
        children: [
          // Question number
          Container(
            width: 36,
            height: 36,
            decoration: BoxDecoration(
              color: Colors.blue.withOpacity(0.1),
              borderRadius: BorderRadius.circular(10),
            ),
            child: Center(
              child: Text(
                'Q${_questions[i].questionNumber}',
                style: const TextStyle(
                  fontSize: 12,
                  fontWeight: FontWeight.bold,
                  color: Colors.blue,
                ),
              ),
            ),
          ),
        ),
```

```
const SizedBox(width: 12),

// Correct answer dropdown
Expanded(
  flex: 3,
  child: DropdownButtonFormField<int>(
    value: _correctChoices[i],
    decoration: InputDecoration(
      labelText: 'Answer',
      border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(10)),
      contentPadding: const EdgeInsets.symmetric(
          horizontal: 12, vertical: 8),
      isDense: true,
    ),
    items: List.generate(
      widget.exam.numChoices,
      (c) => DropdownMenuItem(
        value: c,
        child: Text(
          _choiceLabel(c),
          style: const TextStyle(
              fontWeight: FontWeight.w600),
        ),
      ),
    ),
    onChanged: (val) {
      if (val != null) {
        setState(() => _correctChoices[i] = val);
      }
    },
  ),
),
const SizedBox(width: 12),

// Mark input
Expanded(
  flex: 2,
  child: TextFormField(
    initialValue: _marks[i].toString(),
    keyboardType: TextInputType.number,
```

```
                            decoration: InputDecoration(
                              labelText: 'Mark',
                              border: OutlineInputBorder(
                                  borderRadius: BorderRadius.circular(10)),
                              contentPadding: const EdgeInsets.symmetric(
                                  horizontal: 12, vertical: 8),
                              isDense: true,
                            ),
                            onChanged: (val) {
                              final parsed = int.tryParse(val);
                              if (parsed != null && parsed >= 0) {
                                _marks[i] = parsed;
                              }
                            },
                          ),
                        ),
                      ],
                    ),
                  ),
                );
              }),

              const SizedBox(height: 80), // space for FAB
            ],
          ),
    );
  }
}
```

End edit exam screen code

live scan screen code

```
import 'dart:async';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:camera/camera.dart';
import 'package:audioplayers/audioplayers.dart';
import 'package:provider/provider.dart';
```

```
import '../utils/omr_pipeline.dart';
import '../providers/auth_provider.dart';
import '../providers/sync_provider.dart';
import '../db/database_helper.dart';
import '../models/exam.dart';
import '../models/result.dart';
import '../models/question.dart';
import 'dart:convert';

class LiveScanScreen extends StatefulWidget {
  final Exam exam;
  const LiveScanScreen({super.key, required this.exam});

  @override
  State<LiveScanScreen> createState() => _LiveScanScreenState();
}

class _LiveScanScreenState extends State<LiveScanScreen>
    with TickerProviderStateMixin {
  // Camera
  CameraController? _cameraController;
  bool _isCameraReady = false;

  // Scanning loop
  bool _isProcessing = false;
  Timer? _scanTimer;
  String _statusText = 'Initializing...';

  // Confirmation: require same ID twice to prevent misreads
  String? _pendingId;
  int _confirmCount = 0;
  static const int _requiredConfirmations = 2;
  String? _lastScannedId; // cooldown: skip if same sheet still present

  // Feedback
  final AudioPlayer _audioPlayer = AudioPlayer();
  late AnimationController _flashController;
  late Animation<double> _flashAnimation;

  // Results
  int _scannedCount = 0;
```

```
final List<Map<String, dynamic>> _sessionResults = [];

// Torch
bool _isTorchOn = false;
// Focus point visual
Offset? _focusPoint;

@override
void initState() {
  super.initState();
  _flashController = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 800),
  );
  _flashAnimation = CurvedAnimation(
    parent: _flashController,
    curve: Curves.easeOut,
  );
  _initCamera();
}

@override
void dispose() {
  _scanTimer?.cancel();
  _cameraController?.dispose();
  _flashController.dispose();
  _audioPlayer.dispose();
  super.dispose();
}

Future<void> _initCamera() async {
  try {
    final cameras = await availableCameras();
    if (cameras.isEmpty) return;

    final camera = cameras.firstWhere(
      (c) => c.lensDirection == CameraLensDirection.back,
      orElse: () => cameras.first,
    );

    // 720p — best balance of focus speed and detail for OMR
```

```
    _cameraController = CameraController(
      camera,
      ResolutionPreset.high,
      enableAudio: false,
      imageFormatGroup: ImageFormatGroup.jpeg,
    );

    await _cameraController!.initialize();

    // Do NOT call setFocusMode — Android uses CONTINUOUS_VIDEO by default
    try {
      await _cameraController!.setFlashMode(FlashMode.off);
    } catch (_) {}

    if (!mounted) return;
    setState(() {
      _isCameraReady = true;
      _statusText = 'Camera ready...';
    });

    // Warm-up: let continuous AF lock before first capture
    await Future.delayed(const Duration(seconds: 2));
    if (mounted) {
      setState(() => _statusText = 'Place paper under camera');
      _startScanLoop();
    }
  } catch (e) {
    if (mounted) setState(() => _statusText = 'Camera error: $e');
  }
}

void _startScanLoop() {
  _scanTimer = Timer.periodic(const Duration(milliseconds: 2500), (_) {
    if (!_isProcessing && _isCameraReady && mounted) {
      _captureAndProcess();
    }
  });
}

Future<void> _onTapFocus(TapDownDetails details) async {
  if (!_isCameraReady || _cameraController == null) return;
```

```
  try {
    final RenderBox box = context.findRenderObject() as RenderBox;
    final Offset local = box.globalToLocal(details.globalPosition);
    final double dx = (local.dx / box.size.width).clamp(0.0, 1.0);
    final double dy = (local.dy / box.size.height).clamp(0.0, 1.0);
    setState(() => _focusPoint = local);
    await _cameraController!.setFocusPoint(Offset(dx, dy));
    await _cameraController!.setExposurePoint(Offset(dx, dy));
    await _cameraController!.setFocusMode(FocusMode.auto);
    Future.delayed(const Duration(milliseconds: 1500), () {
      if (mounted) setState(() => _focusPoint = null);
    });
  } catch (_) {}
}

Future<void> _captureAndProcess() async {
  if (!_isCameraReady || _cameraController == null || _isProcessing) return;
  setState(() {
    _isProcessing = true;
    _statusText = '🔍 Scanning...';
  });

  XFile? photo;
  try {
    photo = await _cameraController!.takePicture();
  } catch (_) {
    if (mounted)
      setState(() {
        _isProcessing = false;
        _statusText = 'Place paper under camera';
      });
    return;
  }

  try {
    // Use the exact same pipeline as "take pic" — no rotation
    final Map<String, dynamic> result = await compute(
      processImageInIsolate,
      photo.path,
    );
```

```
final scannedAnswers = result['answers'] as List<int>;
final idDigits = result['id'] as List<int>;
final studentIdStr = idDigits.join();

// Skip cooldown: same sheet still under camera
if (studentIdStr == _lastScannedId) {
  if (mounted)
    setState(() {
      _isProcessing = false;
      _statusText = '✅ Done! Remove paper for next';
    });
  try {
    await File(photo.path).delete();
  } catch (_) {}
  return;
}

// Confirmation: require 2 consecutive same reads
if (studentIdStr == _pendingId) {
  _confirmCount++;
} else {
  _pendingId = studentIdStr;
  _confirmCount = 1;
}

if (_confirmCount < _requiredConfirmations) {
  if (mounted)
    setState(() {
      _isProcessing = false;
      _statusText = '🔍 Hold still — confirming...';
    });
  try {
    await File(photo.path).delete();
  } catch (_) {}
  return;
}

// ── Confirmed! Grade the paper ──
final db = DatabaseHelper();
final correctQuestions = await db.getQuestionsByExamId(widget.exam.id!);
```

```
int totalMaxMark = 0;
int studentMark = 0;
final List<Map<String, dynamic>> answerData = [];

for (int i = 0; i < scannedAnswers.length; i++) {
  final qNum = i + 1;
  final correctQ = correctQuestions.firstWhere(
    (q) => q.questionNumber == qNum,
    orElse: () => Question(
        examId: -1, questionNumber: -1, correctChoice: -1, mark: 0),
  );

  String sel = 'Blank';
  if (scannedAnswers[i] >= 0)
    sel = String.fromCharCode(65 + scannedAnswers[i]);
  else if (scannedAnswers[i] == -2) sel = 'Multi';

  String cor = 'N/A';
  if (correctQ.correctChoice >= 0)
    cor = String.fromCharCode(65 + correctQ.correctChoice);

  final isCorrect = (scannedAnswers[i] == correctQ.correctChoice) &&
      (scannedAnswers[i] >= 0);
  if (correctQ.id != null && correctQ.id != -1) {
    totalMaxMark += correctQ.mark;
    if (isCorrect) studentMark += correctQ.mark;
  }
  answerData.add({
    'question': qNum,
    'selected': sel,
    'correct': cor,
    'isCorrect': isCorrect,
    'mark': correctQ.mark,
  });
}

final scorePercent =
    totalMaxMark > 0 ? (studentMark / totalMaxMark * 100).round() : 0;

final auth = Provider.of<AuthProvider>(context, listen: false);
final student = await db.getStudentByStudentId(studentIdStr, auth.userId);
```

```
final studentName = student?.name ?? 'Unknown ($studentIdStr)';

await db.insertResult(Result(
  examId: widget.exam.id!,
  studentId: studentIdStr,
  studentName: studentName,
  score: scorePercent,
  answers: jsonEncode(answerData),
  date: DateTime.now().toIso8601String(),
  userId: auth.userId,
));

if (mounted) {
  Provider.of<SyncProvider>(context, listen: false).autoSync(auth.userId);
}

try {
  await _audioPlayer.play(AssetSource('sounds/beep.wav'));
} catch (_) {}
_flashController.forward(from: 0.0);

_lastScannedId = studentIdStr;
_pendingId = null;
_confirmCount = 0;

if (mounted) {
  setState(() {
    _isProcessing = false;  // ← unlock the scan loop
    _scannedCount++;
    _sessionResults.insert(0, {
      'studentName': studentName,
      'studentId': studentIdStr,
      'score': scorePercent,
      'totalMark': totalMaxMark,
      'studentMark': studentMark,
    });
    _statusText =
        '✅ $studentName — $studentMark/$totalMaxMark   (remove paper)';
  });
}
```

```
    // Reset cooldown after 3s
    Future.delayed(const Duration(seconds: 3), () {
      if (mounted)
        setState(() {
          _lastScannedId = null;
          _statusText = 'Place next paper under camera';
        });
    });
  } catch (_) {
    _pendingId = null;
    _confirmCount = 0;
    if (mounted)
      setState(() {
        _isProcessing = false;
        _statusText = 'Place paper under camera';
      });
  }

  try {
    await File(photo.path).delete();
  } catch (_) {}
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.black,
    body: Stack(
      children: [
        // ── Full-screen camera preview ──
        if (_isCameraReady && _cameraController != null)
          Positioned.fill(
            child: GestureDetector(
              onTapDown: _onTapFocus,
              child: Center(
                child: AspectRatio(
                  aspectRatio: 1 / _cameraController!.value.aspectRatio,
                  child: CameraPreview(_cameraController!),
                ),
              ),
            ),
```

```
  )
else
  const Positioned.fill(
    child: Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          CircularProgressIndicator(color: Colors.white),
          SizedBox(height: 16),
          Text('Initializing camera...',
              style: TextStyle(color: Colors.white70, fontSize: 16)),
        ],
      ),
    ),
  ),

// ── Focus ring ──
if (_focusPoint != null)
  Positioned(
    left: _focusPoint!.dx - 35,
    top: _focusPoint!.dy - 35,
    child: Container(
      width: 70,
      height: 70,
      decoration: BoxDecoration(
        border: Border.all(color: Colors.amber, width: 2),
        shape: BoxShape.circle,
      ),
      child: const Center(
        child: Icon(Icons.center_focus_strong,
            color: Colors.amber, size: 20),
      ),
    ),
  ),

// ── Top bar ──
Positioned(
  top: 0,
  left: 0,
  right: 0,
  child: SafeArea(
```

```
child: Container(
  padding:
      const EdgeInsets.symmetric(horizontal: 16, vertical: 10),
  decoration: const BoxDecoration(
    gradient: LinearGradient(
      begin: Alignment.topCenter,
      end: Alignment.bottomCenter,
      colors: [Colors.black87, Colors.transparent],
    ),
  ),
  child: Row(
    children: [
      IconButton(
        icon: const Icon(Icons.arrow_back, color: Colors.white),
        onPressed: () => Navigator.of(context).pop(),
      ),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text('Live Scan',
                style: TextStyle(
                    color: Colors.white,
                    fontSize: 16,
                    fontWeight: FontWeight.bold)),
            Text(widget.exam.title,
                style: const TextStyle(
                    color: Colors.white60, fontSize: 12)),
          ],
        ),
      ),
      IconButton(
        icon: Icon(_isTorchOn ? Icons.flash_on : Icons.flash_off,
            color: _isTorchOn ? Colors.amber : Colors.white),
        onPressed: () async {
          setState(() => _isTorchOn = !_isTorchOn);
          try {
            await _cameraController?.setFlashMode(
                _isTorchOn ? FlashMode.torch : FlashMode.off);
          } catch (_) {}
        },
```

```
          ),
          Container(
            padding: const EdgeInsets.symmetric(
                horizontal: 12, vertical: 6),
            decoration: BoxDecoration(
              color:
                  _scannedCount > 0 ? Colors.green : Colors.white24,
              borderRadius: BorderRadius.circular(20),
            ),
            child: Text(
              '$_scannedCount ✓',
              style: const TextStyle(
                  color: Colors.white,
                  fontWeight: FontWeight.bold,
                  fontSize: 13),
            ),
          ),
        ],
      ),
    ),
  ),
),

// ── Corner alignment guide ──
Center(
  child: SizedBox(
    width: MediaQuery.of(context).size.width * 0.85,
    height: MediaQuery.of(context).size.height * 0.65,
    child: CustomPaint(
      painter: _CornerPainter(
        color: _lastScannedId != null ? Colors.green : Colors.white,
      ),
    ),
  ),
),

// ── Bottom status / result ──
Positioned(
  bottom: 0,
  left: 0,
  right: 0,
```

```
child: SafeArea(
  child: Container(
    decoration: const BoxDecoration(
      gradient: LinearGradient(
        begin: Alignment.bottomCenter,
        end: Alignment.topCenter,
        colors: [Colors.black87, Colors.transparent],
      ),
    ),
    padding: const EdgeInsets.fromLTRB(20, 30, 20, 16),
    child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        // Status text
        AnimatedSwitcher(
          duration: const Duration(milliseconds: 300),
          child: Text(
            _statusText,
            key: ValueKey(_statusText),
            textAlign: TextAlign.center,
            style: TextStyle(
              color: _statusText.startsWith('✅')
                  ? Colors.greenAccent
                  : _statusText.startsWith('🔍')
                      ? Colors.amberAccent
                      : Colors.white,
              fontSize: 15,
              fontWeight: FontWeight.w600,
              shadows: const [
                Shadow(blurRadius: 8, color: Colors.black)
              ],
            ),
          ),
        ),
        const SizedBox(height: 6),
        const Text(
          'Tap screen to focus  •  Flash icon for light',
          style: TextStyle(color: Colors.white38, fontSize: 11),
        ),
        // Last result card
        if (_sessionResults.isNotEmpty) ...[
```

```
                    const SizedBox(height: 10),
                    Container(
                      width: double.infinity,
                      padding: const EdgeInsets.all(12),
                      decoration: BoxDecoration(
                        color: Colors.black54,
                        borderRadius: BorderRadius.circular(12),
                        border: Border.all(
                          color: (_sessionResults.first['score'] as int) >= 50
                              ? Colors.green.withOpacity(0.5)
                              : Colors.red.withOpacity(0.5),
                        ),
                      ),
                      child: Row(
                        children: [
                          CircleAvatar(
                            radius: 22,
                            backgroundColor:
                                (_sessionResults.first['score'] as int) >= 50
                                    ? Colors.green.withOpacity(0.2)
                                    : Colors.red.withOpacity(0.2),
                            child: Text(

'${_sessionResults.first['studentMark']}/${_sessionResults.first['totalMark']}',
                              textAlign: TextAlign.center,
                              style: TextStyle(
                                fontSize: 10,
                                fontWeight: FontWeight.bold,
                                color:
                                    (_sessionResults.first['score'] as int) >=
                                            50
                                        ? Colors.greenAccent
                                        : Colors.redAccent,
                              ),
                            ),
                          ),
                          const SizedBox(width: 12),
                          Expanded(
                            child: Column(
                              crossAxisAlignment: CrossAxisAlignment.start,
                              children: [
```

```
                                Text(
                                  _sessionResults.first['studentName']
                                      as String,
                                  style: const TextStyle(
                                      color: Colors.white,
                                      fontWeight: FontWeight.bold),
                                ),
                                Text(
                                  'ID: ${_sessionResults.first['studentId']}  •  '

'${_sessionResults.first['studentMark']}/${_sessionResults.first['totalMark']} marks',
                                  style: const TextStyle(
                                      color: Colors.white54, fontSize: 12),
                                ),
                              ],
                            ),
                          ),
                        ),
                      ],
                    ),
                  ),
                ],
              ),
            ),
          ),
        ),

        // ── Processing spinner ──
        if (_isProcessing)
          Positioned(
            top: 90,
            left: 0,
            right: 0,
            child: Center(
              child: Container(
                padding:
                    const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
                decoration: BoxDecoration(
                  color: Colors.black54,
                  borderRadius: BorderRadius.circular(30),
                ),
```

```
                child: const Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    SizedBox(
                      width: 16,
                      height: 16,
                      child: CircularProgressIndicator(
                          color: Colors.white, strokeWidth: 2),
                    ),
                    SizedBox(width: 10),
                    Text('Processing...',
                        style: TextStyle(color: Colors.white, fontSize: 13)),
                  ],
                ),
              ),
            ),
          ),

          // ── Green flash on success ──
          IgnorePointer(
            child: FadeTransition(
              opacity: _flashAnimation.drive(Tween(begin: 0.0, end: 0.5)
                  .chain(CurveTween(curve: Curves.easeOut))),
              child: Container(color: Colors.green),
            ),
          ),
        ],
      ),
    );
  }
}

class _CornerPainter extends CustomPainter {
  final Color color;
  _CornerPainter({required this.color});

  @override
  void paint(Canvas canvas, Size size) {
    final paint = Paint()
      ..color = color.withOpacity(0.85)
      ..strokeWidth = 4
```

```
      ..style = PaintingStyle.stroke
      ..strokeCap = StrokeCap.round;

    const len = 40.0;
    final w = size.width;
    final h = size.height;

    canvas.drawLine(const Offset(0, len), const Offset(0, 0), paint);
    canvas.drawLine(const Offset(0, 0), Offset(len, 0), paint);
    canvas.drawLine(Offset(w - len, 0), Offset(w, 0), paint);
    canvas.drawLine(Offset(w, 0), Offset(w, len), paint);
    canvas.drawLine(Offset(0, h - len), Offset(0, h), paint);
    canvas.drawLine(Offset(0, h), Offset(len, h), paint);
    canvas.drawLine(Offset(w - len, h), Offset(w, h), paint);
    canvas.drawLine(Offset(w, h), Offset(w, h - len), paint);
  }

  @override
  bool shouldRepaint(_CornerPainter old) => old.color != color;
}
End live scan screen



omr pipeline
import 'package:opencv_dart/opencv_dart.dart';

class OMRScanner {
  final int questions = 5;
  final int choices = 5;
  final List<int> answers = [1, 2, 0, 2, 4];
  final int widthImg = 700;
  final int heightImg = 700;

  /// **** Equivalent of splitBoxes in Python ***
  List<Mat> splitBoxes(Mat image) {
    List<Mat> boxes = [];
    int boxHeight = (image.rows / questions).floor();
    int boxWidth = (image.cols / choices).floor();
    for (int r = 0; r < questions; r++) {
      for (int c = 0; c < choices; c++) {
        int startY = r * boxHeight;
```

```
      int startX = c * boxWidth;
      int endY = (r == questions - 1) ? image.rows : (startY + boxHeight);
      int endX = (c == choices - 1) ? image.cols : (startX + boxWidth);
      int w = endX - startX;
      int h = endY - startY;
      double cx = startX + w / 2.0;
      double cy = startY + h / 2.0;
      final box = getRectSubPix(image, (w, h), Point2f(cx, cy));
      boxes.add(box);
    }
  }
  return boxes;
}

/// Count non-zero pixels like cv2.countNonZero
int countNonZeroBox(Mat image) {
  // Use a robust center square crop instead of unavailable bitwise_and/circle
  int w = image.cols;
  int h = image.rows;
  int cropSize = ((w < h ? w : h) * 0.6).toInt();
  double cx = w / 2.0;
  double cy = h / 2.0;
  // Use getRectSubPix to extract a center square region
  final centerBox =
      getRectSubPix(image, (cropSize, cropSize), Point2f(cx, cy));
  return countNonZero(centerBox);
}

VecPoint reorder(VecPoint points) {
  if (points.length != 4) return points;
  List<Point> pts = points.toList();

  List<int> sums = pts.map((p) => p.x + p.y).toList();
  List<int> diffs = pts.map((p) => p.y - p.x).toList();

  int tl_idx = 0;
  int minSum = sums[0];
  int br_idx = 0;
  int maxSum = sums[0];
  int tr_idx = 0;
  int minDiff = diffs[0];
```

```
  int bl_idx = 0;
  int maxDiff = diffs[0];

  for (int i = 1; i < 4; i++) {
    if (sums[i] < minSum) {
      tl_idx = i;
      minSum = sums[i];
    }
    if (sums[i] > maxSum) {
      br_idx = i;
      maxSum = sums[i];
    }
    if (diffs[i] < minDiff) {
      tr_idx = i;
      minDiff = diffs[i];
    }
    if (diffs[i] > maxDiff) {
      bl_idx = i;
      maxDiff = diffs[i];
    }
  }

  return VecPoint.fromList([
    pts[tl_idx],
    pts[tr_idx],
    pts[br_idx],
    pts[bl_idx],
  ]);
}

List<VecPoint> rectContour(VecVecPoint contours) {
  List<VecPoint> rectCon = [];
  for (var i = 0; i < contours.length; i++) {
    final cont = contours[i];
    final area = contourArea(cont);
    if (area > 1000) {
      final peri = arcLength(cont, true);
      final approx = approxPolyDP(cont, 0.02 * peri, true);
      if (approx.length == 4) {
        rectCon.add(approx);
      }
```

```
    }
  }
  rectCon.sort((a, b) => contourArea(b).compareTo(contourArea(a)));
  return rectCon;
}

/// Crop the document using 4 Page markers
List<int> clusterCoordinates(List<int> vals,
    {int dist = 15, int minBubbles = 3}) {
  if (vals.isEmpty) return [];
  vals.sort();
  List<List<int>> groups = [];
  for (int v in vals) {
    if (groups.isEmpty || (v - groups.last.last) > dist) {
      groups.add([v]);
    } else {
      groups.last.add(v);
    }
  }
  // Filter for "Significant Clusters" (clusters that have enough bubbles to be a real line)
  List<List<int>> significant =
      groups.where((g) => g.length >= minBubbles).toList();

  // Fallback: If no significant clusters found, return all groups to avoid total failure
  if (significant.isEmpty) significant = groups;

  return significant.map((g) {
    int sum = g.reduce((a, b) => a + b);
    return (sum / g.length).round();
  }).toList()
    ..sort();
}
/// Process image and extract both ID and answers dynamically
Map<String, dynamic> processImage(String imagePath) {
  // 1. Load and preprocess image
  final rawImg = imread(imagePath);
  // Scale image proportionally for robust feature detection
  double scale = 1500.0 / rawImg.rows;
  int newWidth = (rawImg.cols * scale).toInt();
  final img = resize(rawImg, (newWidth, 1500));
  final imgGray = cvtColor(img, 6); // 6 = COLOR_BGR2GRAY
```

```
final imgBlur = gaussianBlur(imgGray, (5, 5), 1);
final imgThresh = adaptiveThreshold(imgBlur, 255, 0, 1, 11, 2);

// Apply morphological closing to heal shadows/glare on the frame lines
final kernel = getStructuringElement(0, (5, 5));
final imgClosed = morphologyEx(imgThresh, 3, kernel);

// 2. Locate the ID and Answer Framing boxes
final contoursTuple = findContours(imgClosed, 3, 2);
final contours = contoursTuple.$1;

List<Map<String, dynamic>> distFrames = [];
for (var cont in contours) {
  final area = contourArea(cont);
  if (area > 15000) {
    final peri = arcLength(cont, true);
    final approx = approxPolyDP(cont, 0.02 * peri, true);
    if (approx.length == 4) {
      final rect = boundingRect(approx);
      bool overlap = false;
      for (var db in distFrames) {
        if ((rect.x - (db['x'] as int)).abs() < 50 &&
            (rect.y - (db['y'] as int)).abs() < 50) {
          overlap = true;
          if (area > (db['area'] as double)) {
            db['x'] = rect.x;
            db['y'] = rect.y;
            db['w'] = rect.width;
            db['h'] = rect.height;
            db['area'] = area;
            db['cnt'] = approx;
          }
          break;
        }
      }
      if (!overlap) {
        distFrames.add({
          'x': rect.x,
          'y': rect.y,
          'w': rect.width,
          'h': rect.height,
```

```
          'area': area,
          'cnt': approx
        });
      }
    }
  }
}
if (distFrames.isEmpty)
  throw Exception(
      'No thick framing boxes found on the page at all. Ensure paper is well lit.');

Map<String, dynamic>? idFrame;
Map<String, dynamic>? ansFrame;

for (var f in distFrames) {
  double aspect = f['w'] / f['h'];
  if (aspect > 1.2 && aspect < 4.0) {
    if (ansFrame == null || f['area'] > ansFrame['area']) ansFrame = f;
  } else if (aspect > 0.3 && aspect <= 1.2) {
    if (idFrame == null || f['area'] > idFrame['area']) idFrame = f;
  }
}

if (ansFrame == null || idFrame == null) {
  String debug = distFrames.map((f) => '${f['w']}x${f['h']}').join(', ');
  throw Exception('Could not distinguish boxes. Found: $debug');
}

// 3. Perspective Warp and Fixed Coordinate Read
List<int> solveSection(Map<String, dynamic> frame, bool isId) {
  final Size targetSize = isId ? Size(400, 600) : Size(800, 500);
  final VecPoint src = reorder(frame['cnt'] as VecPoint);
  final VecPoint dst = VecPoint.fromList([
    Point(0, 0),
    Point(targetSize.width.toInt(), 0),
    Point(targetSize.width.toInt(), targetSize.height.toInt()),
    Point(0, targetSize.height.toInt()),
  ]);

  final mat = getPerspectiveTransform(src, dst);
  final warpedColor = warpPerspective(
```

```
    img, mat, (targetSize.width.toInt(), targetSize.height.toInt()));
final warpedGray = cvtColor(warpedColor, 6);
final warpedThreshTuple =
    threshold(warpedGray, 0, 255, 8 + 1); // OTSU + BINARY_INV
final Mat warpedThresh = warpedThreshTuple.$2;

// 1. Detect all bubbles in the warped section
final warpedCntsTuple = findContours(warpedThresh, 3, 2);
final warpedCnts = warpedCntsTuple.$1;
List<Point> seeds = [];
for (var c in warpedCnts) {
  final r = boundingRect(c);
  final a = contourArea(c);
  if (r.width / r.height > 0.6 &&
      r.width / r.height < 1.4 &&
      a > 100 &&
      a < 1500) {
    seeds.add(
        Point((r.x + r.width / 2).toInt(), (r.y + r.height / 2).toInt()));
  }
}

// 2. Discover the Grid via significant clusters (Autonomous)
// ID uses min 5 bubbles per column, Answers uses min 4
List<int> rowLattice = clusterCoordinates(seeds.map((p) => p.y).toList(),
    minBubbles: isId ? 3 : 4);
List<int> colLattice = clusterCoordinates(seeds.map((p) => p.x).toList(),
    minBubbles: isId ? 5 : 5);

// Fallback for ID: If 10 rows not found, use historical defaults for your specific form
if (isId && rowLattice.length != 10)
  rowLattice = [110, 160, 210, 260, 310, 360, 410, 460, 510, 560];

List<int> results = [];
const int bS = 12; // Sampling Box size
const int minFilledPixels = 150;

if (isId) {
  for (int cx in colLattice) {
    int det = -1;
    int mVal = 0;
```

```
    for (int r = 0; r < rowLattice.length; r++) {
      final b = warpedThresh
          .region(Rect(cx - bS, rowLattice[r] - bS, bS * 2, bS * 2));
      int val = countNonZero(b);
      if (val > mVal && val > minFilledPixels) {
        mVal = val;
        det = r;
      }
    }
    results.add(det);
  }
} else {
  // ANS: Dynamically group columns into choice blocks using gap analysis
  if (colLattice.length < 2) return [];

  // Find median gap to distinguish choices vs gutters
  List<int> gaps = [];
  for (int i = 0; i < colLattice.length - 1; i++)
    gaps.add(colLattice[i + 1] - colLattice[i]);
  gaps.sort();
  int medianGap = gaps[gaps.length ~/ 2];

  List<List<int>> blocks = [];
  List<int> curr = [colLattice[0]];
  for (int i = 0; i < colLattice.length - 1; i++) {
    if (colLattice[i + 1] - colLattice[i] < medianGap * 1.8) {
      curr.add(colLattice[i + 1]);
    } else {
      blocks.add(curr);
      curr = [colLattice[i + 1]];
    }
  }
  blocks.add(curr);

  // Filter out blocks that represent question numbers (usually small counts)
  final choiceBlocks = blocks.where((b) => b.length >= 2).toList();

  for (var b in choiceBlocks) {
    for (int ry in rowLattice) {
      // Check if this specific row-block intersection actually contains bubbles
      // (Distinguishes between a blank question and a non-existent one)
```

```
bool rowExistsInBlock = seeds.any((s) =>
    (s.y - ry).abs() < 15 &&
    s.x >= b.first - 20 &&
    s.x <= b.last + 20);
if (!rowExistsInBlock) continue;

// Collect pixel counts for all choices in this row
List<int> vals = [];
for (int c = 0; c < b.length; c++) {
  final box =
      warpedThresh.region(Rect(b[c] - bS, ry - bS, bS * 2, bS * 2));
  vals.add(countNonZero(box));
}

// Find the darkest bubble (highest pixel count)
int maxVal = 0;
int maxIdx = -1;
for (int c = 0; c < vals.length; c++) {
  if (vals[c] > maxVal) {
    maxVal = vals[c];
    maxIdx = c;
  }
}

if (maxVal < minFilledPixels) {
  // Nothing filled at all
  results.add(-1);
} else {
  // Check if a second bubble is genuinely close to the darkest
  // (≥75% of max AND above threshold) → true Multi selection
  int closeCount = 0;
  for (int c = 0; c < vals.length; c++) {
    if (vals[c] > maxVal * 0.75 && vals[c] > minFilledPixels) {
      closeCount++;
    }
  }
  if (closeCount >= b.length) {
    // ALL choices look the same → just printed outlines, nothing selected
    results.add(-1);
  } else if (closeCount > 1) {
    // 2+ bubbles are dark but not all → genuine Multi
```

```
                results.add(-2);
              } else {
                // Only the darkest one stands out → single answer
                results.add(maxIdx);
              }     }     }     }     }
      return results;
    }

    final idDigitsResult = solveSection(idFrame, true);
    final answers = solveSection(ansFrame, false);

    return {
      'id': idDigitsResult,
      'answers': answers,
    };
  }
}

/// Top-level function for use with compute().
/// Runs the exact same OMR pipeline on a background isolate to avoid UI freezing.
Map<String, dynamic> processImageInIsolate(String imagePath) {
  final scanner = OMRScanner();
  return scanner.processImage(imagePath);
}
```

End omr pipeline